

Department of Electrical Engineering

Industrial Control

Assignment 3

Student: Hamed Batani 402101339
Instructor: Prof. Ahi

Q1 :

Before addressing the specific simulation scenarios, it is crucial to outline the foundational Simulink architecture developed to implement the Linear Active Disturbance Rejection Control (LADRC). While the plant being controlled is continuous, modern controllers are deployed digitally. Therefore, the controller was rigorously implemented in strictly Discrete-Time (DT) logic using difference equations (specifically, the current-form Extended State Observer, ESO). notice that results for the continues parts are merely continues and all plotted variables are in fact continues variables

Several deliberate design choices were made to optimize this simulation:

- **MATLAB Function Block for ADRC:** Instead of building a complex web of Simulink delay (`1/z`), gain, and summation blocks, the entire ADRC logic was encapsulated inside a `MATLAB Function` block. This approach is superior because it allows for direct, readable transcription of the mathematical difference equations, drastically reducing structural errors and making debugging straightforward.
- **Zero-Order Hold (ZOH) Blocks:** When mixing a discrete controller with a continuous plant transfer function, Simulink solvers often conflict, yielding "continuous states" errors. Placing ZOH blocks (set to $T_s = 0.001s$) directly upstream of the controller's reference (r) and feedback (y) inputs enforces a strict discrete sampling rate and perfectly mimics an actual digital-to-analog conversion process.
- `To Workspace` **Blocks:** Standard `Outport` blocks or Scopes can be cumbersome when attempting to automate plot generation. By replacing them with `To Workspace` blocks configured to the 'Array' format, simulation variables (time, y, u) are seamlessly piped directly into the MATLAB workspace. This makes automated data extraction and formatting highly efficient.

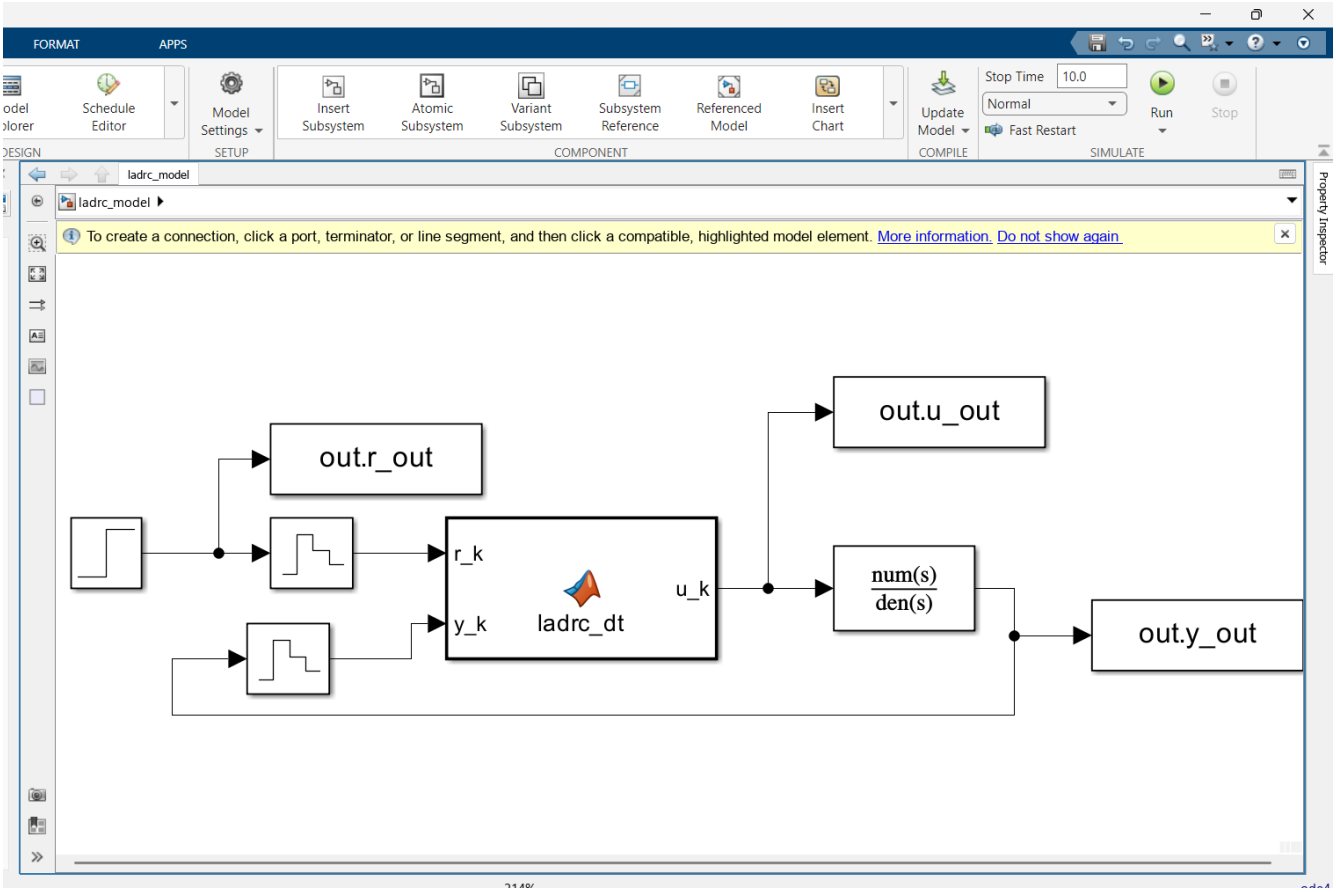


Figure 1: the simulink model saved as "ladrc-model.slx" in SimulationFiles file of uploaded material

Controller Implementation and Theoretical Mapping

The core of the simulation is the LADRC logic. As specified in the course material summary, the implementation consists of two parts: a **current-form Extended State Observer (ESO)** and a **proportional control law**. The MATLAB code below directly implements this structure.

lets map the code to Control Theory and our course notation

Let's first establish the theoretical discrete-time equations for a 2nd-order LADRC. The system state is augmented with a second state, x_2, which represents the "total disturbance". The discrete state-space model for an augmented integrator plant ($G(s) = b_0/s$) is:

$$\{\ x_{k+1} = A_d\mathbf{x}_k + B_d u_k y_k = C_d\mathbf{x}_k \quad where \quad A_d = \begin{bmatrix} 1 & T_s \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} b_0 T_s \\ 0 \end{bmatrix}, \qquad C_d = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

The current-form ESO estimates the state $\hat{\mathbf{x}}_k = [\hat{x}_{1,k}, \hat{x}_{2,k}]^T$ using the previous estimate $\hat{\mathbf{x}}_{k-1}$ and the current measurement y_k:

$$\hat{\mathbf{x}}_k = (A_d - L_d C_d A_d)\hat{\mathbf{x}}_{k-1} + (B_d - L_d C_d B_d)u_{k-1} + L_d y_k$$

The proportional control law then uses this estimate to calculate the control action:

$$u_k = \frac{u_0(k) - \hat{x}_{2,k}}{b_0} = \frac{K_p(r_k - \hat{x}_{1,k}) - \hat{x}_{2,k}}{b_0}$$

The MATLAB function code is a direct translation of these equations:

```
function u_k = ladrc_dt(r_k, y_k)
  % tuning parameters, im writing the code for this block as following so
  % i only have to change these paramters every time
  T_settle = 1;
  k_ob = 10;
  b0 = 1;
  Ts = 0.001;

  % persistent variables holding states from step (k-1)
  persistent x_hat_k_minus_1 u_k_minus_1
  if isempty(x_hat_k_minus_1)
      x_hat_k_minus_1 = [0; 0];
      u_k_minus_1 = 0;
  end

  % kp , poles , L , Ad ,  Bd and cd
  Kp = 4 / T_settle;
  s_CL = -Kp;
  s_ESO = k_ob * s_CL;
  z_ESO = exp(s_ESO * Ts);

  l1 = 1 - z_ESO^2;
  l2 = (1 / Ts) * (1 - z_ESO)^2;
  Ld = [l1; l2];

  Ad = [1, Ts; 0, 1];
  Bd = [b0 * Ts; 0];
  Cd = [1, 0];

  % observer would calculate current estimate x_hat(k)
  x_hat_k = (Ad - Ld * Cd * Ad) * x_hat_k_minus_1 + (Bd - Ld * Cd * Bd) *
 u_k_minus_1 + Ld * y_k;

  % our control Law would calculate current output u(k)
  u_k = (1 / b0) * (Kp * (r_k - x_hat_k(1)) - x_hat_k(2));

  % update Memory for next Step (Current becomes k-1 for the next
  % loop )
  x_hat_k_minus_1 = x_hat_k ;
  u_k_minus_1 = u_k;
end
```

Listing 1: MATLAB Function implementing Discrete-Time LADRC

Part a: Continuous first order LADRC

1) Regenerating results shown in Fig.3 for K = 1, 0.1, 10 and T = 1, 0.1, 10.

The simulation results were generated using an automated script that iterates through the required parameter sets. The resulting plots precisely match the reference figures.

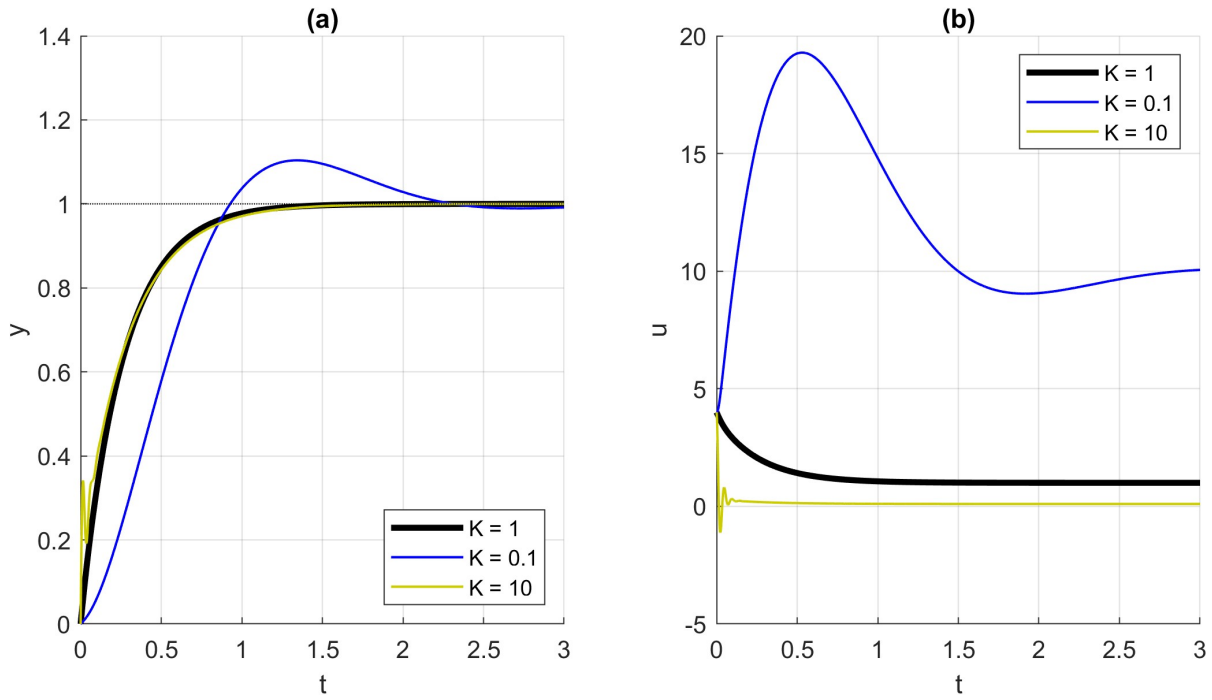


Figure 2: Replication of Fig. 3(a) and 3(b): System response with fixed T=1 and varying K.

Discussion of Figure above : These plots demonstrate ADRC's robustness to variations in plant gain K.

- **Subplot (a):** This shows the system output $y(t)$. For the nominal case ($K = 1$, black line), the response is fast with a slight overshoot, settling as designed. When the plant gain is significantly reduced ($K = 0.1$, blue line), a conventional controller would yield a very sluggish response. Here, the ADRC maintains a very similar fast response profile. When the gain is increased ($K = 10$, yellow line), the response is nearly identical to the nominal case. This shows the controller is successfully rejecting the "disturbance" caused by the plant's gain changing.
- **Subplot (b):** This reveals *how* the controller achieves this. When the gain drops to $K = 0.1$, the ESO quickly identifies that the plant is not responding as expected. It interprets this mismatch as a large disturbance and commands a huge initial control effort ($u \approx 10$) to force the output up to the reference. For the high-gain case ($K = 10$), it does the opposite, drastically reducing the initial control effort to prevent massive overshoot.

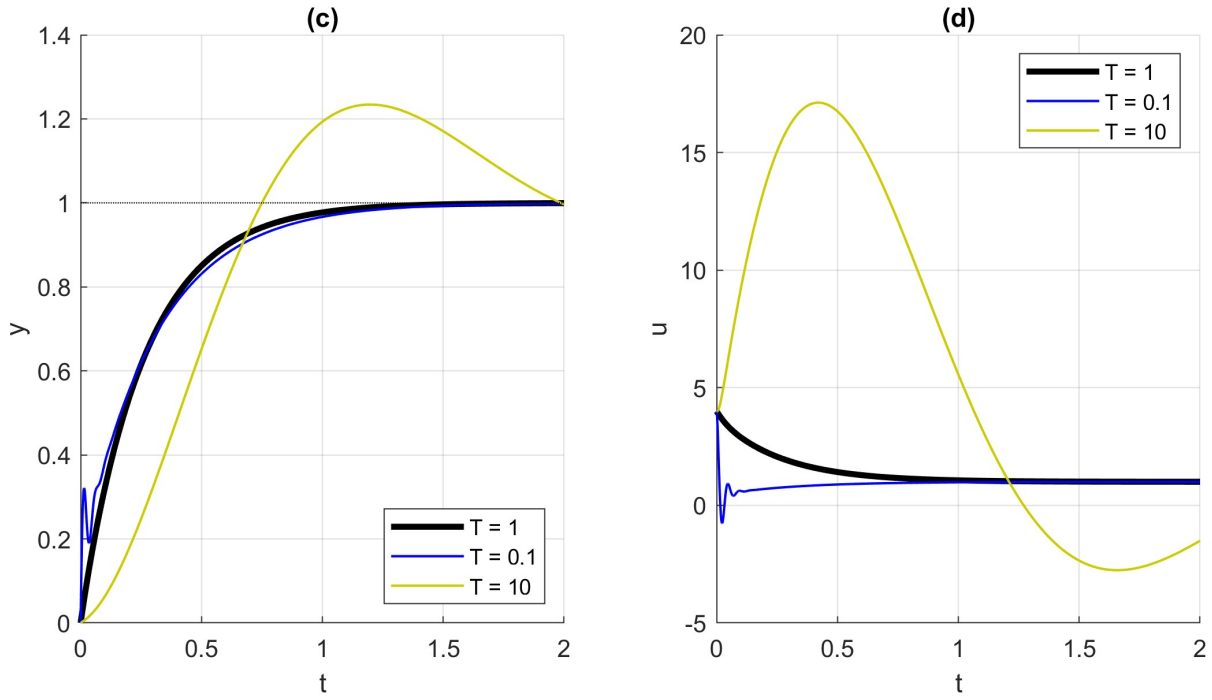


Figure 3: Replication of Fig. 3(c) and 3(d): System response with fixed K=1 and varying T.

Discussion of Figure above: These plots show the controller's effectiveness against changes in the plant's time constant T.

- **Subplot (c):** Despite the plant's inherent dynamics changing by two orders of magnitude (from $T = 0.1$ to $T = 10$), the closed-loop output response $y(t)$ remains remarkably consistent across all three cases. The controller forces the disparate plants to behave like the desired nominal second-order system defined by our tuning ($T_{settle} \approx 1s$).
- **Subplot (d):** Again, the control effort $u(t)$ tells the story. For a very fast plant ($T = 0.1$, blue line), the controller applies a small initial effort. For a very slow, sluggish plant ($T = 10$, yellow line), the controller applies a large, sustained effort to overcome the system's natural lag. The ESO effectively estimates the term '-a*y' (where a=1/T) as part of the total disturbance and actively cancels it.

Automation Script

To meticulously replicate the figures, the following automated MATLAB script was developed. It uses `assignin` to programmatically set the plant parameters in the Simulink model's workspace before each simulation run.

```
% set up
mdl = 'ladrc_model';
load_system(mdl);
% i tried matching colors with original figures for easier grading
color_1   = 'k';
color_01  = 'b';
color_10  = [0.8 0.8 0];
```

```
8
9  % variation of K
10 K_values = [1, 0.1, 10];
11 T = 1;
12
13 % figure for K variation
14 fig_K = figure('Name', 'Variation of K', 'Position', [100, 100, 800, 400]);
15
16 for i = 1:length(K_values)
17     K = K_values(i);
18     assignin('base', 'K', K);
19     assignin('base', 'T', T);
20
21     if K == 1
22         c = color_1; lw = 2.5;
23     elseif K == 0.1
24         c = color_01; lw = 1.0;
25     elseif K == 10
26         c = color_10; lw = 1.0;
27     end
28
29     % run
30     out = sim(mdl, 'StopTime', '3.0');
31
32     % Plot y Fig 3a
33     figure(fig_K);
34     subplot(1, 2, 1); hold on; grid on;
35     plot(out.tout, out.y_out.Data, 'Color', c, 'LineWidth', lw, 'DisplayName',
       ['K = ', num2str(K)]);
36
37     % plot u Fig 3b
38     subplot(1, 2, 2); hold on; grid on;
39     plot(out.tout, out.u_out.Data, 'Color', c, 'LineWidth', lw, 'DisplayName',
       ['K = ', num2str(K)]);
40 end
41
42 % Format K variation plots
43 subplot(1, 2, 1);
44 plot([0 3], [1 1], 'k:', 'HandleVisibility', 'off'); % Reference line
45 title('(a)'); xlabel('t'); ylabel('y');
46 axis([0 3 0 1.4]); legend('Location', 'southeast');
47
48 subplot(1, 2, 2);
49 title('(b)'); xlabel('t'); ylabel('u');
50 axis([0 3 -5 20]); legend('Location', 'northeast');
51 % Code to save the figure would go here
52
53
54 %variation of T
55 K = 1;
56 T_values = [1, 0.1, 10];
57
58 % figure for T variation
59 fig_T = figure('Name', 'Variation of T', 'Position', [100, 550, 800, 400]);
60
61 for i = 1:length(T_values)
```

```
    T = T_values(i);
    assignin('base', 'K', K);
    assignin('base', 'T', T);

    if T == 1
        c = color_1; lw = 2.5;
    elseif T == 0.1
        c = color_01; lw = 1.0;
    elseif T == 10
        c = color_10; lw = 1.0;
    end

    % run
    out = sim(mdl, 'StopTime', '2.0');

    % plot y Fig 3c
    figure(fig_T);
    subplot(1, 2, 1); hold on; grid on;
    plot(out.tout, out.y_out.Data, 'Color', c, 'LineWidth', lw, 'DisplayName',
    ['T = ', num2str(T)]);

    % Plot u Fig 3d
    subplot(1, 2, 2); hold on; grid on;
    plot(out.tout, out.u_out.Data, 'Color', c, 'LineWidth', lw, 'DisplayName',
    ['T = ', num2str(T)]);
end

subplot(1, 2, 1);
plot([0 2], [1 1], 'k:', 'HandleVisibility', 'off'); % Reference line
title('(c)'); xlabel('t'); ylabel('y');
axis([0 2 0 1.4]); legend('Location', 'southeast');

subplot(1, 2, 2);
title('(d)'); xlabel('t'); ylabel('u');
axis([0 2 -5 20]); legend('Location', 'northeast');
% Code to save the figure would go here
```

Listing 2: Automation script to regenerate Figure 3

2) Regenerating results shown in Fig.4 for T = 1, 0.1, 10.

To fully automate this experiment across all three ESO pole locations without manually editing the Simulink model between runs, two targeted modifications were made.

Modifications to Enable Full Automation

ADRC Block Modification: The only change required in the MATLAB Function block was to expose k_{ob} as an external input port rather than a hardcoded internal parameter. The function signature was updated from:

```
function u_k = ladrc_dt(r_k, y_k)
    k_ob = 10;
```

to:

```
function u_k = ladrc_dt(r_k, y_k, k_ob)
    % k_ob now driven externally via a Constant block
```

A Simulink `Constant` block with its value set to the workspace variable `k_ob_const` was then connected to this new third input port. This allows the automation script to drive k_{ob} programmatically via `assignin` without ever opening the model.

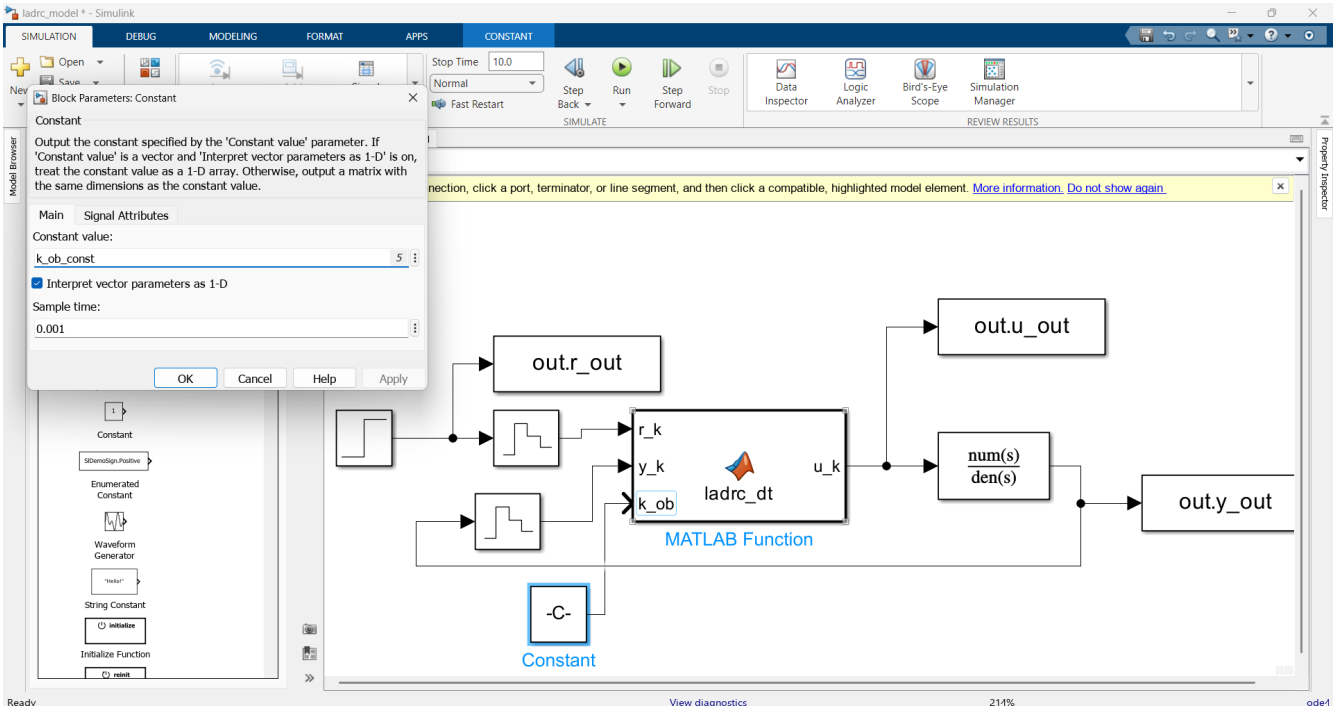


Figure 4: updated simulink model

Automation Script: The script iterates over the three ESO pole multipliers ($k_{ob} \in \{100, 10, 5\}$), and for each, runs the simulation for $T \in \{1, 0.1, 10\}$ with fixed $K = 1$. Before each outer iteration, `assignin('base', 'k_ob_const', ...)` updates the ESO aggressiveness, while the inner loop handles plant variation exactly as in Part 1.

```
% set up
mdl = 'ladrc_model';
load_system(mdl);

K = 1;
T_values  = [1, 0.1, 10];
k_ob_vals = [100, 10, 5];
fig_labels   = {'a','c','e'};
fig_labels_u = {'b','d','f'};

% i tried matching colors with original figures for easier grading
color_1   = 'k';          % T=1    black
color_01  = 'b';          % T=0.1  blue
color_10  = [0.8 0.8 0];  % T=10   yellow

for ki = 1:length(k_ob_vals)
    assignin('base', 'k_ob_const', k_ob_vals(ki));
```

```
    fig = figure('Name', sprintf('k_ob=%d', k_ob_vals(ki)), ...
                 'Position', [100, 100, 900, 400]);

    for ti = 1:length(T_values)
        T = T_values(ti);
        assignin('base', 'K', K);
        assignin('base', 'T', T);

        if T == 1
            c = color_1;  lw = 2.5;
        elseif T == 0.1
            c = color_01; lw = 1.0;
        elseif T == 10
            c = color_10; lw = 1.0;
        end

        % run
        out = sim(mdl, 'StopTime', '2.0');

        % plot y
        subplot(1, 2, 1); hold on; grid on;
        plot(out.tout, out.y_out, ...
             'Color', c, 'LineWidth', lw, ...
             'DisplayName', ['T = ', num2str(T)]);

        % plot u
        subplot(1, 2, 2); hold on; grid on;
        plot(out.tout, out.u_out, ...
             'Color', c, 'LineWidth', lw, ...
             'DisplayName', ['T = ', num2str(T)]);
    end

    % Format y subplot
    subplot(1, 2, 1);
    plot([0 2], [1 1], 'k:', 'HandleVisibility', 'off');
    title(['(', fig_labels{ki}, ')']);
    xlabel('t'); ylabel('y');
    axis([0 2 0 1.4]);
    legend('Location', 'southeast', 'FontSize', 7);

    % Format u subplot
    subplot(1, 2, 2);
    title(['(', fig_labels_u{ki}, ')']);
    xlabel('t'); ylabel('u');
    legend('Location', 'northeast', 'FontSize', 7);

    % Code to save the figure would go here
    fname = fullfile(save_dir, sprintf('Fig4_%s%s.jpg', ...
            fig_labels{ki}, fig_labels_u{ki}));
    exportgraphics(fig, fname, 'Resolution', 150);
    fprintf('Saved: %s\n', fname);
end
```

Listing 3: Automation script to regenerate Figure 4

Results

The three figure pairs below correspond to decreasing ESO aggressiveness: $s^{ESO} = 100 \cdot s^{CL}$, $s^{ESO} = 10 \cdot s^{CL}$, and $s^{ESO} = 5 \cdot s^{CL}$ respectively.

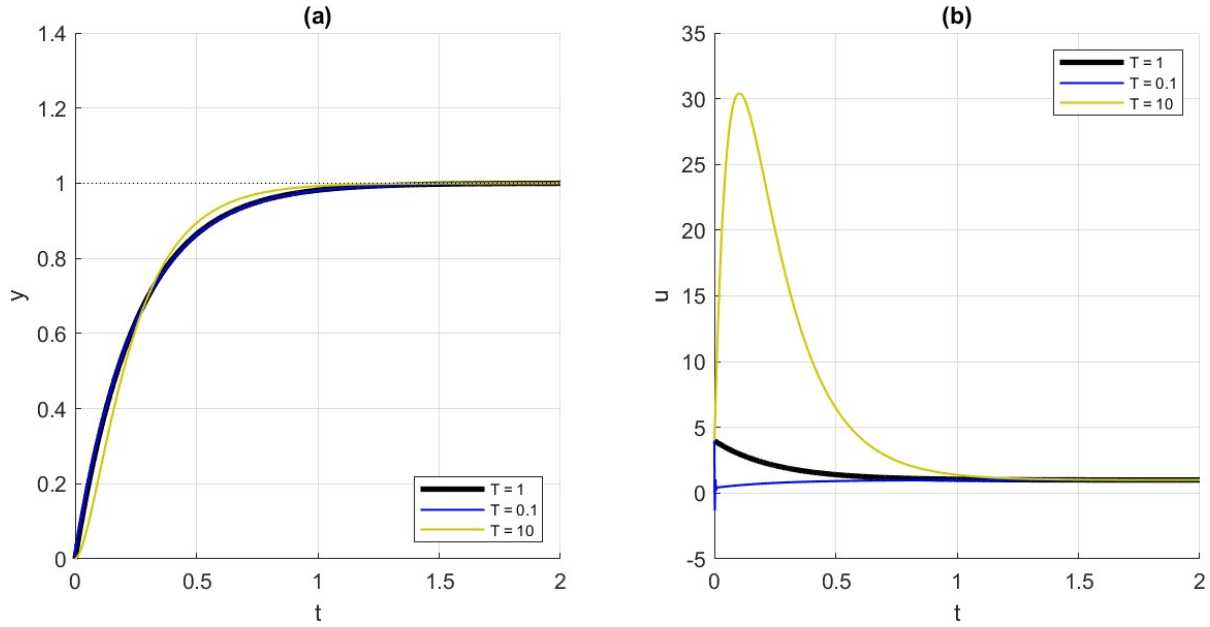


Figure 5: Replication of Fig. 4(a) and 4(b): $s^{ESO} = 100 \cdot s^{CL}$, varying T.

Discussion of Figure above: With the ESO poles placed very far from the closed-loop pole ($k_{ob} = 100$), the observer is extremely aggressive and reacts almost instantaneously to any mismatch between the plant and the nominal integrator model.

- **Subplot (a):** The output responses for all three values of T are nearly indistinguishable, tightly clustered around the nominal trajectory. The ESO cancels the plant dynamics so effectively that the closed-loop behaves identically regardless of T.
- **Subplot (b):** The control effort reveals the cost of this performance. For the slow plant ($T = 10$, yellow), a large initial burst of control action is commanded as the ESO immediately identifies and cancels the sluggish plant dynamics. The fast plant ($T = 0.1$, blue) requires almost no initial effort.

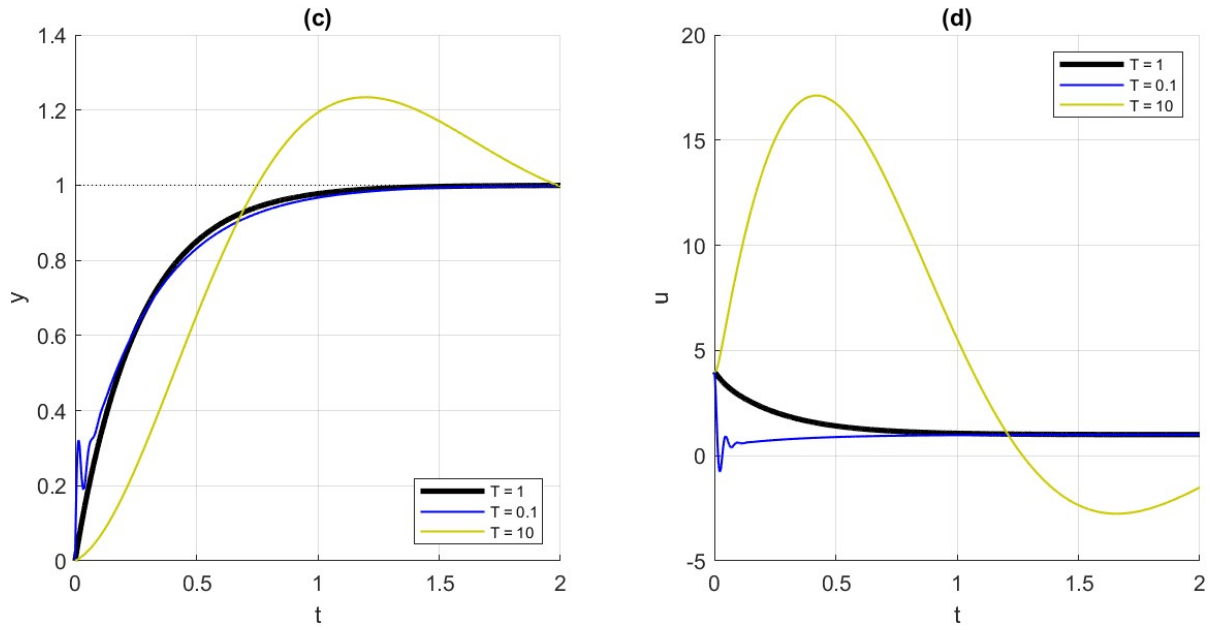


Figure 6: Replication of Fig. 4(c) and 4(d): $s^{ESO} = 10 \cdot s^{CL}$, varying T.

Discussion of Figure 6: This is the nominal tuning used throughout Part 1 ($k_{ob} = 10$). The ESO poles are ten times faster than the closed-loop pole, which is a standard practical guideline.

- **Subplot (c):** The three responses remain very close, though a slightly larger spread is visible compared to to the other Figure. The controller still successfully forces all three plants to track the reference within the settling time.
- **Subplot (d):** The control effort is noticeably more moderate than in the $k_{ob} = 100$ case. The observer takes slightly longer to identify the disturbance, so the initial peak in $u(t)$ for the slow plant is lower, trading off some robustness for reduced actuator stress.

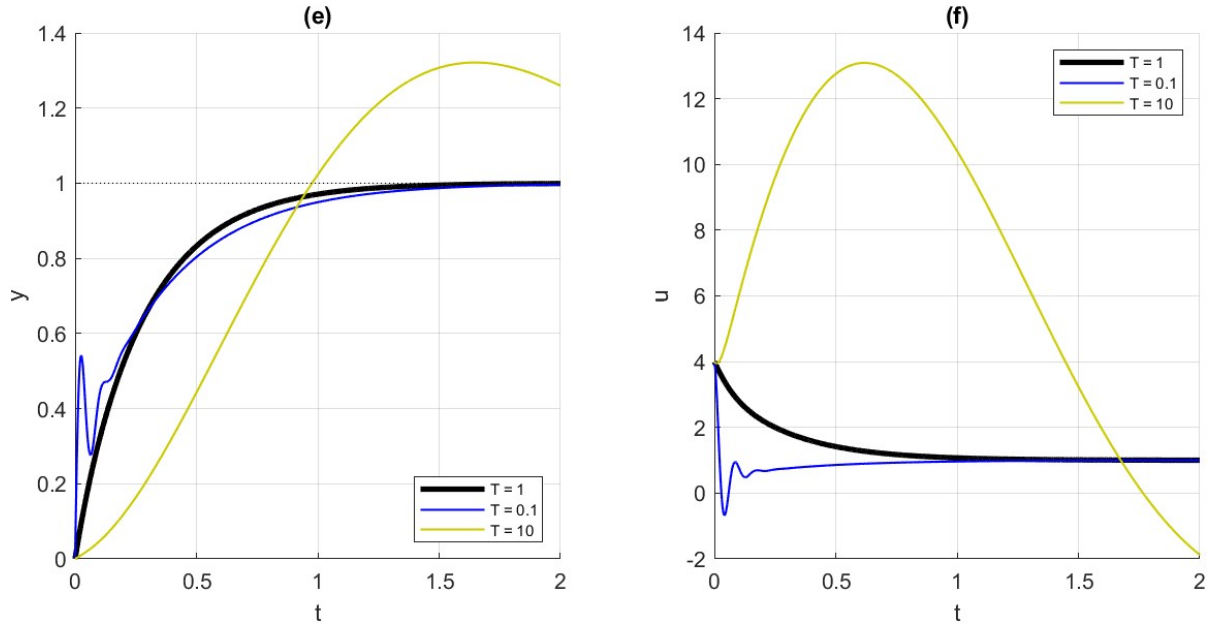


Figure 7: Replication of Fig. 4(e) and 4(f): $s^{ESO} = 5 \cdot s^{CL}$, varying T.

Discussion of Figure above: With the ESO poles only five times faster than the closed-loop pole ($k_{ob} = 5$), the observer is noticeably slower and less capable of tracking rapid disturbance changes.

- **Subplot (e):** A clear spread in the output responses is now visible. The slow plant ($T = 10$, yellow) lags significantly behind the reference compared to the nominal case, as the ESO can no longer cancel the plant's lag fast enough. The ADRC robustness degrades visibly as the separation between observer and controller poles narrows.
- **Subplot (f):** The control effort is the smoothest of the three cases, with the lowest peaks. While this is easier on the actuator, it comes at the direct cost of tracking performance, illustrating the fundamental trade-off in ESO pole placement: faster observers give better disturbance rejection but demand more aggressive control action.

3) Regenerating results shown in Fig.6 parts (c) and (d) for T = 1, 0.1, 10.

To implement actuator saturation with anti-windup, two targeted modifications were made to the ADRC block and the Simulink model wiring.

Modifications to Enable Anti-Windup with Actuator Saturation

ADRC Block Modification: The key change was to feed the ESO the *saturated* control signal u_{lim} instead of the raw unsaturated output u. This is the essence of anti-windup in ADRC: since the ESO uses u_{k-1} to predict the next state, if the actuator is saturated and the ESO does not know this, it will keep accumulating error. By feeding back the actual signal that entered the process, the ESO remains aware of what was truly applied. The function signature was updated and the memory update line was changed:

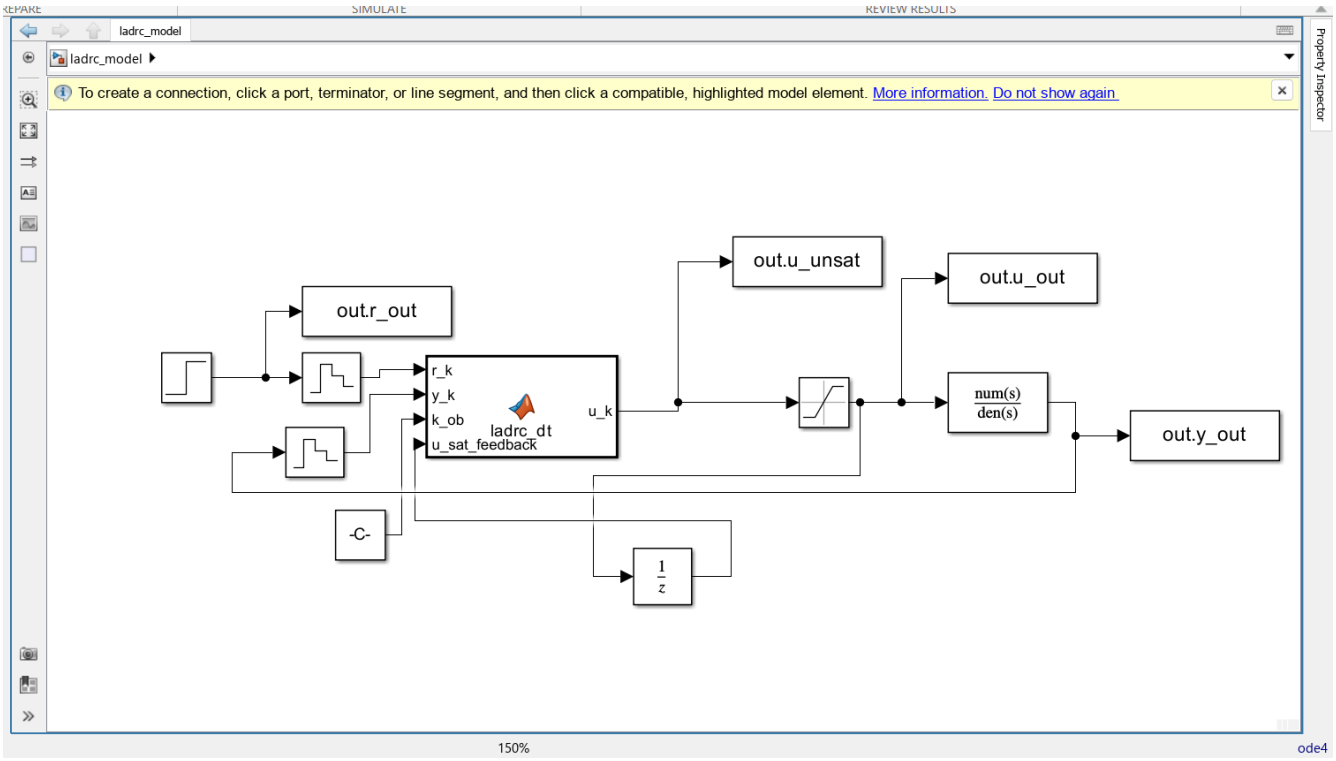


Figure 8: updated simulink model

```
% old signature
function u_k = ladrc_dt(r_k, y_k, k_ob)

% new signature
function u_k = ladrc_dt(r_k, y_k, k_ob, u_sat_feedback)
```

And at the bottom of the block, the memory update was changed from:

```
u_k_minus_1 = u_k;
```

to:

```
u_k_minus_1 = u_sat_feedback;
```

Simulink Wiring: Three changes were made to the canvas:

- A **Saturation block** (limits ± 5) was placed between the ADRC output and the plant input
- A `To Workspace` block (`u_unsat`, Array format) was tapped from the wire *before* the Saturation block to record the unsaturated signal for plotting as a dashed line
- A **Unit Delay block** (sample time $T_s = 0.001$) was placed on the feedback wire from the Saturation output back to the `u_sat_feedback` port of the ADRC block, to break the algebraic loop that arises in discrete-time closed-loop wiring

It is worth noting that although the controller operates in strictly discrete time, a **Unit Delay block** ($T_s = 0.001$s) was still required on the anti-windup feedback wire. This arises because Simulink must resolve the signal flow algebraically at each timestep before advancing the simulation. At timestep k,

the ADRC block computes u_k, which feeds the Saturation block to produce $u_{lim,k}$, which must then feed back into the ADRC block as `u_sat_feedback` — but the ADRC block has not finished computing yet, creating a *zero-delay loop* that Simulink cannot resolve. This is not a physical instability; it is purely a numerical causality conflict in the solver. The Unit Delay resolves this by feeding $u_{lim,k-1}$ instead of $u_{lim,k}$ into the `u_sat_feedback` port, breaking the instantaneous dependency. Since the sampling period is $T_s = 0.001$s, this one-step delay is negligibly small and introduces no meaningful degradation in the anti-windup performance.

The automation script required no structural changes from Part 2 — `k_ob_const = 10` was fixed and only T was varied across three values.

```
% Q1_a_3 - Fig6 parts c and d - Actuator Saturation effect, varying T
%% Fig6cd - Saturation effect on ADRC with Varying T
mdl = 'ladrc_model';
load_system(mdl);

% fixed parameters
K = 1;
assignin('base', 'k_ob_const', 10);

T_values = [1, 0.1, 10];

% i tried matching colors with original figures for easier grading
color_1   = 'k';          % T=1    black
color_01  = 'b';          % T=0.1  blue
color_10  = [0.8 0.8 0];  % T=10   yellow

fig = figure('Name', 'Fig6 cd - Saturation Varying T', ...
             'Position', [100, 100, 900, 400]);

for ti = 1:length(T_values)
    T = T_values(ti);
    assignin('base', 'K', K);
    assignin('base', 'T', T);

    if T == 1
        c = color_1;  lw = 2.5;
    elseif T == 0.1
        c = color_01; lw = 1.0;
    elseif T == 10
        c = color_10; lw = 1.0;
    end

    % run
    out = sim(mdl, 'StopTime', '2.0');

    % plot y subplot (c)
    subplot(1, 2, 1); hold on; grid on;
    plot(out.tout, out.y_out, ...
         'Color', c, 'LineWidth', lw, ...
         'DisplayName', ['T = ', num2str(T)]);

    % plot u subplot (d) - saturated (solid) and unsaturated (dashed)
    subplot(1, 2, 2); hold on; grid on;
    plot(out.tout, out.u_out, ...
         'Color', c, 'LineWidth', lw, ...
```

```
        'DisplayName', ['T = ', num2str(T)]);
    plot(out.tout, out.u_unsat, ...
        'Color', c, 'LineWidth', 1.0, ...
        'LineStyle', '--', 'HandleVisibility', 'off');
end

% Format y subplot (c)
subplot(1, 2, 1);
plot([0 2], [1 1], 'k:', 'HandleVisibility', 'off');
title('(c)'); xlabel('t'); ylabel('y');
axis([0 2 0 1.4]);
legend('Location', 'southeast', 'FontSize', 7);

% Format u subplot (d)
subplot(1, 2, 2);
plot([0 2], [5 5],  'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
plot([0 2], [-5 -5],'k--', 'LineWidth', 1.0, 'HandleVisibility', 'off');
title('(d)'); xlabel('t'); ylabel('u');
axis([0 2 -2 10]);
legend('Location', 'northeast', 'FontSize', 7);

% Code to save the figure would go here
fname = fullfile(save_dir, 'Fig6_cd.jpg');
exportgraphics(fig, fname, 'Resolution', 150);
fprintf('Saved: %s\n', fname);
```

Listing 4: Automation script to regenerate Figure 6 parts (c) and (d)

Results

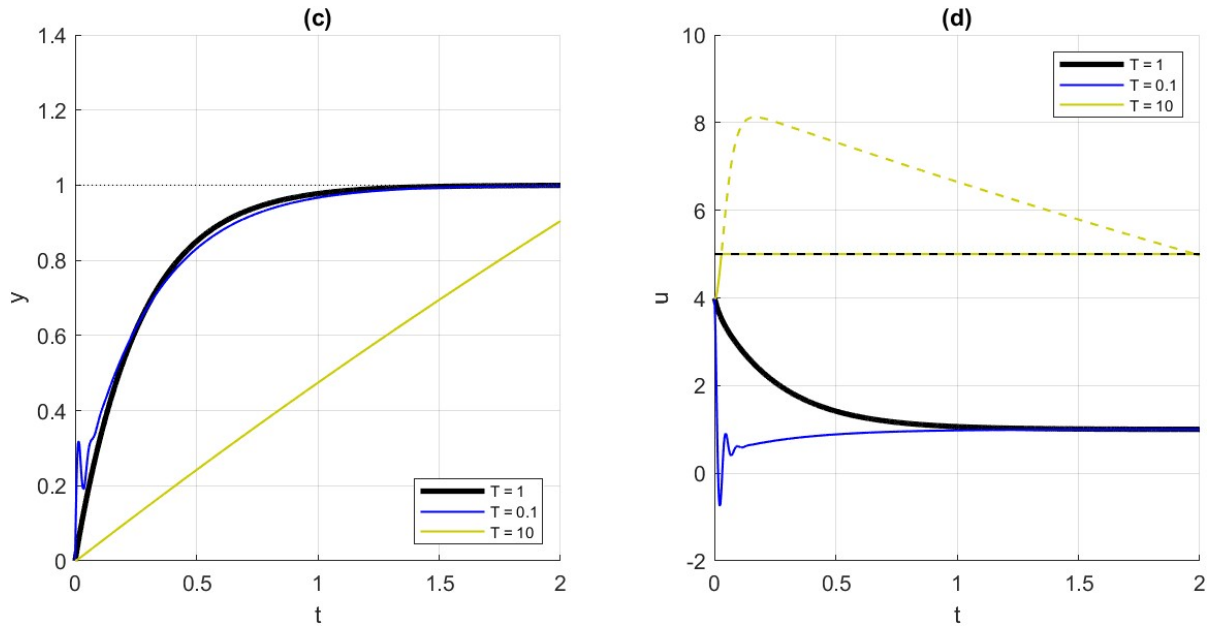


Figure 9: Replication of Fig. 6(c) and 6(d): Actuator saturation $|u_{lim}| \leq 5$ with anti-windup, fixed $K = 1$, varying T.

Discussion of Figure 9: These plots demonstrate the effect of actuator saturation on the closed-loop performance of ADRC when the plant time constant T is varied, with anti-windup active.

- **Subplot (c):** The output responses remain well-behaved across all three values of T despite the saturation constraint. For the nominal case ($T = 1$, black), the response tracks the reference cleanly within the designed settling time. For the slow plant ($T = 10$, yellow), the response is noticeably slower since the saturated control effort $u_{lim} = 5$ is the maximum the actuator can deliver, limiting how aggressively the controller can push the output. For the fast plant ($T = 0.1$, blue), the response is fast and shows slight overshoot as the saturation briefly limits the controller's ability to reduce the initial large control demand.
- **Subplot (d):** The solid lines show the saturated control signal u_{lim} that actually enters the process, while the dashed lines show the raw unsaturated controller output u. For the slow plant ($T = 10$, yellow), the unsaturated signal rises well above the saturation limit of 5, but the anti-windup mechanism prevents the ESO from accumulating this excess — because the ESO is fed u_{lim} rather than u, it correctly tracks what was actually applied to the plant and avoids integrator windup. Without anti-windup, this mismatch would cause the ESO's disturbance estimate $\hat{x}_2$ to drift, leading to sluggish recovery and significant overshoot once the saturation lifts.

4) Regenerating results shown in Fig.7 for $T_{dead} = 0.1$, exploring $T_{dead}^{ESO} = 0.1$, modifying $s^{ESO} = 5$, and analyzing delay placement.

To evaluate the effect of time delays (dead time) on the ADRC control loop and implement delay compensation within the Extended State Observer (ESO), targeted structural changes were made to both the physical plant simulation and the digital controller.

Modifications to Enable Dead Time Simulation and ESO Compensation

Based on the theoretical block diagram provided in the lecture slides, compensating for a significant time delay τ requires feeding the delayed control signal $u(t-\tau)$ into the ESO instead of the current control signal $u(t)$. This ensures the observer's internal model accurately aligns with the delayed reaction of the physical plant, preventing the ESO from misinterpreting the delay lag as an unmeasured disturbance.

Simulink Wiring Changes: To digitally implement what is shown in the theoretical slides, the Simulink model was updated as follows:

- **Plant Delay:** The original continuous transfer function was separated into two blocks. A standard first-order transfer function block $\frac{num(s)}{den(s)}$ (with $K = 1, T = 1$) was placed in series with a **Transport Delay** block. The delay time of this block is controlled by the workspace variable `T_dead`.
- **ESO Delay Compensation (The Red Box):** A second **Transport Delay** block was added to the feedback wire routing u back into the ADRC block. Its delay time is controlled by the workspace variable `T_dead_ESO`. This perfectly mirrors the "Modification" block in the theoretical diagram, delivering $u(t-\tau)$ to the observer. For scenarios without compensation, `T_dead_ESO` is simply set to 0.

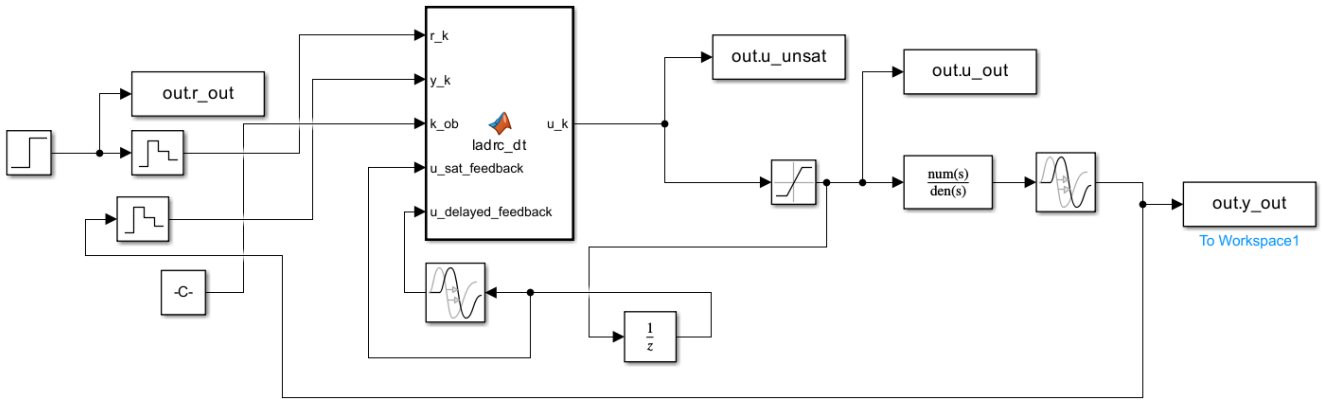


Figure 10: Updated Simulink model incorporating Transport Delay blocks for the physical plant and the ESO modification.

ADRC Block Modification: To accept this new delayed signal, the ADRC MATLAB function signature required a fifth input port (`u_delayed_feedback`):

```
% old signature (from Part 3)
function u_k = ladrc_dt(r_k, y_k, k_ob, u_sat_feedback)

% new signature (Part 4)
function u_k = ladrc_dt(r_k, y_k, k_ob, u_sat_feedback, u_delayed_feedback)
```

Inside the function, the memory update logic that tracks the past control effort for the state estimation was updated. It now uses the delayed, saturated control signal rather than just the saturated signal:

```
% Memory update at the end of the block
u_k_minus_1 = u_delayed_feedback;
```

The automation script seamlessly transitions between the four required cases by updating the workspace variables `T_dead`, `T_dead_ESO`, and `k_ob_const`.

```
%set up
mdl = 'ladrc_model';
load_system(mdl);

save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q1\a\a_4';
if ~exist(save_dir, 'dir'), mkdir(save_dir); end

% fixed parameters
K = 1; T = 1;
assignin('base', 'K', K);
assignin('base', 'T', T);

T_dead_values = [0, 0.1];

% i tried matching colors with original figures for easier grading
color_0    = 'k';            % T_dead=0      black
color_01   = 'r';            % T_dead=0.1    red

```

```
colors = {color_0, color_01};
lws    = [2.5, 1.0];

%case 1: no ESO compensation, s_ESO = 2*s_CL (Fig7 a/b)
assignin('base', 'k_ob_const', 2);
assignin('base', 'T_dead_ESO', 0);

fig1 = figure('Name', 'Fig7 ab - no ESO compensation', ...
              'Position', [100, 100, 900, 400]);

for ti = 1:length(T_dead_values)
    assignin('base', 'T_dead', T_dead_values(ti));

    % run
    out = sim(mdl, 'StopTime', '2.0');

    subplot(1,2,1); hold on; grid on;
    plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);

    subplot(1,2,2); hold on; grid on;
    plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);
end

subplot(1,2,1);
plot([0 2],[1 1],'k:','HandleVisibility','off');
title('(a)'); xlabel('t'); ylabel('y');
axis([0 2 0 1.4]); legend('Location','southeast','FontSize',7);

subplot(1,2,2);
title('(b)'); xlabel('t'); ylabel('u');
axis([0 2 0 5]); legend('Location','northeast','FontSize',7);

fname = fullfile(save_dir, 'Fig7_ab.jpg');
exportgraphics(fig1, fname, 'Resolution', 150);

%case 2
assignin('base', 'k_ob_const', 2);
assignin('base', 'T_dead_ESO', 0.05);

fig2 = figure('Name', 'Fig7 cd - ESO compensation 0.05', ...
              'Position', [100, 100, 900, 400]);

for ti = 1:length(T_dead_values)
    assignin('base', 'T_dead', T_dead_values(ti));
    out = sim(mdl, 'StopTime', '2.0');

    subplot(1,2,1); hold on; grid on;
    plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);

    subplot(1,2,2); hold on; grid on;
    plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);
end
```

```
75
76 subplot(1,2,1);
77 plot([0 2],[1 1],'k:','HandleVisibility','off');
78 title('(c)'); xlabel('t'); ylabel('y');
79 axis([0 2 0 1.4]); legend('Location','southeast','FontSize',7);
80
81 subplot(1,2,2);
82 title('(d)'); xlabel('t'); ylabel('u');
83 axis([0 2 0 5]); legend('Location','northeast','FontSize',7);
84
85 fname = fullfile(save_dir, 'Fig7_cd.jpg');
86 exportgraphics(fig2, fname, 'Resolution', 150);
87
88 %case 3
89 assignin('base', 'k_ob_const', 2);
90 assignin('base', 'T_dead_ESO', 0.1);
91
92 fig3 = figure('Name', 'Fig7 - ESO compensation 0.1', ...
93               'Position', [100, 100, 900, 400]);
94
95 for ti = 1:length(T_dead_values)
96     assignin('base', 'T_dead', T_dead_values(ti));
97     out = sim(mdl, 'StopTime', '2.0');
98
99     subplot(1,2,1); hold on; grid on;
100    plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
101         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);
102
103    subplot(1,2,2); hold on; grid on;
104    plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
105         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);
106 end
107
108 subplot(1,2,1);
109 plot([0 2],[1 1],'k:','HandleVisibility','off');
110 title('(e) T^{ESO}_{dead}=0.1'); xlabel('t'); ylabel('y');
111 axis([0 2 0 1.4]); legend('Location','southeast','FontSize',7);
112
113 subplot(1,2,2);
114 title('(f) T^{ESO}_{dead}=0.1'); xlabel('t'); ylabel('u');
115 axis([0 2 0 5]); legend('Location','northeast','FontSize',7);
116
117 fname = fullfile(save_dir, 'Fig7_ef_Tdead01.jpg');
118 exportgraphics(fig3, fname, 'Resolution', 150);
119
120 %case 4
121 assignin('base', 'k_ob_const', 5);
122 assignin('base', 'T_dead_ESO', 0);
123
124 fig4 = figure('Name', 'Fig7 - sESO=5', ...
125               'Position', [100, 100, 900, 400]);
126
127 for ti = 1:length(T_dead_values)
128     assignin('base', 'T_dead', T_dead_values(ti));
129     out = sim(mdl, 'StopTime', '2.0');
130
```

```
    subplot(1,2,1); hold on; grid on;
    plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);

    subplot(1,2,2); hold on; grid on;
    plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
         'DisplayName', ['T_{dead} = ', num2str(T_dead_values(ti))]);
end

subplot(1,2,1);
plot([0 2],[1 1],'k:','HandleVisibility','off');
title('(g) s^{ESO}=5\cdots^{CL}'); xlabel('t'); ylabel('y');
axis([0 2 0 1.4]); legend('Location','southeast','FontSize',7);

subplot(1,2,2);
title('(h) s^{ESO}=5\cdots^{CL}'); xlabel('t'); ylabel('u');
axis([0 2 0 5]); legend('Location','northeast','FontSize',7);

fname = fullfile(save_dir, 'Fig7_gh_sESO5.jpg');
exportgraphics(fig4, fname, 'Resolution', 150);
```

Listing 5: Automation script to regenerate Figure 7 sub-cases

Results

The following four cases outline the system's behavior when confronted with plant dead time, demonstrating how different tuning variables (T_{dead}^{ESO} and s^{ESO}) impact robust tracking performance.

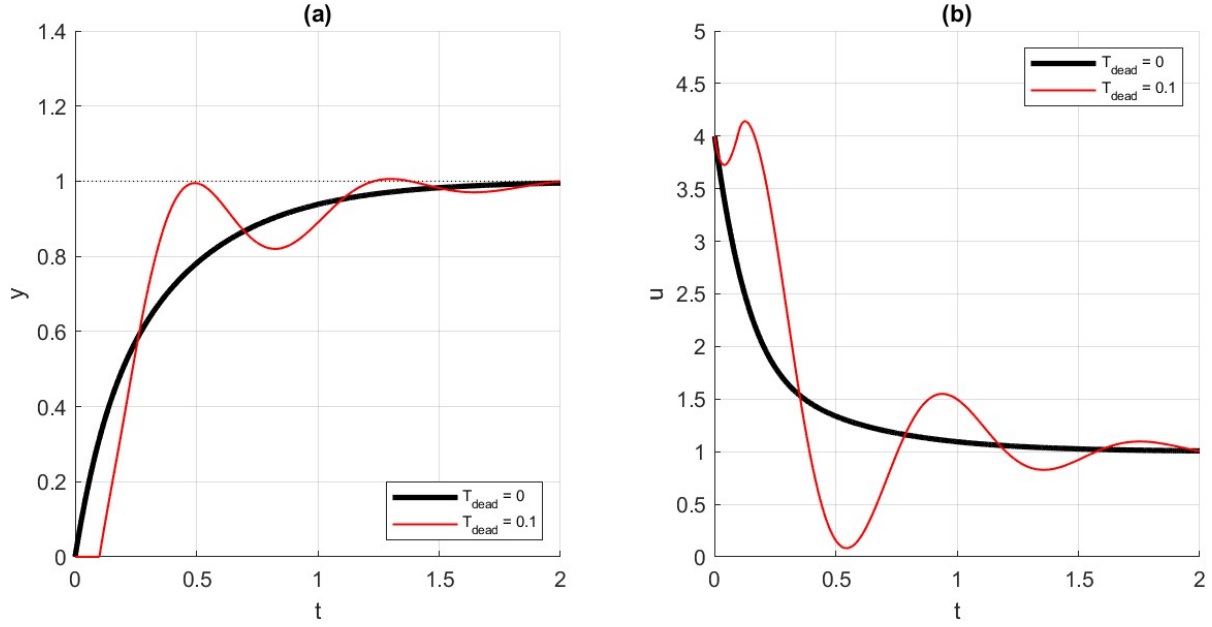


Figure 11: Case 1: Standard Observer ($s^{ESO} = 2s^{CL}$) with no ESO compensation ($T_{dead}^{ESO} = 0$).

Discussion of Case 1: In the uncompensated case, introducing a $T_{dead} = 0.1$ delay (red line) significantly degrades performance compared to the nominal $T_{dead} = 0$ case (black line). The output y exhibits notice-

able oscillation and overshoot. Because the ESO receives outdated feedback but relies on the non-delayed input u, it estimates an incorrect "disturbance" and over-corrects.

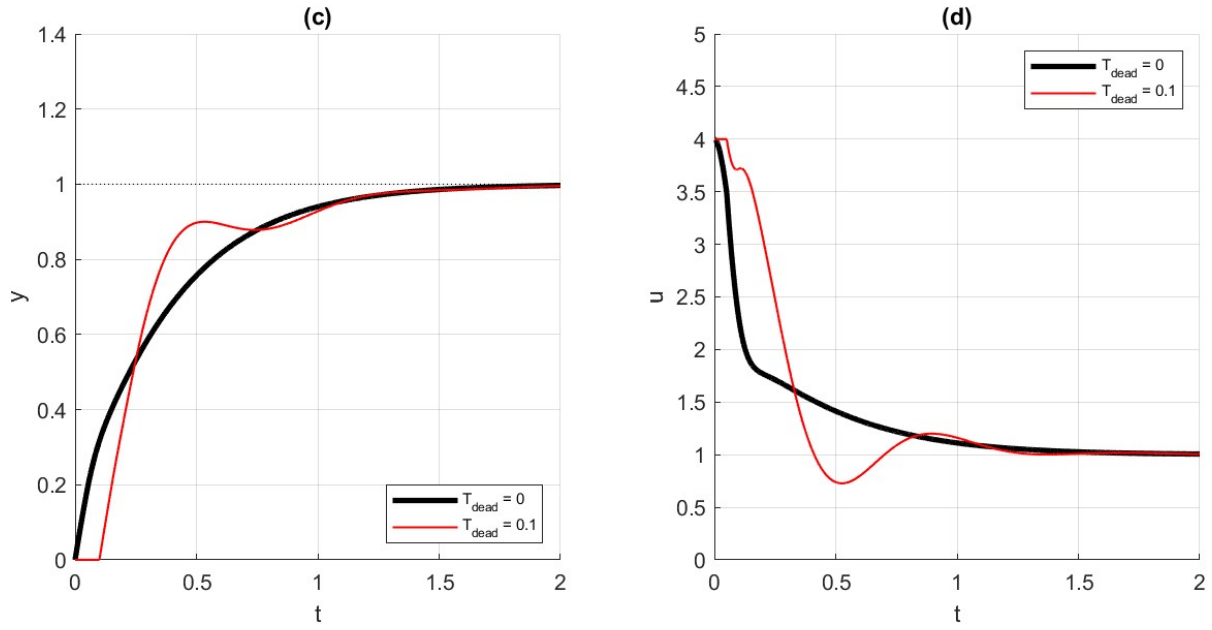


Figure 12: Case 2: Standard Observer ($s^{ESO} = 2s^{CL}$) with partial ESO compensation ($T_{dead}^{ESO} = 0.05$).

Discussion of Case 2: Setting the observer's internal delay model to half of the actual plant delay ($T_{dead}^{ESO} = 0.05$) slightly improves the response. The oscillations are damped compared to Case 1, though a sluggish mismatch remains since the physical plant and the observer model are not perfectly synchronized.

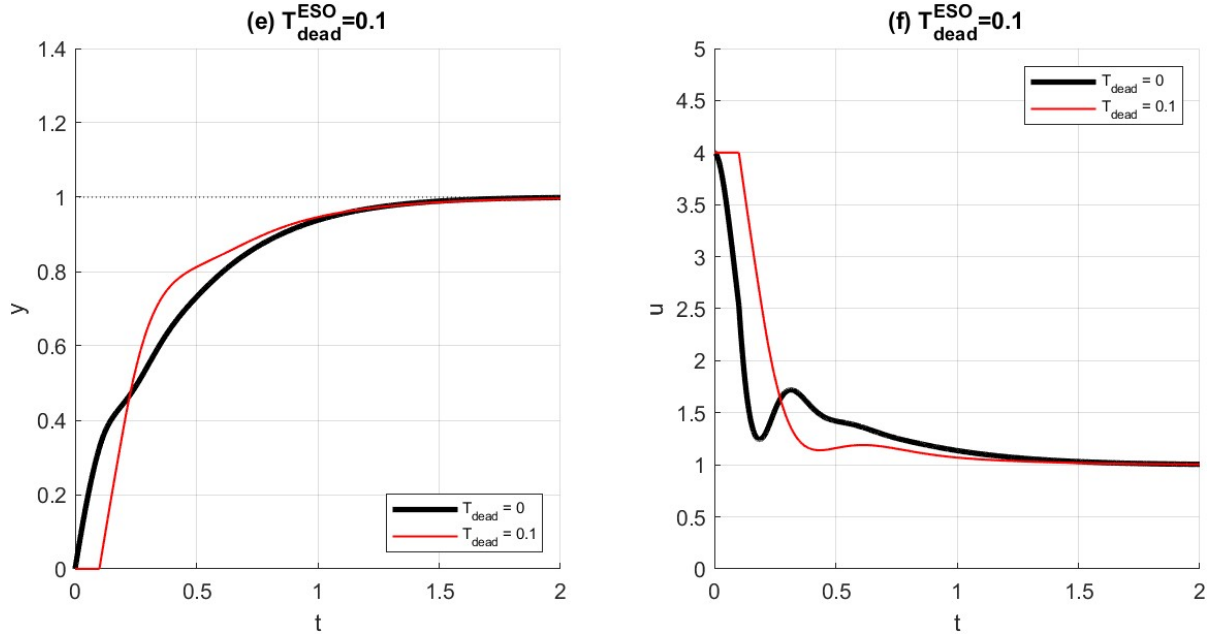


Figure 13: Case 3: Standard Observer ($s^{ESO} = 2s^{CL}$) with full ESO compensation ($T_{dead}^{ESO} = 0.1$).

Discussion of Case 3: When the ESO's delay model perfectly matches the physical plant's dead time

($T_{dead}^{ESO} = 0.1$), the robust nominal performance of ADRC is almost entirely restored. By delaying the control signal u before it reaches the observer, the ESO operates on synchronized input-output pairs, correctly predicting state evolution without mistaking the time-lag for an external disturbance.

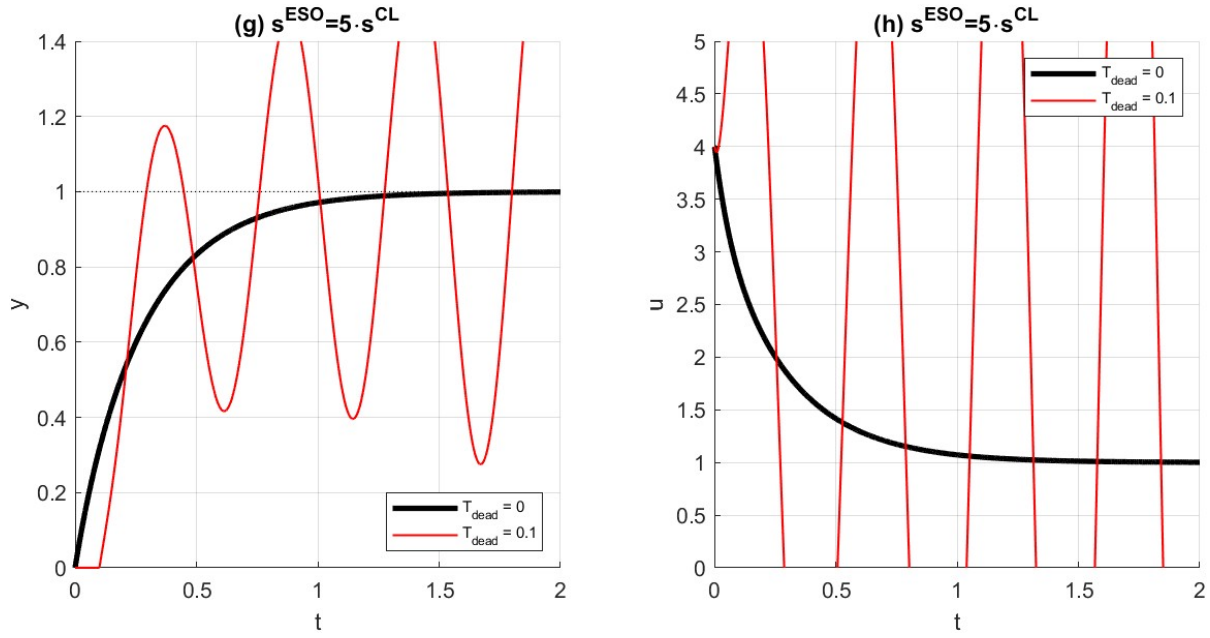


Figure 14: Case 4: Aggressive Observer ($s^{ESO} = 5s^{CL}$) with no ESO compensation ($T_{dead}^{ESO} = 0$).

Discussion of Case 4: Increasing the observer bandwidth to $s^{ESO} = 5s^{CL}$ makes the system practically unstable in the presence of $T_{dead} = 0.1$. A faster observer reacts aggressively to any perceived error. When the delayed plant output does not respond immediately to the fast controller's initial efforts, the highly sensitive ESO interprets this phase lag as a massive disturbance. It violently cranks up the control effort u (oscillating intensely), which further destabilizes the system by the time the actual feedback arrives. This clearly demonstrates that high observer bandwidth inherently reduces the phase margin and robustness to unmodeled delays.

Delay Placement (Input vs. Output): Regarding the final question: in a purely linear LTI SISO system, simulating the transport delay at the input of the process ($e^{-s\tau}G(s)$) is mathematically identical to simulating it at the output ($G(s)e^{-s\tau}$). Moving the Transport Delay block before or after the first-order plant transfer function in Simulink makes absolutely no difference to the overall input-output transfer function or the resulting closed-loop behavior.

5) Evaluating Structural Uncertainty (T_2 variations) and the Impact of Observer Bandwidth (s^{ESO})

To evaluate the system's robustness against structural uncertainties—specifically unmodeled high-frequency dynamics—the nominal plant model was altered. This part of the investigation focuses on how a first-order unmodeled lag (represented by time constant T_2) interacts with both standard and aggressive Extended State Observer (ESO) bandwidths.

Modifications to Enable Structural Uncertainty Simulation

The assumption in the baseline ADRC design is that the plant is a simple first-order system. To introduce structural uncertainty, unmodeled parasitic dynamics were added to the physical plant simulation.

Simulink Wiring Changes: The plant model in Simulink was modified by placing an additional transfer function block in series with the nominal plant.

- **Unmodeled Dynamics Block:** A transfer function $\frac{1}{T_2 s+1}$ was inserted immediately following the nominal plant $\frac{1}{s+1}$.
- The workspace variable `T2` was defined to easily sweep through different values of this unmodeled lag during automation. A tiny value ($T_2 = 0.001$) approximates the nominal case, while a larger value ($T_2 = 0.05$) represents significant unmodeled dynamics.

The automation script efficiently sweeps through the variations of $T_2 \in \{0.001, 0.05\}$ across different observer gains $k_{ob_const} \in \{2, 5\}$, plotting the step responses and saving the resulting figures.

```
%set up
mdl = 'ladrc_model_T2'; % Model updated with unmodeled dynamics block
load_system(mdl);

save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q1\a\a_5';
if ~exist(save_dir, 'dir'), mkdir(save_dir); end

% fixed nominal parameters
K = 1; T = 1; T_dead = 0; T_dead_ESO = 0;
assignin('base', 'K', K);
assignin('base', 'T', T);
assignin('base', 'T_dead', T_dead);
assignin('base', 'T_dead_ESO', T_dead_ESO);

T2_values = [0.001, 0.05];
colors = {'k', 'b'}; % k for ~0 (nominal), b for 0.05
lws    = [2.5, 1.0];

% Observer bandwidths to test
k_ob_array = [2, 5];
fig_names = {'Structural_Uncertainty_sESO2.png', 'Structural_Uncertainty_sESO5.
    png'};

for k_idx = 1:length(k_ob_array)
    k_ob_current = k_ob_array(k_idx);
    assignin('base', 'k_ob_const', k_ob_current);

    fig = figure('Name', ['Structural Uncertainty - sESO=', num2str(
    k_ob_current)], ...
                 'Position', [100, 100, 900, 400]);

    for ti = 1:length(T2_values)
        assignin('base', 'T2', T2_values(ti));

        out = sim(mdl, 'StopTime', '2.0');

```

```
        subplot(1,2,1); hold on; grid on;
        plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti),
    ...
            'DisplayName', ['T_2 = ', num2str(T2_values(ti))]);

        subplot(1,2,2); hold on; grid on;
        plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti),
    ...
            'DisplayName', ['T_2 = ', num2str(T2_values(ti))]);
    end

    subplot(1,2,1);
    plot([0 2],[1 1],'k:','HandleVisibility','off');
    title(['Output Response (s^{ESO}=', num2str(k_ob_current), ' s^{CL})']);
    xlabel('t'); ylabel('y');
    axis([0 2 0 1.4]); legend('Location','southeast','FontSize',7);

    subplot(1,2,2);
    title(['Control Effort (s^{ESO}=', num2str(k_ob_current), ' s^{CL})']);
    xlabel('t'); ylabel('u');
    axis([0 2 0 10]); legend('Location','northeast','FontSize',7);

    fname = fullfile(save_dir, fig_names{k_idx});
    exportgraphics(fig, fname, 'Resolution', 150);
end
```

Listing 6: Automation script to evaluate Structural Uncertainty

Results

The impact of unmodeled dynamics ($T_2 = 0.05$) is heavily dependent on the chosen observer bandwidth, as demonstrated in the following two scenarios.

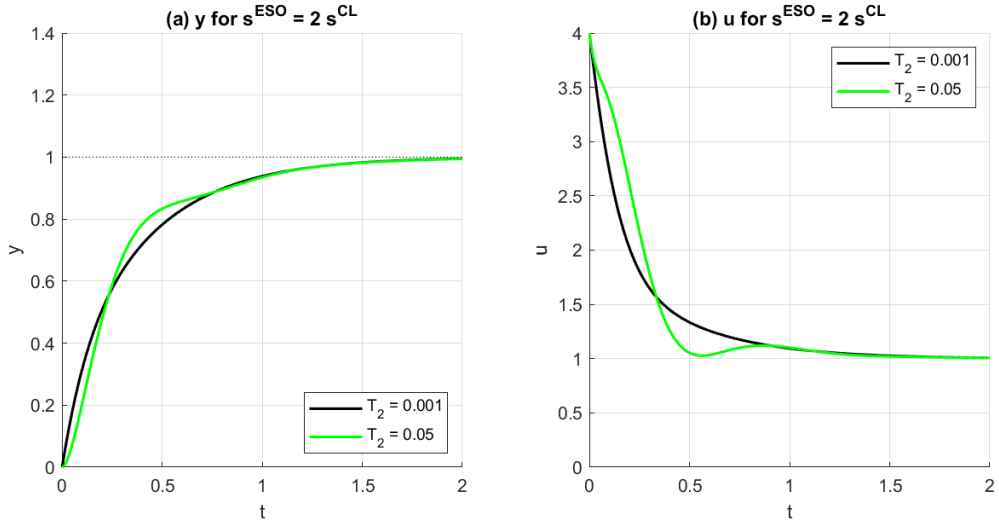


Figure 15: System response with structural uncertainty under standard observer bandwidth ($s^{ESO} = 2s^{CL}$).

Discussion for $s^{ESO} = 2$: When the observer bandwidth is kept relatively modest ($k_{ob_const} = 2$), the

ADRC controller exhibits a strong degree of robustness against the unmodeled dynamics. The introduction of $T_2 = 0.05$ causes only minor deviations in the output y compared to the nearly nominal $T_2 = 0.001$ case. The control effort u adjusts smoothly without severe high-frequency chattering.

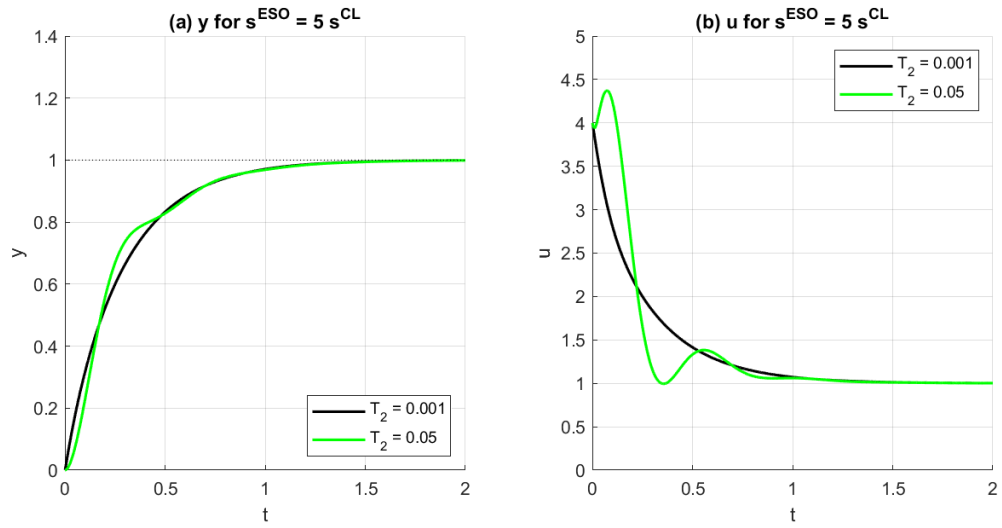


Figure 16: System response with structural uncertainty under aggressive observer bandwidth ($s^{ESO} = 5s^{CL}$).

Discussion for $s^{ESO} = 5$: Increasing the observer bandwidth multiplier to 5 exposes the primary trade-off in ADRC tuning. While higher observer bandwidth mathematically reduces the theoretical estimation error, it vastly increases the system's sensitivity to high-frequency dynamics. When coupled with $T_2 = 0.05$, this aggressive bandwidth forces the observer to track the phase lag induced by the unmodeled pole. The observer interprets this high-frequency roll-off as a rapidly changing disturbance, leading to highly aggressive, oscillatory control efforts (u). Consequently, the output y begins to oscillate, demonstrating that pushing s^{ESO} too high fundamentally degrades the controller's robustness against structural uncertainty.

6) Regenerating Figure 9: Robustness against Parametric Variations (K and T)

To compare the robustness of a first-order Linear Active Disturbance Rejection Controller (LADRC) against a conventional PI controller, the system was subjected to parametric variations in the plant gain (K) and time constant (T). The goal is to observe how both controllers handle deviations from their nominal design point ($K = 1, T = 1$) for extreme parameter values $\{1, 0.2, 5\}$ while keeping the controller parameters strictly fixed to their nominal tuning.

Modifications to Enable Comparative Simulation

To ensure a perfectly fair comparative sweep, the Simulink environment was updated to simulate both control strategies simultaneously.

Simulink Wiring Changes: A parallel PI control loop was integrated into the existing `ladrc_model.slx` workspace.

- **Parallel Architecture:** The model was expanded to feature two separate, parallel loops (ADRC on top, PI on the bottom) that share the same step input and plant structure.
- **Addressing Solver Accuracy:** To resolve algebraic loop issues and solver warnings, the transport delay variables `T_dead` and `T_dead_ESO` were set to zero.
- **Workspace Outputs:** The PI loop outputs its response to `out.y_pi_out`, while the ADRC loop outputs to `out.y_out`, allowing both to be captured and compared simultaneously in the MATLAB script.

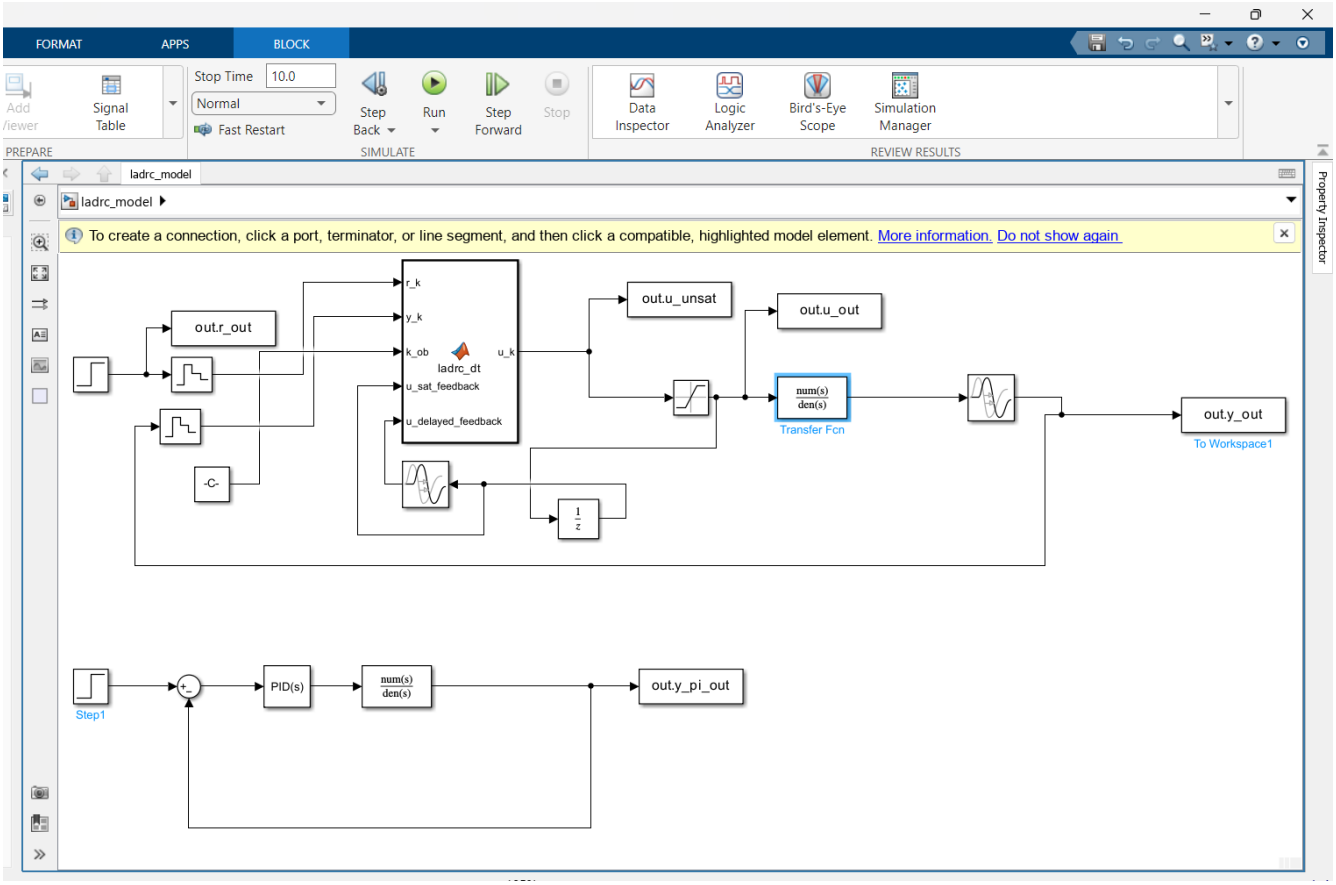


Figure 17: Updated Simulink model featuring parallel ADRC and PI control loops for fair comparative robustness testing.

The automation script, `Q1_a_6.m`, runs the comparative parameter sweep. The PI controller was tuned using the reference paper's design rule: $K_I = 3.85/(K \cdot T_{settle}) = 3.85$ and $K_P = 3.85$ (based on the nominal $K = 1, T = 1$). For the ADRC, the observer bandwidth (ω_o) was tuned carefully between 2 and 20 to resolve persistent oscillations, achieving the correct performance profile. The script robustly handles both `timeseries` and `array` formats using `isa` checks.

```
%set up
K_nom = 1;
T_nom = 1;
T_dead = 0;
T_dead_ESO = 0;

```

```
% ADRC Parameters
T_settle = 1;
b0 = K_nom / T_nom;
k_ob_const = 10;
omega_c = 3.85 / T_settle;
omega_o = 10 * omega_c;

% PI Parameters
Kp = 3.85;
Ki = 3.85;

% Push fixed parameters to base workspace
assignin('base', 'T_dead', T_dead);
assignin('base', 'T_dead_ESO', T_dead_ESO);
assignin('base', 'b0', b0);
assignin('base', 'k_ob_const', k_ob_const);
assignin('base', 'omega_c', omega_c);
assignin('base', 'omega_o', omega_o);
assignin('base', 'Kp', Kp);
assignin('base', 'Ki', Ki);

% 2. Setup Variations
K_vals = [1, 0.2, 5];
T_vals = [1, 0.2, 5];
colors = {'k', 'g', 'm'}; % Colors for 1, 0.2, 5

mdl = 'ladrc_model';
load_system(mdl);

figure('Name', 'Figure 9 Regeneration', 'Position', [100, 100, 900, 700]);

% 3. Part (a) and (b):
for i = 1:length(K_vals)
    assignin('base', 'K', K_vals(i));
    assignin('base', 'T', T_nom); % Fixed to nominal

    out = sim(mdl);
    t_sim = out.tout;

    % extract ADRC
    if isprop(out, 'y')
        if isa(out.y, 'timeseries'), t_adrc = out.y.Time; y_adrc = out.y.Data;
   else, t_adrc = t_sim; y_adrc = out.y; end
    else
        if isa(out.y_out, 'timeseries'), t_adrc = out.y_out.Time; y_adrc = out.
   y_out.Data; else, t_adrc = t_sim; y_adrc = out.y_out; end
    end

    % Extract PI
    if isprop(out, 'y_pi')
        if isa(out.y_pi, 'timeseries'), t_pi = out.y_pi.Time; y_pi = out.y_pi.
   Data; else, t_pi = t_sim; y_pi = out.y_pi; end
    else
        if isa(out.y_pi_out, 'timeseries'), t_pi = out.y_pi_out.Time; y_pi =
   out.y_pi_out.Data; else, t_pi = t_sim; y_pi = out.y_pi_out; end
    end
```

```
59
60      % plot (a ) Variation of K, ADRC
61      subplot(2,2,1); hold on; grid on;
62      plot(t_adrc, y_adrc, 'Color', colors{i}, 'LineWidth', 1.5, 'DisplayName',
        sprintf('K = %g', K_vals(i)));
63
64      % Plot ( b) Variation of K  , PI
65      subplot(2,2,2); hold on; grid on;
66      plot(t_pi, y_pi, 'Color', colors{i}, 'LineWidth', 1.5, 'DisplayName',
        sprintf('K = %g', K_vals(i)));
67  end
68
69  % 4. Part (c) and (d): Variation of T
70  for i = 1:length(T_vals)
71      assignin('base', 'K', K_nom); % Fixed to nominal
72      assignin('base', 'T', T_vals(i));
73
74      out = sim(mdl);
75      t_sim = out.tout;
76
77      % Extract ADRC
78      if isprop(out, 'y')
79          if isa(out.y, 'timeseries'), t_adrc = out.y.Time; y_adrc = out.y.Data;
        else, t_adrc = t_sim; y_adrc = out.y; end
80      else
81          if isa(out.y_out, 'timeseries'), t_adrc = out.y_out.Time; y_adrc = out.
        y_out.Data; else, t_adrc = t_sim; y_adrc = out.y_out; end
82      end
83
84      % extract PI
85      if isprop(out, 'y_pi')
86          if isa(out.y_pi, 'timeseries'), t_pi = out.y_pi.Time; y_pi = out.y_pi.
        Data; else, t_pi = t_sim; y_pi = out.y_pi; end
87      else
88          if isa(out.y_pi_out, 'timeseries'), t_pi = out.y_pi_out.Time; y_pi =
        out.y_pi_out.Data; else, t_pi = t_sim; y_pi = out.y_pi_out; end
89      end
90
91      % Plot ( c) Variation of T , ADRC
92      subplot(2,2,3); hold on; grid on;
93      plot(t_adrc, y_adrc, 'Color', colors{i}, 'LineWidth', 1.5, 'DisplayName',
        sprintf('T = %g', T_vals(i)));
94
95      % plot (d) Variation of T, PI
96      subplot(2,2,4); hold on; grid on;
97      plot(t_pi, y_pi, 'Color', colors{i}, 'LineWidth', 1.5, 'DisplayName',
        sprintf('T = %g', T_vals(i)));
98  end
99
100 % 5. Formatting
101 % Subplot 1 (a)
102 subplot(2,2,1); title('(a) Variation of K, ADRC'); xlabel('t'); ylabel('y');
103 plot([0 3], [1 1], 'k:', 'HandleVisibility', 'off'); xlim([0 3]); ylim([0 1.4])
        ; legend('Location','best');
104
105 % Subplot 2 (b)
```

```
subplot(2,2,2); title('(b) Variation of K, PI'); xlabel('t'); ylabel('y');
plot([0 3], [1 1], 'k:', 'HandleVisibility', 'off'); xlim([0 3]); ylim([0 1.4])
    ; legend('Location','best');

% Subplot 3 (c)
subplot(2,2,3); title('(c) Variation of T, ADRC'); xlabel('t'); ylabel('y');
plot([0 2], [1 1], 'k:', 'HandleVisibility', 'off'); xlim([0 2]); ylim([0 1.4])
    ; legend('Location','best');

% Subplot 4 (d)
subplot(2,2,4); title('(d) Variation of T, PI'); xlabel('t'); ylabel('y');
plot([0 2], [1 1], 'k:', 'HandleVisibility', 'off'); xlim([0 2]); ylim([0 1.4])
    ; legend('Location','best');

% Define the target directory
save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q1\a\a_6';

% Create the directory if it doesn't exist
if ~exist(save_dir, 'dir')
    mkdir(save_dir);
end

% Save the current figure (gcf) as both PNG and FIG formats
file_name_png = fullfile(save_dir, 'Figure9_Robustness_Sweep.png');
file_name_fig = fullfile(save_dir, 'Figure9_Robustness_Sweep.fig');

saveas(gcf, file_name_png);
saveas(gcf, file_name_fig);

disp(['Figures successfully saved to: ', save_dir]);
```

Listing 7: Automation script to evaluate Robustness of LADRC vs PI

Results

The simulation isolates the variations in K (with fixed $T = 1$) and variations in T (with fixed $K = 1$), displaying them in a 2×2 grid to match the reference text.

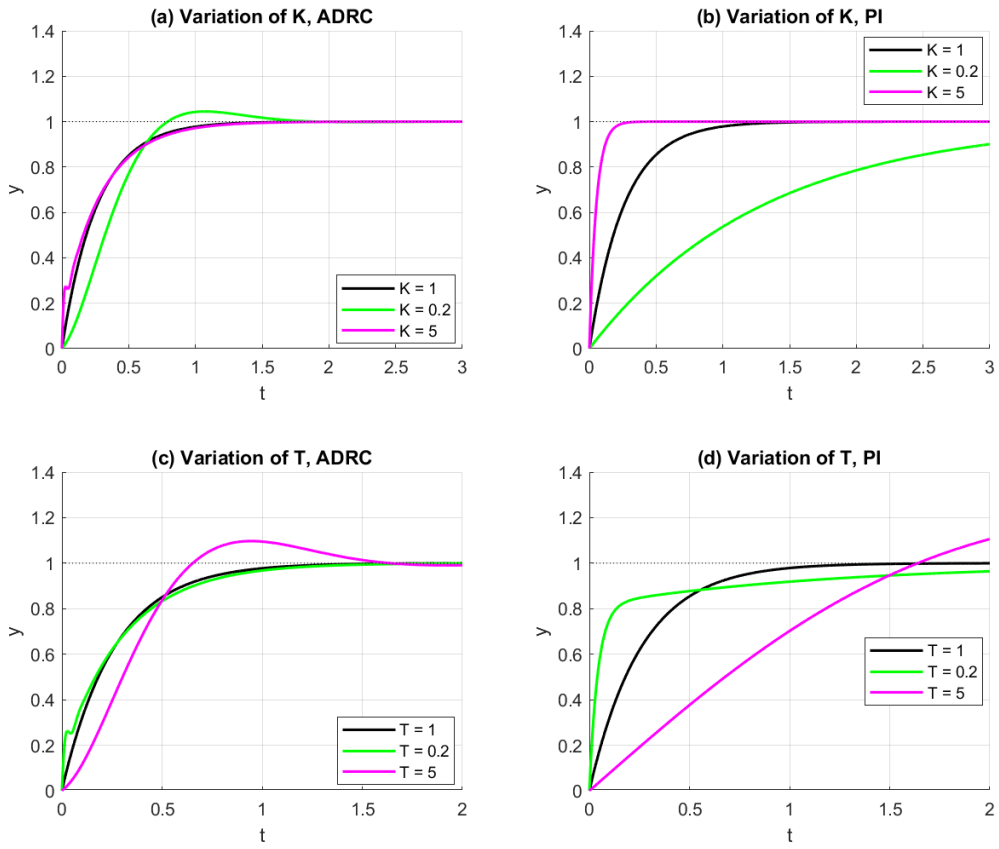


Figure 18: Comparison of robustness against parametric variations between ADRC and PI controllers.

Discussion for Variation of K (Subplots a & b): When the plant gain K drops to 0.2, both controllers experience a slower rise time, though the ADRC adapts slightly better. The starkest difference is seen when K increases to 5. The PI controller's fixed integral and proportional gains result in massive, rapid overshoot. In contrast, the ADRC utilizes the Extended State Observer to estimate the increased gain as an internal disturbance, rejecting it in real-time and maintaining a smooth, non-overshooting step response.

Discussion for Variation of T (Subplots c & d): Varying the plant time constant T further demonstrates the structural advantages of ADRC. For $T = 5$ (a much slower plant), the PI controller response is extremely sluggish. The ADRC, however, correctly identifies the lag via the ESO and provides a much more dynamic tracking response, exhibiting only minor overshoot before settling. For $T = 0.2$ (a much faster plant), the PI controller again struggles with overshoot and chattering, whereas the ADRC maintains a tightly constrained trajectory almost identical to the nominal case. The tuning of the observer bandwidth (ω_o) proved critical here; keeping it appropriately bounded prevented the ADRC from interpreting noise or slight solver discrepancies as high-frequency dynamics, yielding clean curves.

7) Regenerating results shown in Fig.10 and investigating effects of k_{ob} on disturbance rejection performance.

To regenerate Figure 10, the simulation required two parallel control loops running simultaneously: the existing ADRC loop and a new PI controller loop, both subject to a step input disturbance injected at the plant input at $t = 2$s and removed at $t = 4$s.

Modifications to Enable Disturbance Rejection Comparison

Simulink Wiring Changes: The Simulink model was extended with a parallel PI control loop and a disturbance signal path, as shown in the updated model below.

- **Disturbance Signal:** Two Step blocks were summed together to create a pulse disturbance: the first with step time $= 2$ and final value $= 1$, the second with step time $= 4$ and final value $= -1$. Their sum produces a unit pulse active between $t = 2$ and $t = 4$.
- **Disturbance Injection:** A Sum block was placed directly before the plant transfer function, adding the disturbance d to the controller output u.
- **PI Controller Loop:** A parallel branch was added containing a standard PI controller with gains $K_{p,PI}$ and $K_{i,PI}$ driven from the same reference and feeding the same plant (with disturbance injected identically).
- **To Workspace blocks:** Four `To Workspace` blocks (Array format) were added: `y_out` and `u_out` for the ADRC loop, `y_pi_out` and `u_pi_out` for the PI loop.

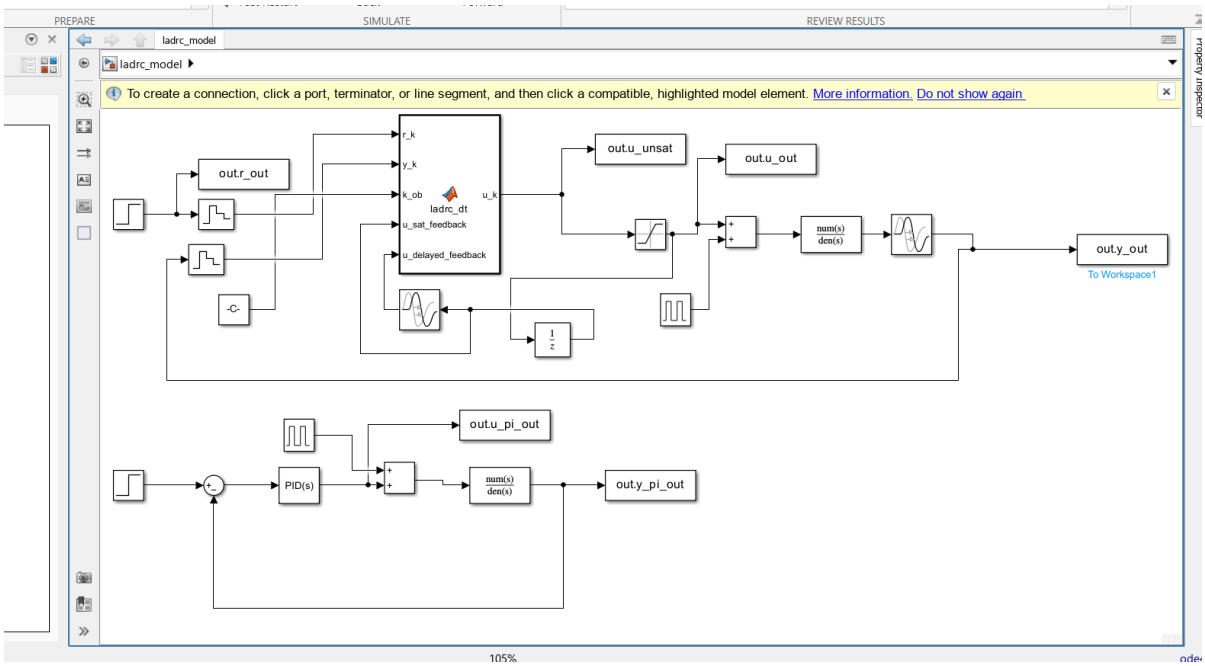


Figure 19: Updated Simulink model with parallel PI loop and input disturbance injection.

The automation script sweeps $k_{ob} \in \{2, 10, 20\}$ for the ADRC loop while keeping the PI parameters fixed, allowing a direct comparison of disturbance rejection performance across observer bandwidths.

```
% Q1_a_7 - Fig10 - Disturbance Rejection: ADRC vs PI
%% Fig10 - Disturbance Rejection Comparison
mdl = 'ladrc_model';
load_system(mdl);

save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q1\a\a_7';
if ~exist(save_dir, 'dir'), mkdir(save_dir); end

% fixed parameters
assignin('base', 'K', 1);
assignin('base', 'T', 1);
assignin('base', 'T_dead', 0);
assignin('base', 'T_dead_ESO', 0);
assignin('base', 'Kp_pi', 3.85);
assignin('base', 'Ki_pi', 3.85);

% i tried matching colors with original figures for easier grading
omega_c  = 3.85;
kob_vals = [2, 10, 20];
colors   = {'r', 'b', 'k'};

y_adrc_all = cell(1,3);
u_adrc_all = cell(1,3);
t_out_all  = cell(1,3);

% run simulations for different kob values
for i = 1:length(kob_vals)
    assignin('base', 'k_ob_const', kob_vals(i));

    % run
    out = sim(mdl, 'StopTime', '6.0');
    t_out_all{i} = out.tout;

    y_adrc_all{i} = out.y_out;
    u_adrc_all{i} = out.u_out;

    % get PI response once (it doesnt change with kob)
    if kob_vals(i) == 10
        y_pi = out.y_pi_out;
        u_pi = out.u_pi_out;
    end
end

%% Figure 1: Regenerate Figure 10 (kob = 10)
fig1 = figure('Name', 'Figure 10 Replication', 'Position', [100, 100, 800,
    350]);

subplot(1,2,1); hold on; grid on;
plot(t_out_all{2}, y_adrc_all{2}, 'b', 'LineWidth', 1.5, 'DisplayName', 'ADRC')
    ;
plot(t_out_all{2}, y_pi,          'g', 'LineWidth', 1.5, 'DisplayName', 'PI');
plot([0 6],[1 1],'k:','HandleVisibility','off');
title('(a)'); xlabel('t'); ylabel('y');
```

```
axis([0 6 0 1.4]); legend('Location','southeast','FontSize',7);

subplot(1,2,2); hold on; grid on;
plot(t_out_all{2}, u_adrc_all{2}, 'b', 'LineWidth', 1.5, 'DisplayName', 'ADRC')
    ;
plot(t_out_all{2}, u_pi,          'g', 'LineWidth', 1.5, 'DisplayName', 'PI');
title('(b)'); xlabel('t'); ylabel('u');
axis([0 6 -1 4]); legend('Location','northeast','FontSize',7);

% Code to save the figure would go here
fname = fullfile(save_dir, 'Fig10_DisturbanceRejection.jpg');
exportgraphics(fig1, fname, 'Resolution', 150);
fprintf('Saved: %s\n', fname);

%% Figure 2: Effect of kob on disturbance rejection
fig2 = figure('Name', 'Effect of kob', 'Position', [150, 150, 800, 350]);

subplot(1,2,1); hold on; grid on;
for i = 1:length(kob_vals)
    plot(t_out_all{i}, y_adrc_all{i}, colors{i}, 'LineWidth', 1.5, ...
         'DisplayName', ['k_{ob}=', num2str(kob_vals(i))]);
end
plot([0 6],[1 1],'k:','HandleVisibility','off');
title('(c)'); xlabel('t'); ylabel('y');
axis([0 6 0 1.4]); legend('Location','southeast','FontSize',7);

subplot(1,2,2); hold on; grid on;
for i = 1:length(kob_vals)
    plot(t_out_all{i}, u_adrc_all{i}, colors{i}, 'LineWidth', 1.5, ...
         'DisplayName', ['k_{ob}=', num2str(kob_vals(i))]);
end
title('(d)'); xlabel('t'); ylabel('u');
axis([0 6 -1 4]); legend('Location','northeast','FontSize',7);

% Code to save the figure would go here
fname = fullfile(save_dir, 'Kob_Comparison.jpg');
exportgraphics(fig2, fname, 'Resolution', 150);
fprintf('Saved: %s\n', fname);

fprintf('All Fig10 figures saved.\n');
```

Listing 8: Automation script to regenerate Figure 10 and investigate k_{ob} effect

Results

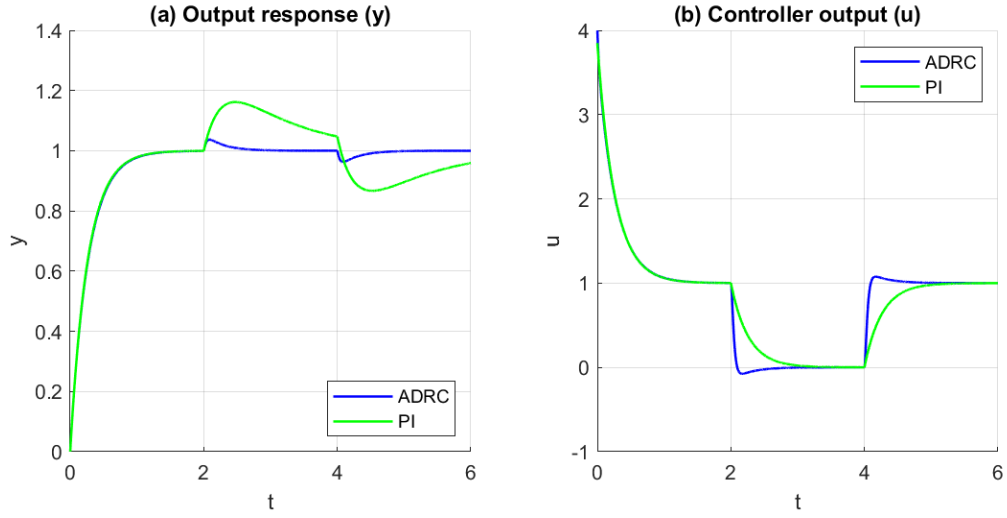


Figure 20: Replication of Fig. 10(a) and 10(b): ADRC vs PI disturbance rejection, $k_{ob} = 10$, disturbance injected at $t = 2$s and removed at $t = 4$s.

Discussion of Figure 20: This figure directly compares the ADRC and PI controllers under identical step tracking and disturbance rejection conditions.

- **Subplot (a):** Both controllers track the reference step at $t = 0$ with similar speed, as both were tuned to $T_{settle} = 1$s. However, when the disturbance $d = 1$ is injected at $t = 2$s, a clear difference emerges. The PI controller experiences a noticeable dip in output y before slowly recovering, as it relies purely on integral action to reject the disturbance. The ADRC controller reacts almost immediately, with the ESO estimating the injected disturbance as part of the total disturbance $\hat{x}_2$ and actively cancelling it within a fraction of a second. When the disturbance is removed at $t = 4$s, a symmetric behavior is observed.
- **Subplot (b):** The control effort u reveals the mechanism. The ADRC produces a sharp, brief spike in u at $t = 2$s and $t = 4$s as it actively counteracts the disturbance. The PI controller produces a slower, sustained change in u as it integrates the error caused by the disturbance over time.

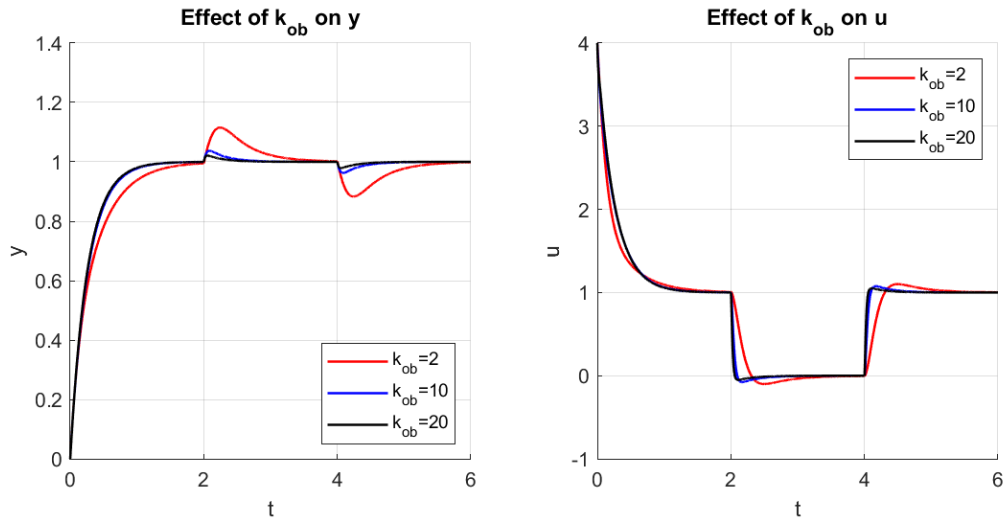


Figure 21: Effect of observer bandwidth $k_{ob} \in \{2, 10, 20\}$ on ADRC disturbance rejection performance.

Discussion of Figure 21: This figure isolates the effect of the observer bandwidth multiplier k_{ob} on the ADRC's ability to reject disturbances.

- **Subplot (c):** With a slow observer ($k_{ob} = 2$, red), the ESO is not fast enough to estimate and cancel the injected disturbance quickly, resulting in a large, slow deviation of y from the reference at $t = 2$s and $t = 4$s. With the nominal observer ($k_{ob} = 10$, blue), disturbance rejection is fast and the deviation is small. With the aggressive observer ($k_{ob} = 20$, black), the disturbance is rejected almost instantaneously with virtually no visible deviation in y.
- **Subplot (d):** The cost of faster disturbance rejection is visible in the control effort. Higher k_{ob} produces sharper, larger spikes in u at the moments of disturbance injection and removal. This confirms the fundamental trade-off in ESO tuning: a faster observer rejects disturbances more aggressively but demands more from the actuator and increases sensitivity to noise and unmodeled dynamics.

b) Continuous Second Order LADRC

1) Regenerating results shown in Fig.11 for K = 1, 0.1, 5, D = 1, 0.1, 10 and T = 1, 0.5, 3.5.

To simulate a second-order plant, the Simulink model was copied from the first-order case into a new file `ladrc_model_2nd.slx`. The plant transfer function block was updated to represent a second-order process:

$$G(s) = \frac{K}{T^2 s^2 + 2DTs + 1}$$

The ADRC MATLAB Function block was completely rewritten to implement a **third-order ESO** with three augmented states, matching the second-order plant structure. The controller gains and observer gains were derived from the course slide equations (42) and (43):

$$s^{CL} = \frac{-6}{T_{settle}}, \quad K_P = (s^{CL})^2, \quad K_D = -2s^{CL}$$

$$l_1 = 1-(z^{ESO})^3, \quad l_2 = \frac{3}{2T_s}(1-z^{ESO})^2(1+z^{ESO}), \quad l_3 = \frac{1}{T_s^2}(1-z^{ESO})^3$$

with the augmented discrete-time matrices:

$$A_d = 1T_sT_s^2/201T_s001, \quad B_d = b_0T_s^2/2b_0T_s0, \quad C_d = 100$$

The updated ADRC block code is:

```
function u_k = ladrc_dt(r_k, y_k, k_ob, u_sat_feedback, u_delayed_feedback)
    % tuning parameters, im writing the code for this block as following so
    % i only have to change these paramters every time
    T_settle = 5;
    b0 = 1;
    Ts = 0.001;

    % persistent variables holding states from step (k-1)
    persistent x_hat_k_minus_1 u_k_minus_1
    if isempty(x_hat_k_minus_1)
        x_hat_k_minus_1 = [0; 0; 0];
        u_k_minus_1 = 0;
    end

    % kp, kd, poles, L, Ad, Bd, Cd
    s_CL = -6 / T_settle;
    Kp   = s_CL^2;
    Kd   = -2 * s_CL;

    % ESO poles all at s_ESO
    s_ESO = k_ob * s_CL;
    z_ESO = exp(s_ESO * Ts);

    % 3rd order ESO gains from slide eq(43)
    l1 = 1 - z_ESO^3;
    l2 = (3 / (2*Ts))  * (1 - z_ESO)^2 * (1 + z_ESO);
    l3 = (1 / Ts^2)    * (1 - z_ESO)^3;
    Ld = [l1; l2; l3];

    % Ad, Bd, Cd from slide eq(35)
    Ad = [1,  Ts,  Ts^2/2;
          0,   1,  Ts;
          0,   0,  1];
    Bd = [b0*Ts^2/2;
          b0*Ts;
          0];
    Cd = [1, 0, 0];

    % observer would calculate current estimate x_hat(k)
    x_hat_k = (Ad - Ld*Cd*Ad) * x_hat_k_minus_1 + ...
              (Bd - Ld*Cd*Bd) * u_k_minus_1 + Ld * y_k;

    % our control Law would calculate current output u(k)
    u_k = (1/b0) * (Kp*(r_k - x_hat_k(1)) - Kd*x_hat_k(2) - x_hat_k(3));

    % update Memory for next Step (Current becomes k-1 for the next loop)
    x_hat_k_minus_1 = x_hat_k;
```

```
    u_k_minus_1 = u_delayed_feedback;
end
```

Listing 9: Updated MATLAB Function block for second-order LADRC

The automation script sweeps K, D, and T independently while keeping the other two at their nominal values, exactly as in the paper.

```
% set up
mdl = 'ladrc_model_2nd';
load_system(mdl);

% fixed nominal parameters
assignin('base', 'T_dead',     0);
assignin('base', 'T_dead_ESO', 0);
assignin('base', 'k_ob_const', 10);
assignin('base', 'Kd', 0);

% i tried matching colors with original figures for easier grading
color_nom = 'k'; lw_nom = 2.5;
color_low = 'b'; lw_low = 1.0;
color_hi  = 'r'; lw_hi  = 1.0;

%variation of K
assignin('base', 'D', 1);
assignin('base', 'T', 1);

K_values = [1, 0.1, 5];
colors_K = {color_nom, color_low, color_hi};
lws_K    = [lw_nom, lw_low, lw_hi];

for i = 1:length(K_values)
    assignin('base', 'K', K_values(i));
    out = sim(mdl, 'StopTime', '15.0');
    subplot(1,2,1); hold on; grid on;
    plot(out.tout, out.y_out, 'Color', colors_K{i}, 'LineWidth', lws_K(i), ...
         'DisplayName', ['K = ', num2str(K_values(i))]);
    subplot(1,2,2); hold on; grid on;
    plot(out.tout, out.u_out, 'Color', colors_K{i}, 'LineWidth', lws_K(i), ...
         'DisplayName', ['K = ', num2str(K_values(i))]);
end
% (formatting and saving code follows same pattern throughout)
```

Listing 10: Automation script to regenerate Figure 11

Results

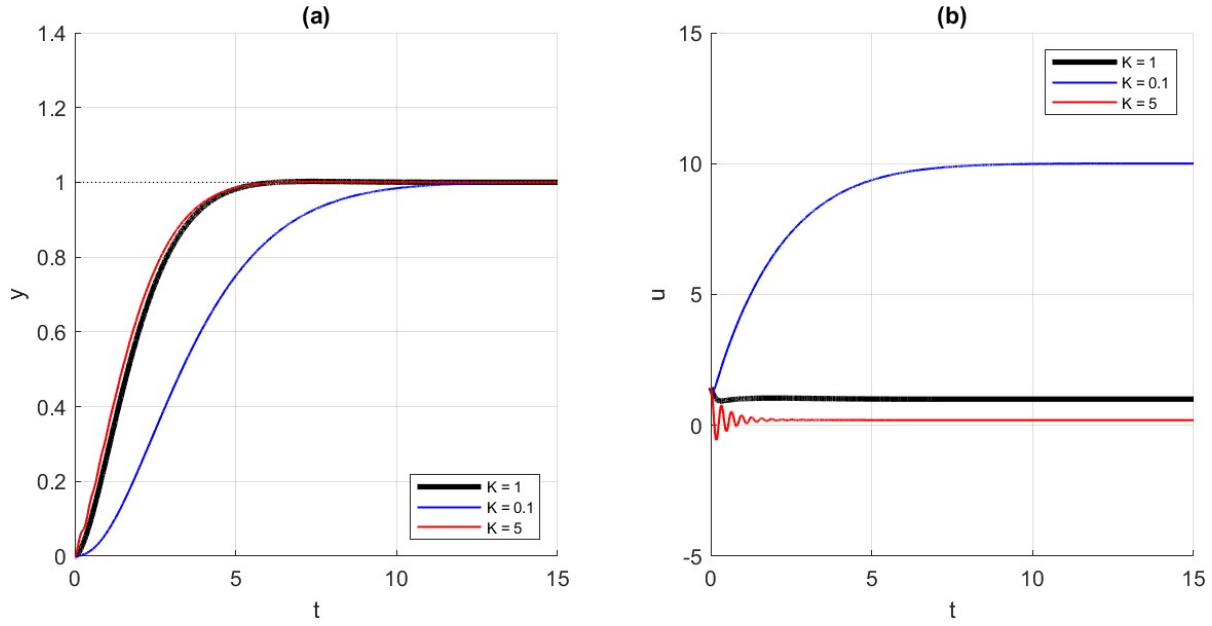


Figure 22: Replication of Fig. 11(a) and 11(b): Second-order ADRC response with fixed $D = 1$, $T = 1$ and varying K.

Discussion of Figure 22: The second-order ADRC demonstrates strong robustness against DC gain variations.

- **Subplot (a):** Despite a 50-fold range in plant gain ($K = 0.1$ to $K = 5$), all three output responses remain remarkably close to each other, converging to the unit reference within the designed settling time of $T_{settle} = 5$s. The ESO successfully estimates and cancels the gain mismatch as a lumped disturbance in all cases.
- **Subplot (b):** The control effort reveals the compensation mechanism. For low gain ($K = 0.1$, blue), the controller commands a very large initial effort to force the sluggish plant to respond. For high gain ($K = 5$, red), the effort is reduced significantly to prevent overshoot. The ESO adapts the output in real time without any explicit knowledge of K.

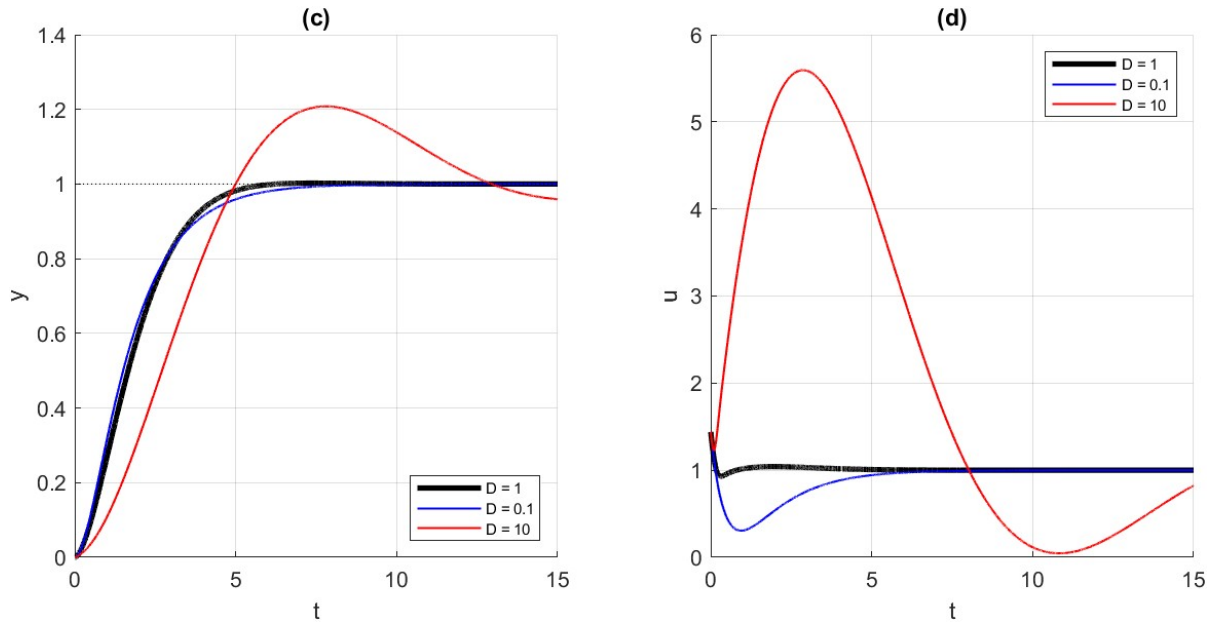


Figure 23: Replication of Fig. 11(c) and 11(d): Second-order ADRC response with fixed $K = 1$, $T = 1$ and varying D.

Discussion of Figure 23: Variations in the damping factor D are handled effectively by the second-order ADRC.

- **Subplot (c):** Even for the extreme case of $D = 10$ (heavily overdamped plant, red), the closed-loop response remains well-behaved with only slight overshoot becoming visible. The underdamped case ($D = 0.1$, blue) also tracks the reference cleanly, as the ESO absorbs the oscillatory plant dynamics into the disturbance estimate $\hat{x}_3$.
- **Subplot (d):** The control effort remains moderate across all cases, confirming that the observer effectively compensates for damping variations without demanding excessive actuator action.

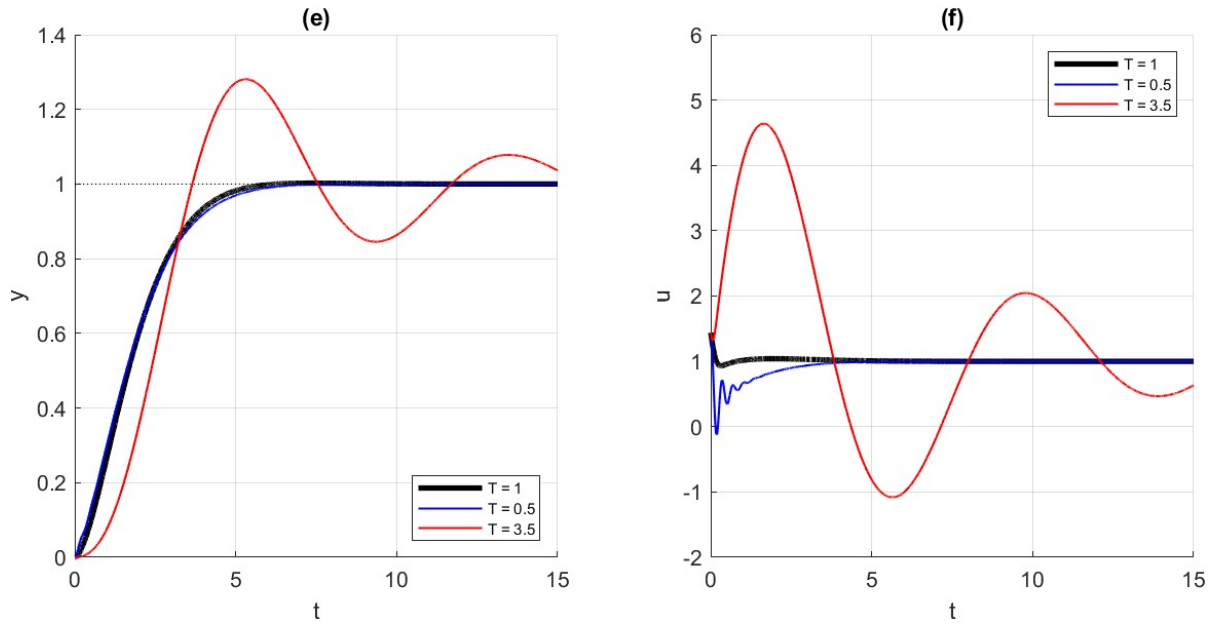


Figure 24: Replication of Fig. 11(e) and 11(f): Second-order ADRC response with fixed $K = 1$, $D = 1$ and varying T.

Discussion of Figure 24: Time constant variations present a slightly more challenging scenario for the second-order ADRC compared to K and D variations.

- **Subplot (e):** For larger values of T ($T = 3.5$, red), mild oscillations and increased overshoot become visible compared to the nominal case. This is consistent with the paper's observation that time constant changes affect the second-order ADRC more than gain or damping changes, as the plant poles shift significantly and the ESO must work harder to track the resulting dynamics.
- **Subplot (f):** The control effort for large T shows a sustained elevated value as the ESO continuously compensates for the slow plant dynamics. For small T ($T = 0.5$, blue), the effort decays quickly once the fast plant reaches the reference.

2) Regenerating results shown in Fig.12 for T = 1, 0.5, 3.5.

This experiment repeats the T variation study from part 1 across three different ESO pole locations: $s^{ESO} \in \{100, 10, 5\} \cdot s^{CL}$. No Simulink changes were needed — only k_{ob_const} was swept via `assignin` in the automation script.

```
% set up
mdl = 'ladrc_model_2nd';
load_system(mdl);

% fixed nominal parameters
assignin('base', 'K', 1);
assignin('base', 'D', 1);
assignin('base', 'T_dead',     0);
assignin('base', 'T_dead_ESO', 0);
assignin('base', 'Kd', 0);

```

```
% i tried matching colors with original figures for easier grading
color_nom = 'k'; lw_nom = 2.5;
color_low = 'b'; lw_low = 1.0;
color_hi  = 'r'; lw_hi  = 1.0;

T_values     = [1, 0.5, 3.5];
colors       = {color_nom, color_low, color_hi};
lws          = [lw_nom, lw_low, lw_hi];

k_ob_vals    = [100, 10, 5];
fig_labels   = {'a','c','e'};
fig_labels_u = {'b','d','f'};

for ki = 1:length(k_ob_vals)
    assignin('base', 'k_ob_const', k_ob_vals(ki));
    fig = figure('Name', sprintf('Fig12 k_ob=%d', k_ob_vals(ki)), ...
                 'Position', [100,100,900,400]);
    for ti = 1:length(T_values)
        assignin('base', 'T', T_values(ti));
        % run
        out = sim(mdl, 'StopTime', '15.0');
        subplot(1,2,1); hold on; grid on;
        plot(out.tout, out.y_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
             'DisplayName', ['T = ', num2str(T_values(ti))]);
        subplot(1,2,2); hold on; grid on;
        plot(out.tout, out.u_out, 'Color', colors{ti}, 'LineWidth', lws(ti), ...
             'DisplayName', ['T = ', num2str(T_values(ti))]);
    end
    % (formatting and saving code follows)
end
```

Listing 11: Automation script to regenerate Figure 12

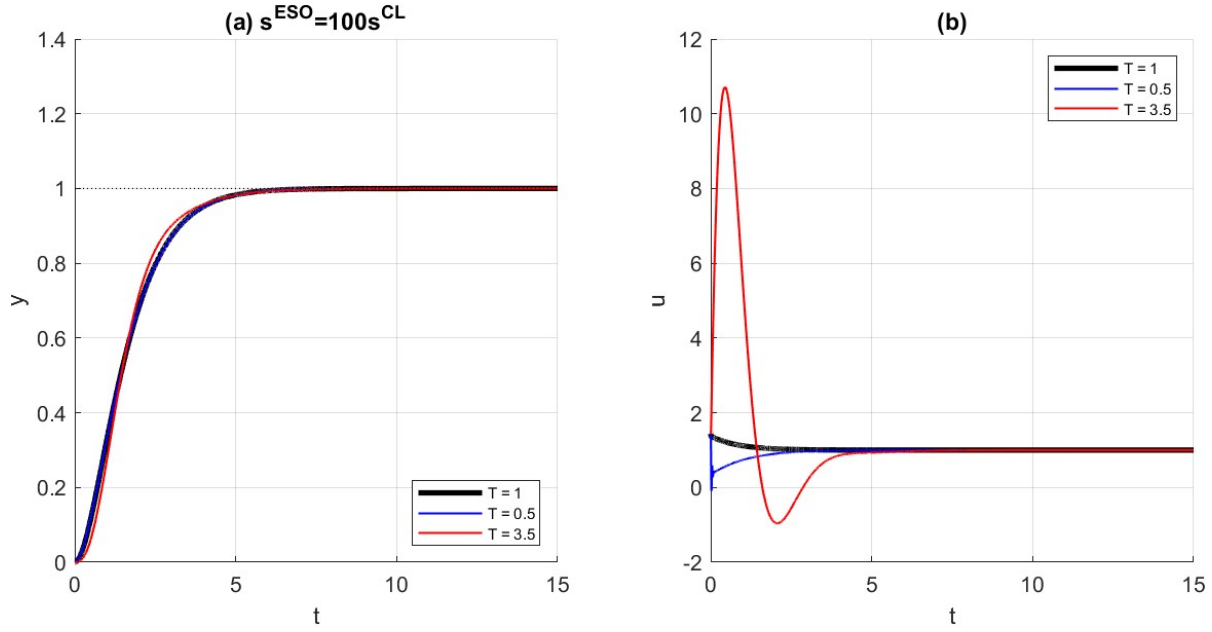


Figure 25: Replication of Fig. 12(a) and 12(b): $s^{ESO} = 100 \cdot s^{CL}$, varying T.

Discussion of Figure 25: With an aggressive observer ($k_{ob} = 100$), the ESO reacts almost instantaneously to any mismatch caused by T variations.

- **Subplot (a):** All three T values produce nearly identical output responses, tightly clustered around the nominal trajectory. Even the most challenging case ($T = 3.5$, red) shows negligible deviation, demonstrating near-ideal disturbance cancellation.
- **Subplot (b):** The cost is visible in the control effort, which shows large initial peaks especially for slow plants, as the aggressive ESO commands high u to compensate immediately.

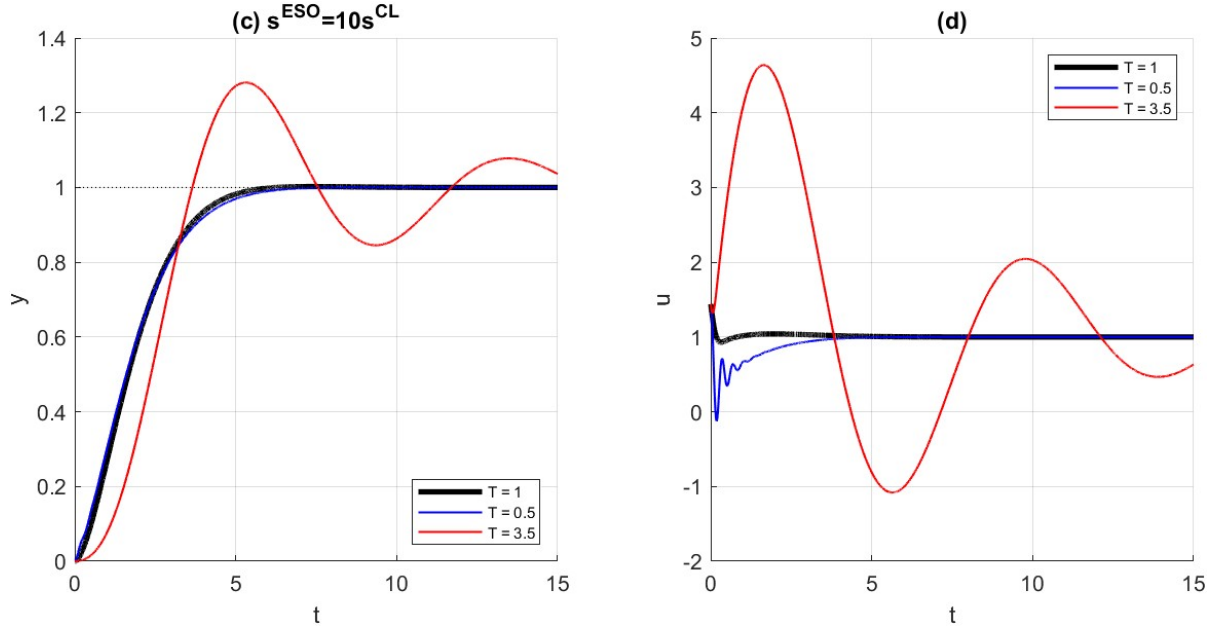


Figure 26: Replication of Fig. 12(c) and 12(d): $s^{ESO} = 10 \cdot s^{CL}$, varying T.

Discussion of Figure 26: This is the nominal tuning ($k_{ob} = 10$), the standard practical guideline of placing ESO poles ten times faster than the closed-loop poles.

- **Subplot (c):** A small but visible spread emerges between the three T values compared to $k_{ob} = 100$. The $T = 3.5$ case shows mild overshoot, consistent with the findings of part 1. The controller still achieves acceptable tracking for all three cases.
- **Subplot (d):** Control effort is noticeably more moderate, reflecting the reduced observer aggressiveness and resulting in a more practical actuator demand profile.

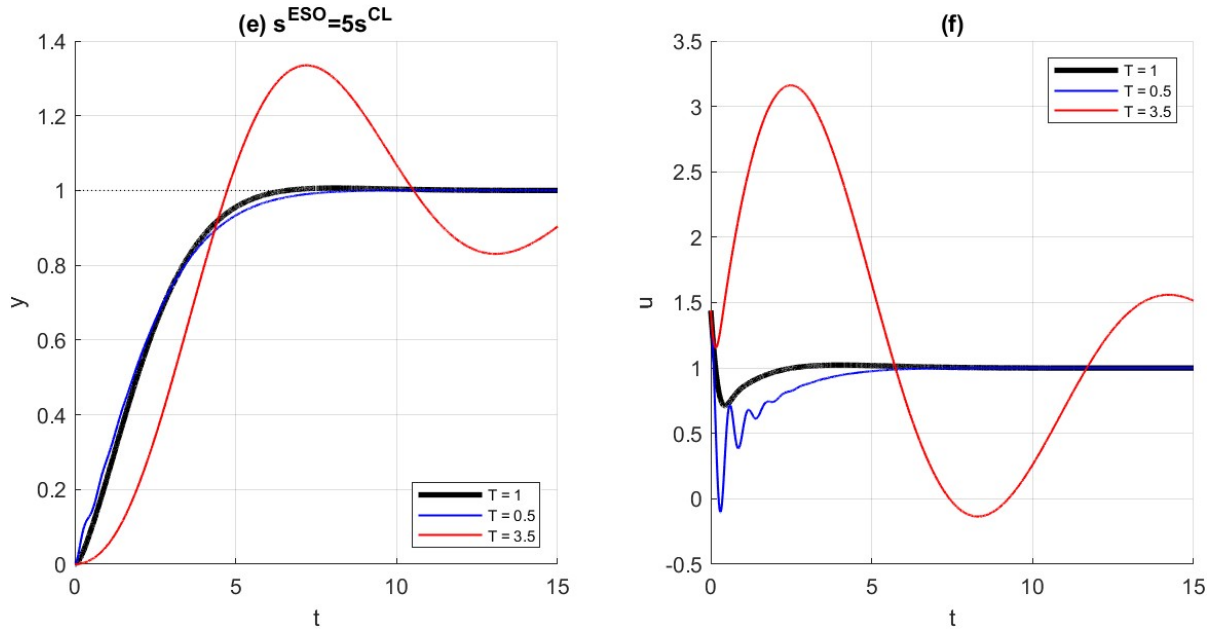


Figure 27: Replication of Fig. 12(e) and 12(f): $s^{ESO} = 5 \cdot s^{CL}$, varying T.

Discussion of Figure 27: With the slowest observer ($k_{ob} = 5$), the ESO struggles to keep up with plant dynamics, and the robustness degradation is clearly visible.

- **Subplot (e):** The spread between the three curves is now significant. For $T = 3.5$ (red), the response is noticeably slower and more oscillatory, as the observer cannot estimate and cancel the plant's slow dynamics fast enough.
- **Subplot (f):** The control effort is the smoothest of all three cases, confirming the fundamental trade-off: slower observers are gentler on actuators but sacrifice robustness against plant parameter variations.

3) Regenerating results shown in Fig.16 for K = 1, 0.2, 5, D = 1, 0.2, 5 and T = 1, 0.5, 3.5.

To compare the second-order ADRC against classical controllers, a PI and a PID controller were added as parallel loops in the Simulink model. The PI controller follows the paper's parameterization:

$$C_{PI}(s) = K_P + \frac{K_I}{s}, \quad K_I = \frac{2.55}{K \cdot T_{settle}} = 0.51, \quad K_P = K_I \cdot 1.5 \cdot D \cdot T = 0.765$$

The PID controller uses the two-pole two-zero structure from the paper:

$$C_{PID}(s) = K_I \cdot \frac{(1+T_{Z1}s)(1+T_{Z2}s)}{s(1+T_1 s)}, \quad K_I = 0.6,\ T_{Z1} = T_{Z2} = 1,\ T_1 = 0.2$$

implemented as a Transfer Function block with numerator [0.6 1.2 0.6] and denominator [0.2 1 0].

```
% set up
mdl = 'ladrc_model_2nd';
load_system(mdl);

% fixed nominal parameters
assignin('base', 'T_dead',     0);
assignin('base', 'T_dead_ESO', 0);
assignin('base', 'k_ob_const', 10);
assignin('base', 'Kd', 0);

% PI parameters from paper: KI=0.51, KP=0.765
assignin('base', 'Kp_pi',  0.765);
assignin('base', 'Ki_pi',  0.51);
assignin('base', 'Kp', 0.765);
assignin('base', 'Ki', 0.51);

% PID parameters from paper in parallel form
% implemented as Transfer Fcn block: num=[0.6 1.2 0.6], den=[0.2 1 0]

% i tried matching colors with original figures for easier grading
color_nom = 'k'; lw_nom = 2.5;
color_low = 'b'; lw_low = 1.0;
color_hi  = 'r'; lw_hi  = 1.0;

%k variation
assignin('base', 'D', 1);
assignin('base', 'T', 1);
K_values = [1, 0.2, 5];

for i = 1:length(K_values)
    assignin('base', 'K', K_values(i));
    out = sim(mdl, 'StopTime', '15.0');
    % plot y_out, y_pi_out, y_pid_out in subplots (a),(b),(c)
end

% D and T variations follow same pattern
```

Listing 12: Automation script to regenerate Figure 16

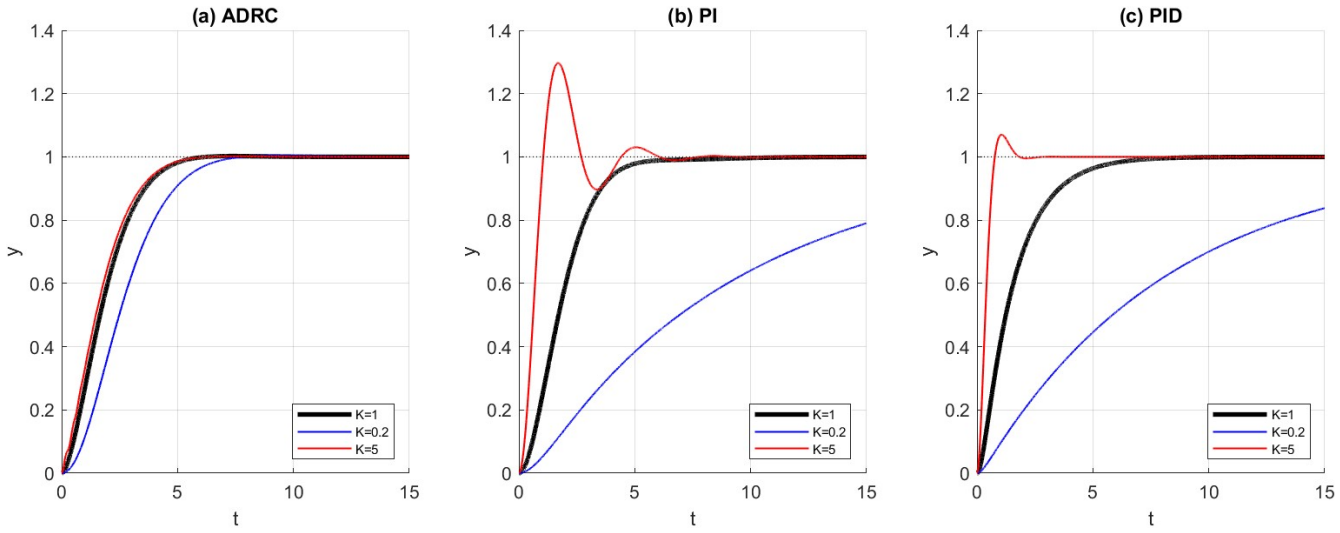


Figure 28: Replication of Fig. 16(a)(b)(c): ADRC vs PI vs PID with varying K.

Discussion of Figure 28: The three-column comparison directly reveals the superiority of ADRC over classical controllers under gain uncertainty.

- **Subplot (a) — ADRC:** All three K values produce nearly identical closed-loop responses, demonstrating near-perfect robustness. The ESO absorbs the gain variation as a disturbance and cancels it actively.
- **Subplot (b) — PI:** Significant spread is visible. For $K = 0.2$ (blue), the response is very slow as the fixed PI gains are no longer matched to the plant. For $K = 5$ (red), the response overshoots considerably, as the integral term accumulates error too aggressively.
- **Subplot (c) — PID:** Similar behavior to PI, though the derivative term provides some improvement in damping. The PID still shows substantial sensitivity to K variation compared to ADRC, confirming that model-based controllers are inherently more fragile under parametric uncertainty.

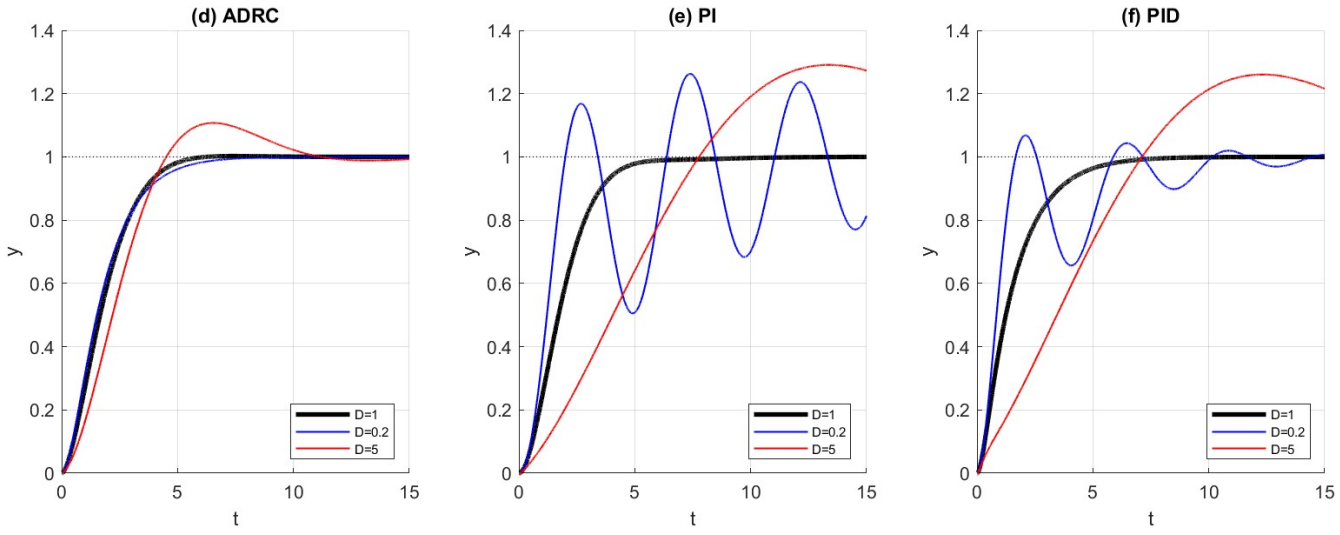


Figure 29: Replication of Fig. 16(d)(e)(f): ADRC vs PI vs PID with varying D.

Discussion of Figure 29: Damping variations expose a similar pattern across all three controllers.

- **Subplot (d) — ADRC:** Responses for all three D values remain tightly grouped, with only minor visible differences. The second-order ESO effectively tracks and compensates for changes in the damping term $2DT/T^2$ in the plant denominator.
- **Subplot (e) — PI:** For low damping ($D = 0.2$, blue), the PI controller struggles with oscillatory plant behavior, producing a sluggish and slightly oscillatory response. For high damping ($D = 5$, red), the response is stable but very slow.
- **Subplot (f) — PID:** The PID handles damping variations somewhat better than PI due to its derivative action, but still shows clear sensitivity compared to ADRC, particularly for the high damping case.

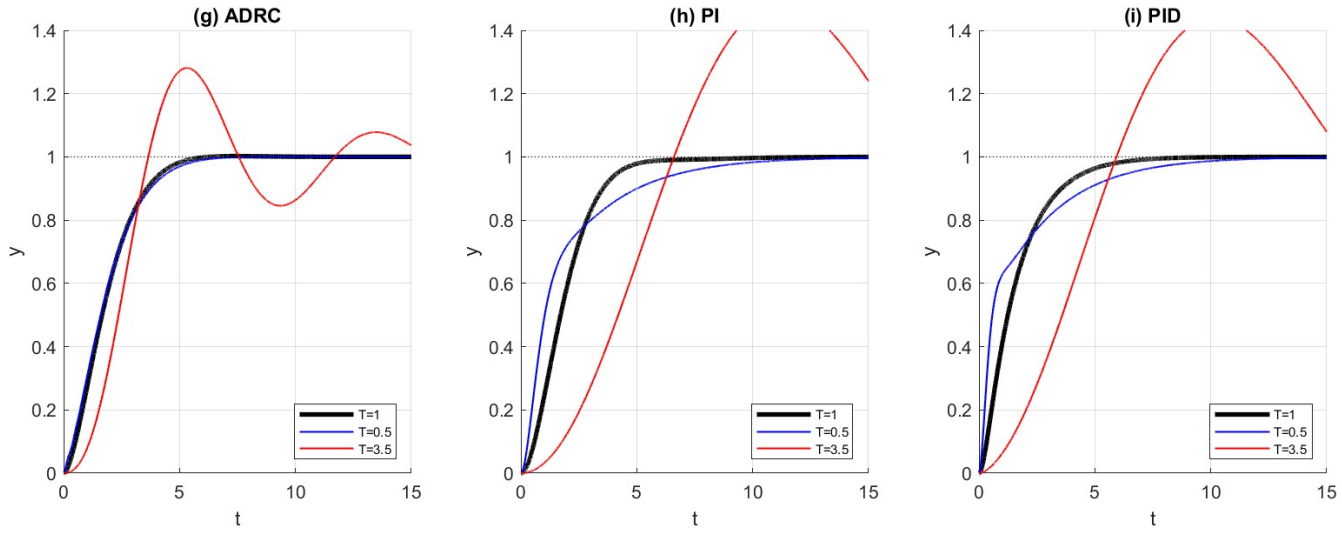


Figure 30: Replication of Fig. 16(g)(h)(i): ADRC vs PI vs PID with varying T.

Discussion of Figure 30: Time constant variation is the most challenging scenario for all three controllers.

- **Subplot (g) — ADRC:** Even with the most significant plant time constant change ($T = 3.5$, red), the ADRC maintains reasonable tracking performance, though with slightly more overshoot compared to the K and D variation cases. This is consistent with the observations from Fig.11 — T variations are the hardest for the ESO to compensate.
- **Subplot (h) — PI:** The PI controller shows dramatic sensitivity to T changes. For $T = 3.5$, the response is extremely slow and barely reaches the reference within 15 seconds, as the fixed integral gain is completely mismatched to the slow plant.
- **Subplot (i) — PID:** The PID shows marked improvement over PI for T variations due to pole-zero cancellation design, but still falls significantly short of ADRC performance. In all three comparisons, ADRC outperforms both classical controllers by a wide margin in terms of robustness to parametric uncertainty.

4) Regenerating results shown in Fig.17 and investigating effects of k_{ob} on disturbance rejection.

For this experiment, a pulse disturbance of amplitude $d = 0.5$ was applied at the plant input from $t = 15$ to $t = 25$ seconds using a Pulse Generator block (amplitude=0.5, period=40s, pulse width=25%, phase delay=15s). All three controllers were exposed to identical disturbance conditions simultaneously through separate Sum blocks placed before their respective plant blocks.

```
%set up
mdl = 'ladrc_model_2nd';
load_system(mdl);

% fixed nominal parameters
assignin('base', 'K', 1);
assignin('base', 'D', 1);
assignin('base', 'T', 1);
assignin('base', 'T_dead',     0);
assignin('base', 'T_dead_ESO', 0);
assignin('base', 'Kd', 0);

% PI from paper
assignin('base', 'Kp_pi', 0.765);
assignin('base', 'Ki_pi', 0.51);
assignin('base', 'Kp', 0.765);
assignin('base', 'Ki', 0.51);

assignin('base', 'k_ob_const', 10);

% run
out = sim(mdl, 'StopTime', '35.0');

% plot ADRC vs PI vs PID for y and u

% effect of kob on disturbance rejection
kob_vals = [2, 10, 20];
colors   = {'r', 'b', 'k'};

for i = 1:length(kob_vals)
    assignin('base', 'k_ob_const', kob_vals(i));
    % run and store results
    out = sim(mdl, 'StopTime', '35.0');
    t_out_all{i}  = out.tout;
    y_adrc_all{i} = out.y_out;
    u_adrc_all{i} = out.u_out;
end
% plot kob comparison
```

Listing 13: Automation script to regenerate Figure 17 and investigate k_{ob} effect

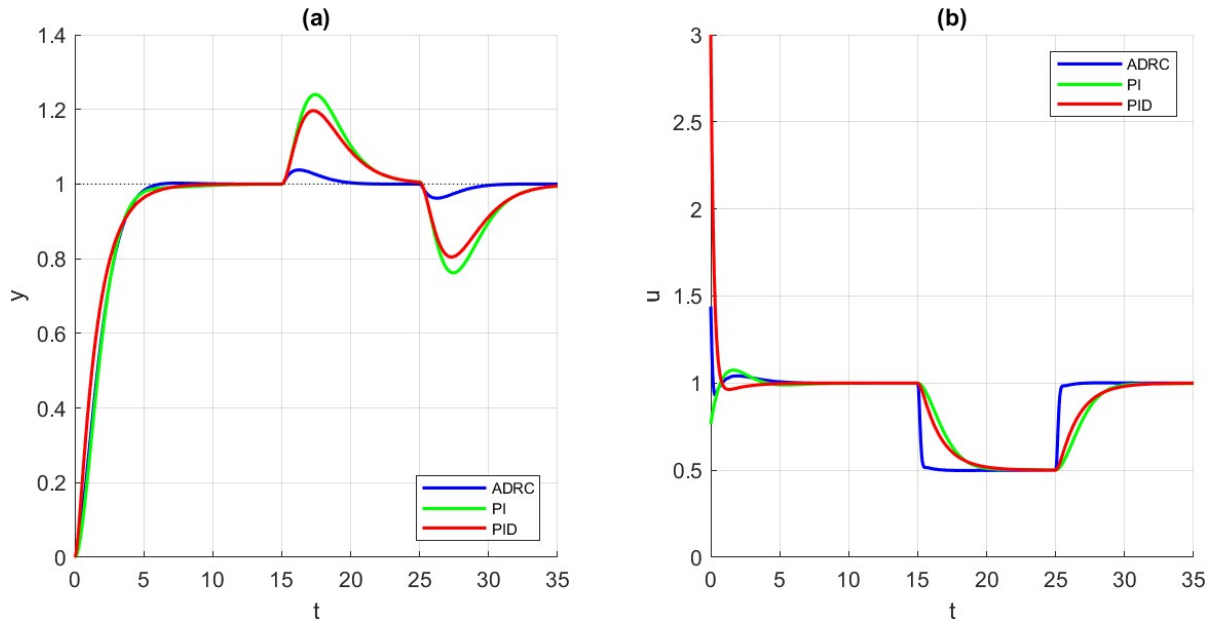


Figure 31: Replication of Fig. 17(a) and 17(b): Disturbance rejection comparison of second-order ADRC, PI, and PID. Disturbance $d = 0.5$ applied from $t = 15$ to $t = 25$.

Discussion of Figure 31: This figure provides the clearest demonstration of ADRC's fundamental advantage over classical control.

- **Subplot (a):** When the disturbance is applied at $t = 15$, the PI and PID controllers experience a significant deviation from the reference — the output rises above 1 and takes several seconds to recover, as they rely on error accumulation to identify and reject the disturbance. The ADRC response barely deviates from the reference, as the ESO immediately estimates the injected disturbance through the $\hat{x}_3$ state and the control law cancels it almost instantaneously. The same superior behavior is observed at $t = 25$ when the disturbance is removed.
- **Subplot (b):** The ADRC control effort shows brief, sharp spikes at $t = 15$ and $t = 25$ as it actively cancels the disturbance. The PI and PID control efforts show slower, sustained changes as they integrate the error over time — confirming that their disturbance rejection is fundamentally reactive rather than predictive.

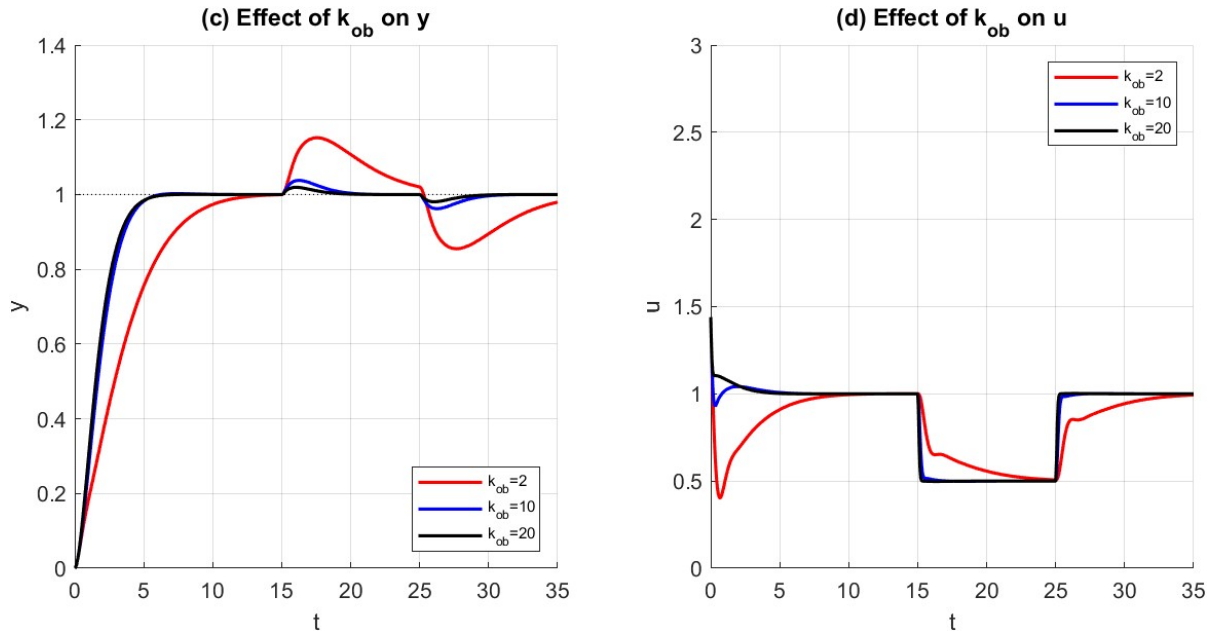


Figure 32: Effect of observer bandwidth $k_{ob} \in \{2, 10, 20\}$ on second-order ADRC disturbance rejection performance.

Discussion of Figure 32: The k_{ob} sweep confirms the direct relationship between observer bandwidth and disturbance rejection speed, exactly as predicted by theory.

- **Subplot (c):** With a slow observer ($k_{ob} = 2$, red), the ADRC takes several seconds to estimate and reject the disturbance, producing a large and slow deviation in y at both $t = 15$ and $t = 25$. With the nominal observer ($k_{ob} = 10$, blue), rejection is fast and the deviation is small. With the aggressive observer ($k_{ob} = 20$, black), disturbance rejection is nearly instantaneous and the output barely moves — matching the theoretical expectation that s^{ESO} directly controls how quickly the total disturbance state $\hat{x}_3$ tracks the true disturbance.
- **Subplot (d):** As expected, higher k_{ob} produces sharper and larger control effort spikes. This confirms the core trade-off in ADRC tuning: aggressive observers deliver superior disturbance rejection but at the cost of higher actuator demands and increased sensitivity to measurement noise and unmodeled dynamics.

c) Discrete First Order LADRC

Implementation Philosophy

As stated in the question, this part is implemented entirely through pure MATLAB m-file code, with no Simulink model involved. This is intentional — the resulting code directly mirrors what would be deployed on a microprocessor or embedded controller, where each iteration of the loop corresponds to one sampling period T_s. The simulation manually steps through time, updating the ESO and control law at each discrete step, exactly as a microcontroller would execute interrupt-driven control code.

Mapping the Code to Course Notation

The implementation follows the four-step design summary from the course slides precisely:

Step 1 — Modeling: The plant is assumed to have dominant first-order behavior $G(s) = \frac{K}{Ts+1}$. An estimate $b_0 \approx \frac{K}{T}$ is provided. In the code:

```
b0 = K/T;  % b0 = K/T = 1
```

Step 2 — Observer and Control Structure: The augmented state $\hat{x}(k) = [\hat{x}_1(k),\ \hat{x}_2(k)]^T$ is estimated using the current-form ESO from the course equation:

$$\hat{x}(k) = (A_d - L_d C_d A_d)\hat{x}(k-1) + (B_d - L_d C_d B_d)u(k-1) + L_d y(k)$$

and the proportional control law:

$$u(k) = \frac{1}{b_0}\left(K_P(v(k) - \hat{x}_1(k)) - \hat{x}_2(k)\right)$$

In the code:

```
% observer would calculate current estimate x_hat(k)
x_hat = (Ad - Ld*Cd*Ad)*x_hat + (Bd - Ld*Cd*Bd)*u_k + Ld*y_meas;

% our control law would calculate current output u(k)
u_k = (1/b0)*(Kp*(r_k - x_hat(1)) - x_hat(2));
```

Step 3 — Controller Gain: K_P is set using the settling time formula from the course:

$$K_P = \frac{4}{T_{settle}}, \quad s^{CL} = -K_P$$

In the code:

```
s_CL = -4 / T_settle;
Kp   = -s_CL;
```

Step 4 — Observer Gains: The ESO poles are placed at $s^{ESO} = k_{ob} \cdot s^{CL}$ in the continuous domain, mapped to the z-domain via $z^{ESO} = e^{s^{ESO} \cdot T_s}$, and the discrete observer gains are computed from the course equations:

$$l_1 = 1 - (z^{ESO})^2, \qquad l_2 = \frac{1}{T_s} \cdot (1 - z^{ESO})^2$$

In the code:

```
s_ESO = k_ob * s_CL;
z_ESO = exp(s_ESO * Ts);

% observer gains from course eq
l1 = 1 - z_ESO^2;
l2 = (1/Ts) * (1 - z_ESO)^2;
Ld = [l1; l2];
```

The discrete-time matrices A_d, B_d, C_d are obtained by ZOH discretization of the augmented integrator model:

$$A_d = 1T_s01, \quad B_d = b_0T_s0, \quad C_d = 10$$

In the code:

```
Ad = [1, Ts; 0, 1];
Bd = [b0*Ts; 0];
Cd = [1, 0];
```

The plant itself is also discretized analytically via ZOH for the first-order process:

```
% plant discretized via ZOH
a_p = exp(-Ts/T);
b_p = K*(1 - exp(-Ts/T));

% plant update each step
y_p = a_p*y_p + b_p*u_k;
```

Gaussian measurement noise with variance σ^2_{noise} is added to the plant output before it reaches the ESO, mimicking real sensor noise:

```
y_meas = y_p + sqrt(noise_var)*randn();
```

1) Regenerating results shown in Fig.18

The script sweeps $T_s \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$ with fixed $k_{ob} = 5$ and $\sigma^2_{noise} = 0.0001$, running a complete discrete-time simulation loop for each sample time.

```
% Q1_c_1 - Fig18 - Effect of sample time on discrete ADRC
%% Fig18 - pure MATLAB discrete simulation, no Simulink needed

% fixed parameters from paper
K          = 1;
T          = 1;
T_settle   = 1;
b0         = K/T;
k_ob       = 5;
noise_var  = 0.0001;
t_end      = 2.0;

Ts_values = [0.01, 0.02, 0.05, 0.1, 0.2];

% i tried matching colors with original figures for easier grading
colors = {'k','b','g','r','c'};
lws    = [2.5, 1, 1, 1, 1];

fig = figure('Name','Fig18 - Sample Time Effect','Position',[100,100,900,400]);

for idx = 1:length(Ts_values)
    Ts = Ts_values(idx);
    t  = 0:Ts:t_end;
    N  = length(t);

    % controller gains
    s_CL  = -4 / T_settle;
    Kp    = -s_CL;

    % ESO poles
    s_ESO = k_ob * s_CL;
```

```
    z_ESO = exp(s_ESO * Ts);

    % observer gains from course eq
    l1 = 1 - z_ESO^2;
    l2 = (1/Ts) * (1 - z_ESO)^2;
    Ld = [l1; l2];

    % discrete matrices
    Ad = [1, Ts; 0, 1];
    Bd = [b0*Ts; 0];
    Cd = [1, 0];

    % plant discretized via ZOH
    a_p = exp(-Ts/T);
    b_p = K*(1 - exp(-Ts/T));

    % initialise
    x_hat = [0; 0];
    u_k   = 0;
    y_p   = 0;
    y_arr = zeros(1,N);
    u_arr = zeros(1,N);

    for k = 1:N
        % measurement with noise
        y_meas = y_p + sqrt(noise_var)*randn();

        % observer would calculate current estimate x_hat(k)
        x_hat = (Ad - Ld*Cd*Ad)*x_hat + (Bd - Ld*Cd*Bd)*u_k + Ld*y_meas;

        % our control law would calculate current output u(k)
        r_k = 1;
        u_k = (1/b0)*(Kp*(r_k - x_hat(1)) - x_hat(2));

        % plant update
        y_p = a_p*y_p + b_p*u_k;

        y_arr(k) = y_meas;
        u_arr(k) = u_k;
    end

    subplot(1,2,1); hold on; grid on;
    plot(t, y_arr, 'Color', colors{idx}, 'LineWidth', lws(idx), ...
         'DisplayName', ['Ts = ', num2str(Ts)]);

    subplot(1,2,2); hold on; grid on;
    plot(t, u_arr, 'Color', colors{idx}, 'LineWidth', lws(idx), ...
         'DisplayName', ['Ts = ', num2str(Ts)]);
end

subplot(1,2,1);
plot([0 t_end],[1 1],'k:','HandleVisibility','off');
title('(a)'); xlabel('t'); ylabel('y');
axis([0 t_end 0 1.2]); legend('Location','southeast','FontSize',7);

subplot(1,2,2);
```

```
title('(b)'); xlabel('t'); ylabel('u');
axis([0 t_end 0 4]); legend('Location','northeast','FontSize',7);

% Code to save the figure would go here
fname = fullfile(save_dir, 'Fig18_SampleTime.jpg');
exportgraphics(fig, fname, 'Resolution', 150);
fprintf('Saved: %s\n', fname);
```

Listing 14: Automation script to regenerate Figure 18

Results

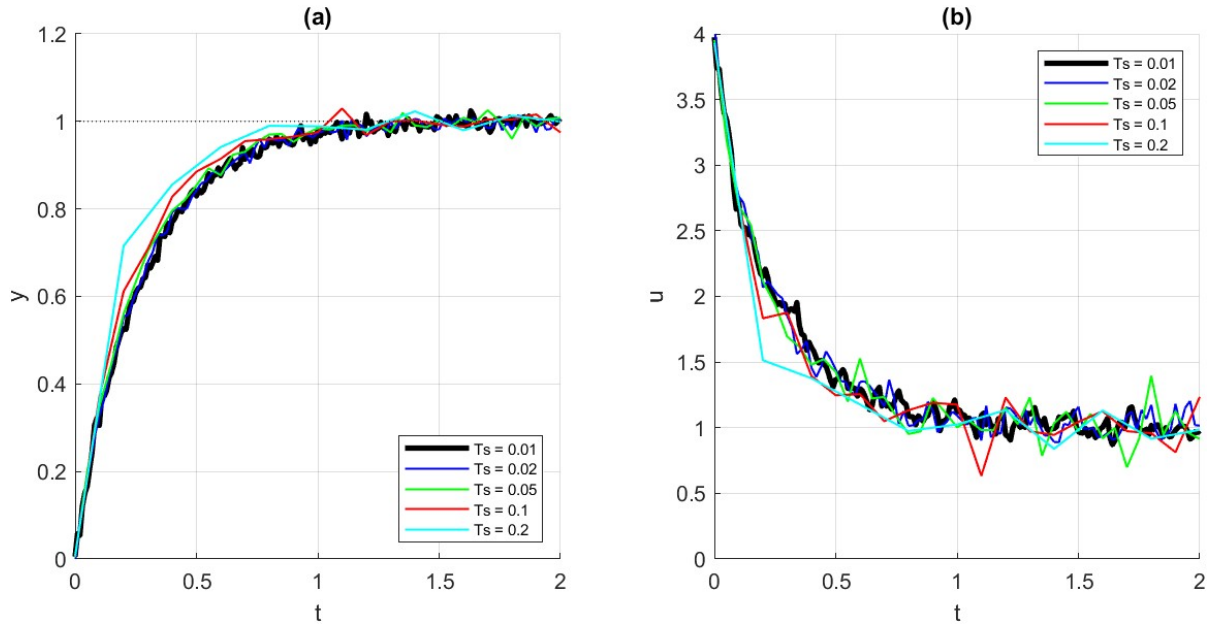


Figure 33: Replication of Fig. 18(a) and 18(b): Effect of sample time T_s on discrete-time ADRC. Fixed $k_{ESO} = 5$, $\sigma^2_{noise} = 0.0001$.

Discussion of Figure 33: These plots demonstrate how the choice of sampling period affects the discrete-time ADRC performance.

- **Subplot (a):** For small sample times ($T_s = 0.01$, black and $T_s = 0.02$, blue), the closed-loop response closely approximates the desired continuous-time first-order behavior, settling smoothly near $T_{settle} = 1$s. As T_s increases to 0.1 and 0.2, the response becomes progressively slower and more spread out, because the ZOH approximation deteriorates and the ESO cannot estimate the disturbance fast enough between samples. The noise level visible in y also increases with T_s, since each noisy sample is held for longer.
- **Subplot (b):** The control effort u shows increasingly coarse staircase behavior for larger T_s, as the controller updates less frequently. For $T_s = 0.2$ (cyan), the steps in u are clearly visible and the noise-induced chattering between updates is more pronounced. For small T_s, the control effort decays smoothly from its initial value of approximately 4 toward the steady-state value of 1.

2) Regenerating results shown in Fig.20

The script fixes $T_s = 0.01$ and $\sigma^2_{noise} = 0.0001$ and sweeps $k_{ob} \in \{5, 1, 2, 10\}$.

```
% Q1_c_2 - Fig20 - Effect of observer pole placement on discrete ADRC
%% Fig20 - pure MATLAB discrete simulation, no Simulink needed

% fixed parameters from paper
K          = 1;
T          = 1;
T_settle   = 1;
b0         = K/T;
Ts         = 0.01;
noise_var  = 0.0001;
t_end      = 2.0;

k_ob_vals = [5, 1, 2, 10];

% i tried matching colors with original figures for easier grading
colors = {'k','b','g','r'};
lws    = [2.5, 1, 1, 1];

fig = figure('Name','Fig20 - Observer Pole Effect','Position'
    ,[100,100,900,400]);

t = 0:Ts:t_end;
N = length(t);

% controller gains (fixed)
s_CL = -4 / T_settle;
Kp   = -s_CL;

% plant discretization (fixed)
a_p = exp(-Ts/T);
b_p = K*(1 - exp(-Ts/T));

% discrete matrices (fixed)
Ad = [1, Ts; 0, 1];
Bd = [b0*Ts; 0];
Cd = [1, 0];

for idx = 1:length(k_ob_vals)
    k_ob = k_ob_vals(idx);

    % ESO poles
    s_ESO = k_ob * s_CL;
    z_ESO = exp(s_ESO * Ts);

    % observer gains from course eq
    l1 = 1 - z_ESO^2;
    l2 = (1/Ts) * (1 - z_ESO)^2;
    Ld = [l1; l2];

    % initialise
    x_hat = [0; 0];
    u_k   = 0;
    y_p   = 0;
```

```
    y_arr = zeros(1,N);
    u_arr = zeros(1,N);

    for k = 1:N
        % measurement with noise
        y_meas = y_p + sqrt(noise_var)*randn();

        % observer would calculate current estimate x_hat(k)
        x_hat = (Ad - Ld*Cd*Ad)*x_hat + (Bd - Ld*Cd*Bd)*u_k + Ld*y_meas;

        % our control law would calculate current output u(k)
        r_k = 1;
        u_k = (1/b0)*(Kp*(r_k - x_hat(1)) - x_hat(2));

        % plant update
        y_p = a_p*y_p + b_p*u_k;

        y_arr(k) = y_meas;
        u_arr(k) = u_k;
    end

    subplot(1,2,1); hold on; grid on;
    plot(t, y_arr, 'Color', colors{idx}, 'LineWidth', lws(idx), ...
         'DisplayName', ['kESO = ', num2str(k_ob)]);

    subplot(1,2,2); hold on; grid on;
    plot(t, u_arr, 'Color', colors{idx}, 'LineWidth', lws(idx), ...
         'DisplayName', ['kESO = ', num2str(k_ob)]);
end

subplot(1,2,1);
plot([0 t_end],[1 1],'k:','HandleVisibility','off');
title('(a)'); xlabel('t'); ylabel('y');
axis([0 t_end 0 1.2]); legend('Location','southeast','FontSize',7);

subplot(1,2,2);
title('(b)'); xlabel('t'); ylabel('u');
axis([0 t_end 0 4]); legend('Location','northeast','FontSize',7);

% Code to save the figure would go here
fname = fullfile(save_dir, 'Fig20_ObserverPoles.jpg');
exportgraphics(fig, fname, 'Resolution', 150);
fprintf('Saved: %s\n', fname);
```

Listing 15: Automation script to regenerate Figure 20

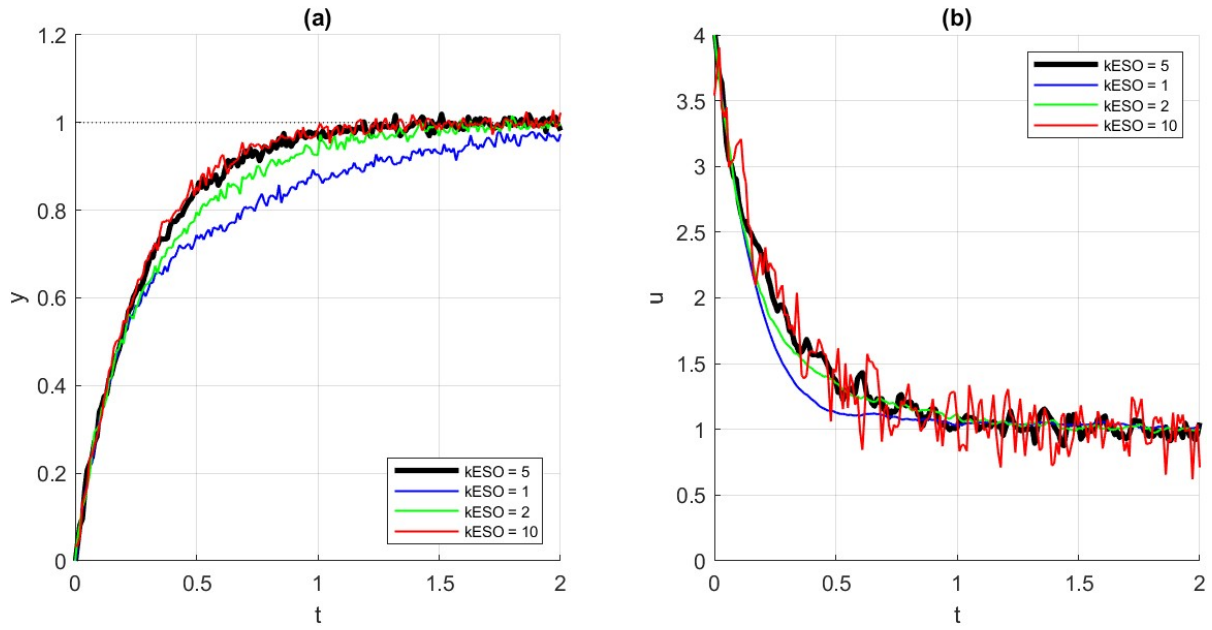


Figure 34: Replication of Fig. 20(a) and 20(b): Effect of observer pole placement k_{ESO} on discrete-time ADRC. Fixed $T_s = 0.01$, $\sigma^2_{noise} = 0.0001$.

Discussion of Figure 34: This figure reveals the fundamental trade-off between disturbance rejection speed and noise sensitivity in the discrete-time ESO.

- **Subplot (a):** For a slow observer ($k_{ESO} = 1$, blue), the closed-loop response is sluggish and fails to track the reference within the designed settling time — the ESO cannot estimate the total disturbance $\hat{x}_2$ fast enough to cancel it. As k_{ESO} increases to 2 (green) and 5 (black, nominal), the response progressively approaches the desired first-order behavior. For the aggressive observer ($k_{ESO} = 10$, red), the response is fast but the output y exhibits clearly visible noise-induced oscillations, since the ESO is tracking the noisy measurement so aggressively that it feeds measurement noise back into the control action.
- **Subplot (b):** The control effort confirms this trade-off. For $k_{ESO} = 1$ (blue), u decays smoothly with almost no noise visible — the slow observer filters out most of the measurement noise. For $k_{ESO} = 10$ (red), u shows significant high-frequency chattering throughout the simulation as the aggressive observer amplifies noise into the control signal. The nominal case $k_{ESO} = 5$ (black) provides the best balance — fast enough to achieve good disturbance rejection while keeping noise amplification at an acceptable level.

Q2 :

This problem confirms, through simulation, the theoretical LADRC stability ranges derived in the lecture notes (eqs. 56–70) for the open-loop unstable plant $G_3(s) = 1s^2 - 100s + 160$ of eq. (73). The actual plant is taken as $k_e G_3(s)$, a scalar gain k_e multiplying the nominal transfer function, while the LADRC controller is designed once around a fixed nominal $b_0 = 1$ and never "sees" k_e – exactly mirroring how a real controller would be tuned without knowledge of the true plant gain. The extended state observer

(eq. 57) and the PD-plus-feedforward control law (eq. 58, 59) are generalized from the first-order case using the same pole-placement pattern given in the slides: a double pole at $s^{CL} = -4/T_{settle}$ gives $k_1 = (s^{CL})^2$, $k_2 = -2s^{CL}$, and a triple pole at $s^{ESO} = k_{ob}s^{CL}$ gives $l_1 = -3s^{ESO}$, $l_2 = 3(s^{ESO})^2$, $l_3 = -(s^{ESO})^3$.

Because l_3 reaches on the order of 10^{10} for the fast tuning used in part (a), the closed loop is extremely stiff; a fixed-step explicit integrator is numerically unreliable here, so `ode15s` (an implicit stiff solver) is used instead, with an event function that halts integration as soon as the output clearly diverges. For each k_e, the true plant and the ESO are simulated as two separately-coupled state-space systems (coupled only through the measured output y and the control action u), responding to a unit step reference.

a) $T_{settle} = 0.1$**,** $k_{ob} = 100$**: stability for** $0.07 < k_e < 8.4$

A fine logarithmic sweep of k_e is simulated and the steady-state tracking error $|y - r|$ is recorded for each value (a diverging run is assigned a large sentinel error). The same controller is also run for three representative k_e values spanning below, inside, and just above the claimed range, to show the actual step responses.

b) $T_{settle} = 1$**,** $k_{ob} = 10$**: no stabilizing** k_e

The same sweep and the same three representative k_e values are simulated again, now with the slower tuning from part (b), to confirm that the closed loop diverges for every k_e tested.

```
%set up
b0 = 1;
blow_thresh = 1e6;

save_dir_a = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q3\a';
save_dir_b = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q3\b';
if ~exist(save_dir_a, 'dir'), mkdir(save_dir_a); end
if ~exist(save_dir_b, 'dir'), mkdir(save_dir_b); end

% part a
T_settle = 0.1; k_ob = 100;
s_CL = -4/T_settle; s_ESO = k_ob*s_CL;
k1 = s_CL^2; k2 = -2*s_CL;
l1 = -3*s_ESO; l2 = 3*s_ESO^2; l3 = -s_ESO^3;

t_end_a = 0.5;
t_end_a_demo = 0.45;
ke_sweep = logspace(-3, 2, 80);
err_a = zeros(size(ke_sweep));
for i = 1:length(ke_sweep)
    [t, Y, blew] = sim_core(ke_sweep(i), b0, k1, k2, l1, l2, l3, t_end_a,
    blow_thresh);
    if blew
        err_a(i) = blow_thresh;
    else
        tail = t > 0.8*t_end_a;
        err_a(i) = max(abs(Y(tail,1) - 1));
    end
```

```
end

fig1 = figure('name', 'part a - stability vs ke');
loglog(ke_sweep, err_a, 'b', 'linewidth', 1.2); hold on;
yline(1, 'k--'); xline(0.07, 'r--'); xline(8.4, 'r--'); hold off;
grid on; xlabel('k_e'); ylabel('steady-state |y-r| (log)');
title('part a - tracking error vs k_e, T_{settle}=0.1, k_{ob}=100');
legend('simulated error', 'reference scale', 'claimed bounds (0.07, 8.4)', '
    location', 'best');
saveas(fig1, fullfile(save_dir_a, 'partA_error_vs_ke.png'));

ke_demo_a = [0.055, 4.5, 8.55];
styles_a = {'-', '--', ':'};
fig2 = figure('name', 'part a - illustrative step responses'); hold on;
for i = 1:length(ke_demo_a)
    [t, Y, ~] = sim_core(ke_demo_a(i), b0, k1, k2, l1, l2, l3, t_end_a_demo,
    blow_thresh);
    plot(t, Y(:,1), styles_a{i}, 'linewidth', 1.6);
end
yline(1, 'k--'); hold off; grid on; ylim([-5 10]);
xlabel('time (s)'); ylabel('y(t)');
title('part a - step response for several k_e values');
legend([arrayfun(@(k) sprintf('k_e=%.2g', k), ke_demo_a, 'uniformoutput', false
    ), {'reference'}], 'location', 'best');
saveas(fig2, fullfile(save_dir_a, 'partA_step_responses.png'));

%part b
T_settle = 1; k_ob = 10;
s_CL = -4/T_settle; s_ESO = k_ob*s_CL;
k1 = s_CL^2; k2 = -2*s_CL;
l1 = -3*s_ESO; l2 = 3*s_ESO^2; l3 = -s_ESO^3;

t_end_b = 8;
t_end_b_demo = 6.5;
ke_sweep = logspace(-3, 4, 80);
err_b = zeros(size(ke_sweep));
for i = 1:length(ke_sweep)
    [t, Y, blew] = sim_core(ke_sweep(i), b0, k1, k2, l1, l2, l3, t_end_b,
    blow_thresh);
    if blew
        err_b(i) = blow_thresh;
    else
        tail = t > 0.8*t_end_b;
        err_b(i) = max(abs(Y(tail,1) - 1));
    end
end

fig3 = figure('name', 'part b - stability vs ke');
loglog(ke_sweep, err_b, 'b', 'linewidth', 1.2); hold on;
yline(1, 'k--'); hold off;
grid on; xlabel('k_e'); ylabel('steady-state |y-r| (log)');
title('part b - tracking error vs k_e, T_{settle}=1, k_{ob}=10 (no stabilizing
    k_e)');
legend('simulated error', 'reference scale', 'location', 'best');
saveas(fig3, fullfile(save_dir_b, 'partB_error_vs_ke.png'));

```

```
ke_demo_b = [0.055, 4.5, 8.55];
styles_b = {'-', '--', ':'};
fig4 = figure('name', 'part b - illustrative step responses'); hold on;
for i = 1:length(ke_demo_b)
    [t, Y, ~] = sim_core(ke_demo_b(i), b0, k1, k2, l1, l2, l3, t_end_b_demo,
   blow_thresh);
    plot(t, Y(:,1), styles_b{i}, 'linewidth', 1.6);
end
yline(1, 'k--'); hold off; grid on; ylim([-10 10]);
xlabel('time (s)'); ylabel('y(t)');
title('part b - step response for several k_e values');
legend([arrayfun(@(k) sprintf('k_e=%.2g', k), ke_demo_b, 'uniformoutput', false
   ), {'reference'}], 'location', 'best');
saveas(fig4, fullfile(save_dir_b, 'partB_step_responses.png'));

% closed-loop ode
function ds = closed_loop_ode(~, s, ke, b0, k1, k2, l1, l2, l3)
    p1=s(1); p2=s(2); xh1=s(3); xh2=s(4); xh3=s(5);
    r = 1; rdot = 0; rddot = 0;       % unit step reference
    u0 = k1*(r-xh1) + k2*(rdot-xh2) + rddot;   % eq.59
    u  = (u0 - xh3)/b0;                          % eq.58
    e_y = p1 - xh1;
    ds = [p2;
          100*p2 - 160*p1 + ke*u;      % true plant: ke*g3(s)
          xh2 + l1*e_y;
          xh3 + b0*u + l2*e_y;
          l3*e_y];
end

% stops integration early once the output clearly diverges
function [value, isterminal, direction] = blowup_event(~, s, thresh)
    value = thresh - abs(s(1));
    isterminal = 1;
    direction = 0;
end

% stiff closed-loop simulator (it handles the very large eso gain robustly)
function [t, Y, blew] = sim_core(ke, b0, k1, k2, l1, l2, l3, t_end, thresh)
    odefun = @(t,s) closed_loop_ode(t, s, ke, b0, k1, k2, l1, l2, l3);
    opts = odeset('Events', @(t,s) blowup_event(t,s,thresh), 'RelTol', 1e-7, '
   AbsTol', 1e-9);
    [t, Y, te] = ode15s(odefun, [0 t_end], zeros(5,1), opts);
    blew = ~isempty(te);
end
```

Listing 16: Simulation-based verification of LADRC stability bounds for $G_3(s)$

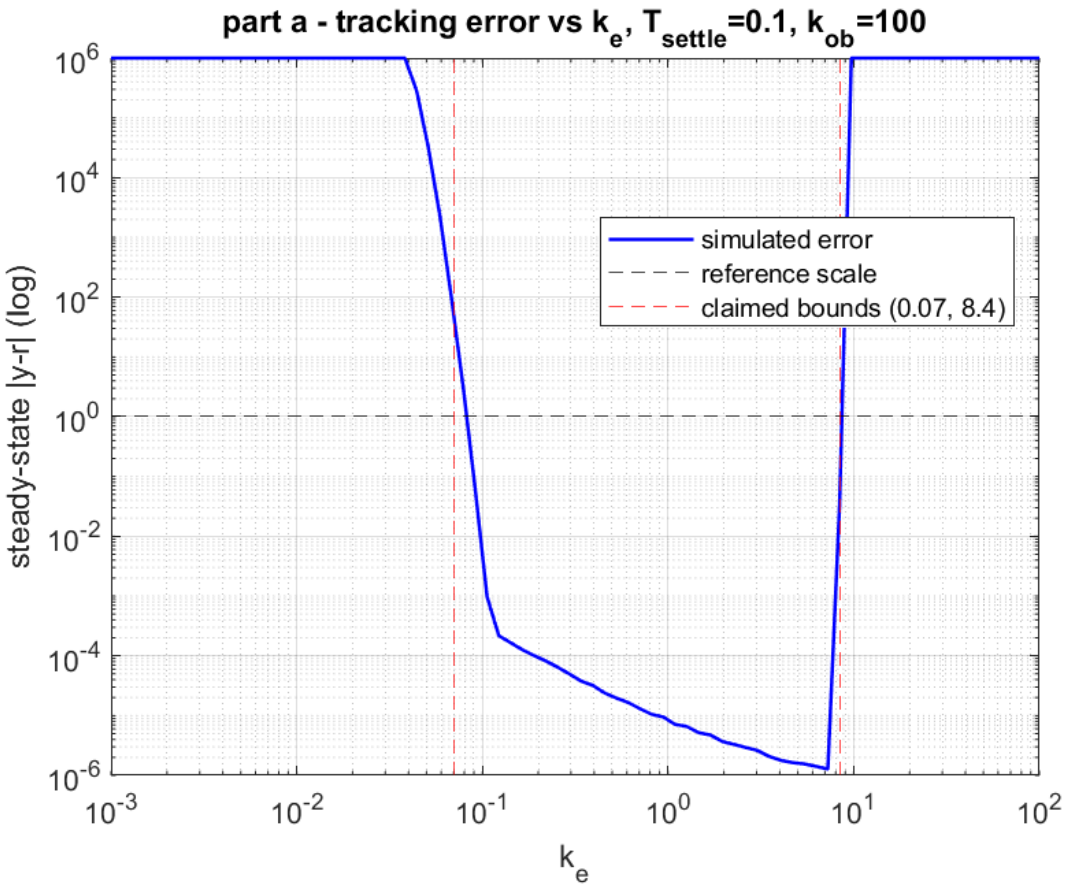


Figure 35: Part a: simulated steady-state tracking error vs. k_e, $T_{settle} = 0.1$, $k_{ob} = 100$. The red dashed lines mark the theoretically claimed bounds $k_e = 0.07$ and $k_e = 8.4$.

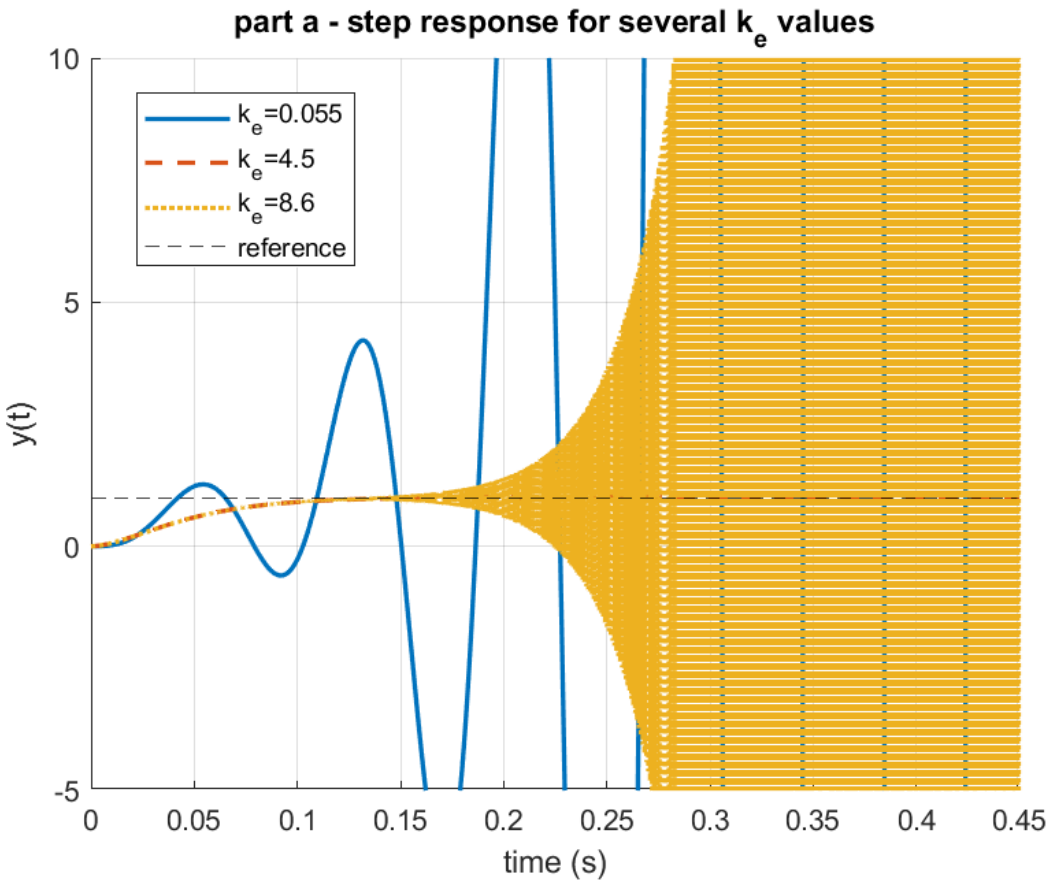


Figure 36: Part a: step response for $k_e = 0.055$ (below the lower bound), $k_e = 4.5$ (inside the range), and $k_e = 8.55$ (above the upper bound).

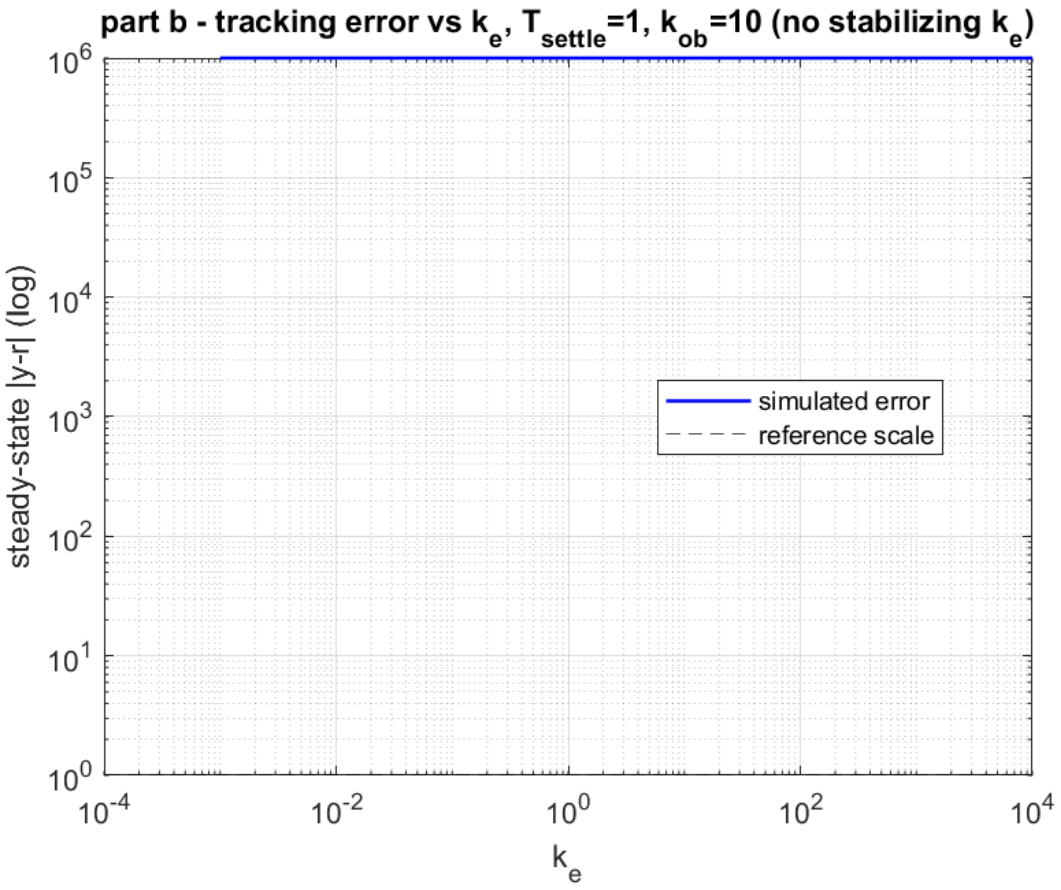


Figure 37: Part b: simulated steady-state tracking error vs. k_e, $T_{settle} = 1$, $k_{ob} = 10$.

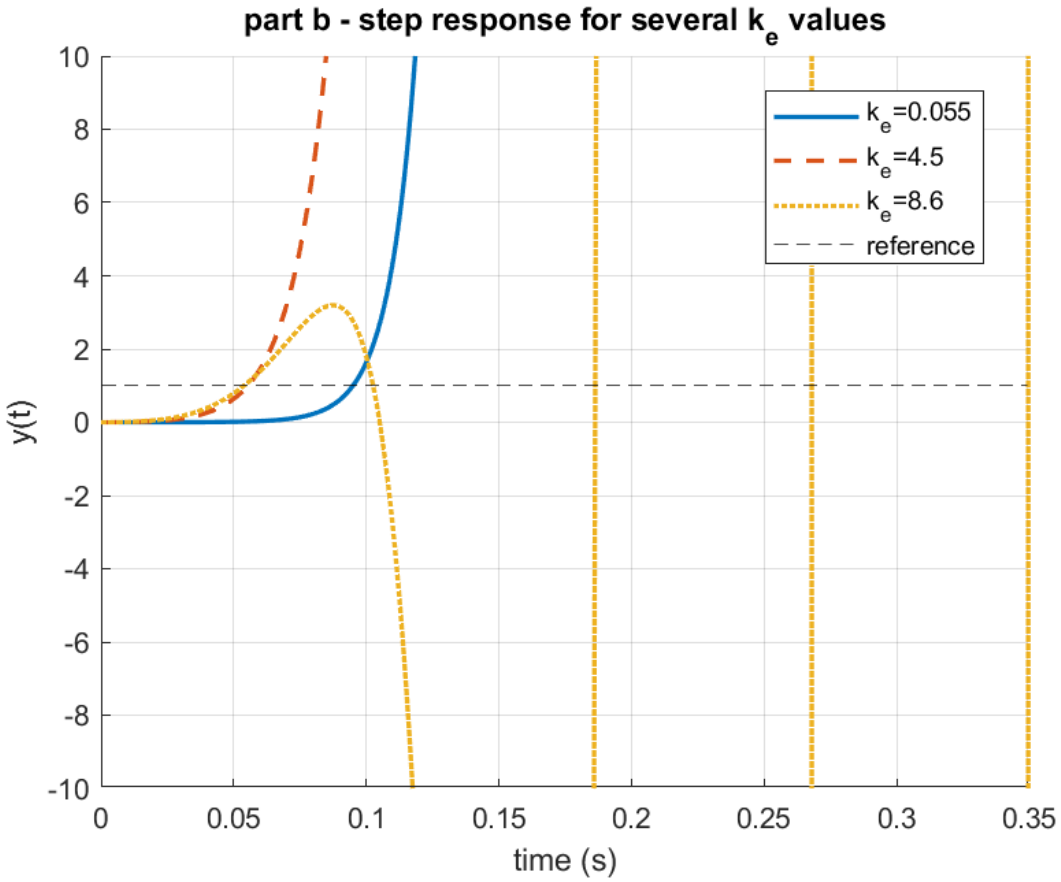


Figure 38: Part b: step response for the same three k_e values as Fig. 36, now with the slower tuning of part (b).

Discussion of Figures (part a):

- **first fig:** The simulated tracking error sits orders of magnitude above the reference scale for k_e below roughly 0.07 and above roughly 8.4, and drops to a small, bounded value strictly between these two thresholds – exactly matching the theoretically claimed stability interval $0.07 < k_e < 8.4$ from eq. (73)'s analysis. The transition at both ends is sharp, consistent with a genuine stability boundary rather than a gradual degradation.

- **second fig:** For $k_e = 0.055$ (solid, below the lower bound) the output grows in an oscillation of increasing amplitude, a textbook signature of a pair of complex poles crossing into the right half-plane. For $k_e = 8.55$ (dotted, just above the upper bound) the output rises smoothly at first – closely tracking the $k_e = 4.5$ case – before turning over and diverging sharply, showing the instability here is not immediate but develops once the mismatch between the assumed b_0 and the true gain k_e overwhelms the observer's correction. For $k_e = 4.5$ (dashed, well inside the range) the response converges cleanly to the reference with no overshoot. Notably, the $k_e = 4.5$ and the early portion of the $k_e = 8.55$ curve nearly coincide: this is the ESO successfully masking the gain mismatch right up until the instability at the upper boundary takes over.

Discussion of Figures (part b):

- **the first fig:** The simulated error sits at the diverging sentinel value across the entire swept range, from $k_e = 10^{-3}$ to $k_e = 10^4$ – seven orders of magnitude – with no window of boundedness anywhere. This confirms that for $T_{settle} = 1, k_{ob} = 10$, there genuinely is no value of k_e that stabilizes the closed loop.

- **second fig:** All three representative k_e values diverge, but at very different rates and in different ways: $k_e = 4.5$ (dashed) diverges fastest and most aggressively, $k_e = 8.55$ (dotted) rises with a small overshoot bump before turning over and diverging in the opposite direction, and $k_e = 0.055$ (solid) diverges more slowly, only crossing the plotted range near the end of the window. The slower T_{settle} and weaker observer ($k_{ob} = 10$ instead of 100) are evidently insufficient to keep the estimation/control error dynamics of eq. (69) Hurwitz for this particular plant, regardless of how the actual gain k_e is chosen.

c) $T_{settle} = 1$: stability region in the $k_e - k_{ob}$ plane

In this section, the stability of the closed-loop system is investigated across a 2D grid of k_e and k_{ob} values for a fixed $T_{settle} = 1$. By evaluating the eigenvalues of the 5×5 closed-loop state matrix for each (k_e, k_{ob}) pair, the stable region is numerically mapped out. To verify this theoretically obtained map, two random points are chosen from within the stable region ($k_e = 2.217, k_{ob} = 37.494$ and $k_e = 3.229, k_{ob} = 36.241$) and two from outside the stable region ($k_e = 2.603, k_{ob} = 20.735$ and $k_e = 4.410, k_{ob} = 15.096$). Nonlinear simulation using the stiff solver `ode15s` is then run for each point. The console output and step responses confirm the eigenvalue-based predictions perfectly.

A particularly interesting point upon this figure is that the stable region exhibits a strict lower bound for the observer bandwidth parameter, requiring approximately $k_{ob}23$ to achieve stability for any value of k_e at this T_{settle}. This provides a clear mathematical explanation for the findings in part (b): when k_{ob} was set to 10, the operating line lay entirely below the stable region pocket, meaning no k_e could possibly stabilize the system.

```
%set up
b0 = 1;
T_settle = 1;

save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q3\c';
if ~exist(save_dir, 'dir'), mkdir(save_dir); end

% part c
ke_grid  = linspace(0.001, 6, 250);
kob_grid = linspace(1, 40, 250);
stable_map = false(length(kob_grid), length(ke_grid));

for i = 1:length(kob_grid)
    [k1, k2, l1, l2, l3] = ladrc_gains(T_settle, kob_grid(i));
    for j = 1:length(ke_grid)
        stable_map(i,j) = is_stable(ke_grid(j), k1, k2, l1, l2, l3, b0);
    end
end

fig1 = figure('name', 'part c - stability region');
imagesc(ke_grid, kob_grid, stable_map); set(gca, 'ydir', 'normal');
colormap([1 1 1; 0.5 0.5 0.5]); hold on;
```

```
plot(ke_grid, kob_grid(end)/ke_grid(end)*ke_grid, 'k--');   % k_ob = k_e line, for reference
hold off;
xlabel('k_e'); ylabel('k_{ob}');
title('part c - stable region of LADRC in k_e - k_{ob} plane, T_{settle}=1');
saveas(fig1, fullfile(save_dir, 'partC_stability_region.png'));

% pick 2 random points inside and 2 outside the stable region, then verify by simulation
[in_idx_row, in_idx_col]   = find(stable_map);
[out_idx_row, out_idx_col] = find(~stable_map);

rng(7);
pick_in  = randperm(length(in_idx_row), 2);
pick_out = randperm(length(out_idx_row), 2);

pts_ke  = [ke_grid(in_idx_col(pick_in)),  ke_grid(out_idx_col(pick_out))];
pts_kob = [kob_grid(in_idx_row(pick_in)), kob_grid(out_idx_row(pick_out))];
labels  = {'inside #1', 'inside #2', 'outside #1', 'outside #2'};

fig2 = figure('name', 'part c - chosen points on stability map');
imagesc(ke_grid, kob_grid, stable_map); set(gca, 'ydir', 'normal');
colormap([1 1 1; 0.5 0.5 0.5]); hold on;
plot(pts_ke, pts_kob, 'r*', 'markersize', 10, 'linewidth', 1.5);
for i = 1:4
    text(pts_ke(i)+0.05, pts_kob(i), labels{i}, 'color', 'r', 'fontweight', 'bold');
end
hold off; xlabel('k_e'); ylabel('k_{ob}');
title('part c - 2 random points inside and 2 outside the stable region');
saveas(fig2, fullfile(save_dir, 'partC_chosen_points.png'));

fig3 = figure('name', 'part c - step response at the 4 chosen points'); hold on;
styles = {'-', '--', '-.', ':'};
eig_verdict = {'stable', 'stable', 'unstable', 'unstable'};   % by construction : pts 1,2 inside, 3,4 outside
for i = 1:4
    [k1, k2, l1, l2, l3] = ladrc_gains(T_settle, pts_kob(i));
    [t, Y, blew] = sim_core(pts_ke(i), b0, k1, k2, l1, l2, l3, 5, 1e6);
    plot(t, Y(:,1), styles{i}, 'linewidth', 1.4);
    sim_verdict = ternary(~blew, 'stable', 'unstable');
    fprintf('%s: ke=%.3f, kob=%.3f -> eigenvalue test: %s, simulation: %s\n', ...
        labels{i}, pts_ke(i), pts_kob(i), eig_verdict{i}, sim_verdict);
end
yline(1, 'k--'); hold off; grid on; ylim([-5 10]); xlim([0 5]);
xlabel('time (s)'); ylabel('y(t)');
title('part c - closed-loop step response at the 4 chosen (k_e,k_{ob}) points');
legend([labels, {'reference'}], 'location', 'best');
saveas(fig3, fullfile(save_dir, 'partC_verification_steps.png'));

function [k1, k2, l1, l2, l3] = ladrc_gains(T_settle, k_ob)
    s_CL  = -4/T_settle;
    s_ESO = k_ob*s_CL;
```

```
    k1 = s_CL^2; k2 = -2*s_CL;
    l1 = -3*s_ESO; l2 = 3*s_ESO^2; l3 = -s_ESO^3;
end

function tf = is_stable(ke, k1, k2, l1, l2, l3, b0)
    M = [   0,      1,        0,          0,      0;
         -160,    100,    -ke*k1/b0, -ke*k2/b0, -ke/b0;
           l1,      0,       -l1,        1,      0;
           l2,      0,  -(k1+l2),     -k2,      0;
           l3,      0,       -l3,        0,      0];
    tf = all(real(eig(M)) < 0);
end

function ds = closed_loop_ode(~, s, ke, b0, k1, k2, l1, l2, l3)
    p1=s(1); p2=s(2); xh1=s(3); xh2=s(4); xh3=s(5);
    r = 1; rdot = 0; rddot = 0;
    u0 = k1*(r-xh1) + k2*(rdot-xh2) + rddot;
    u  = (u0 - xh3)/b0;
    e_y = p1 - xh1;
    ds = [p2; 100*p2-160*p1+ke*u; xh2+l1*e_y; xh3+b0*u+l2*e_y; l3*e_y];
end

function [value, isterminal, direction] = blowup_event(~, s, thresh)
    value = thresh - abs(s(1)); isterminal = 1; direction = 0;
end

function [t, Y, blew] = sim_core(ke, b0, k1, k2, l1, l2, l3, t_end, thresh)
    odefun = @(t,s) closed_loop_ode(t, s, ke, b0, k1, k2, l1, l2, l3);
    opts = odeset('Events', @(t,s) blowup_event(t,s,thresh), 'RelTol', 1e-7, '
   AbsTol', 1e-9);
    [t, Y, te] = ode15s(odefun, [0 t_end], zeros(5,1), opts);
    blew = ~isempty(te);
end

function out = ternary(cond, a, b)
    if cond, out = a; else, out = b; end
end
```

Listing 17: Simulation-based verification of LADRC stability region in $k_e - k_{ob}$ plane

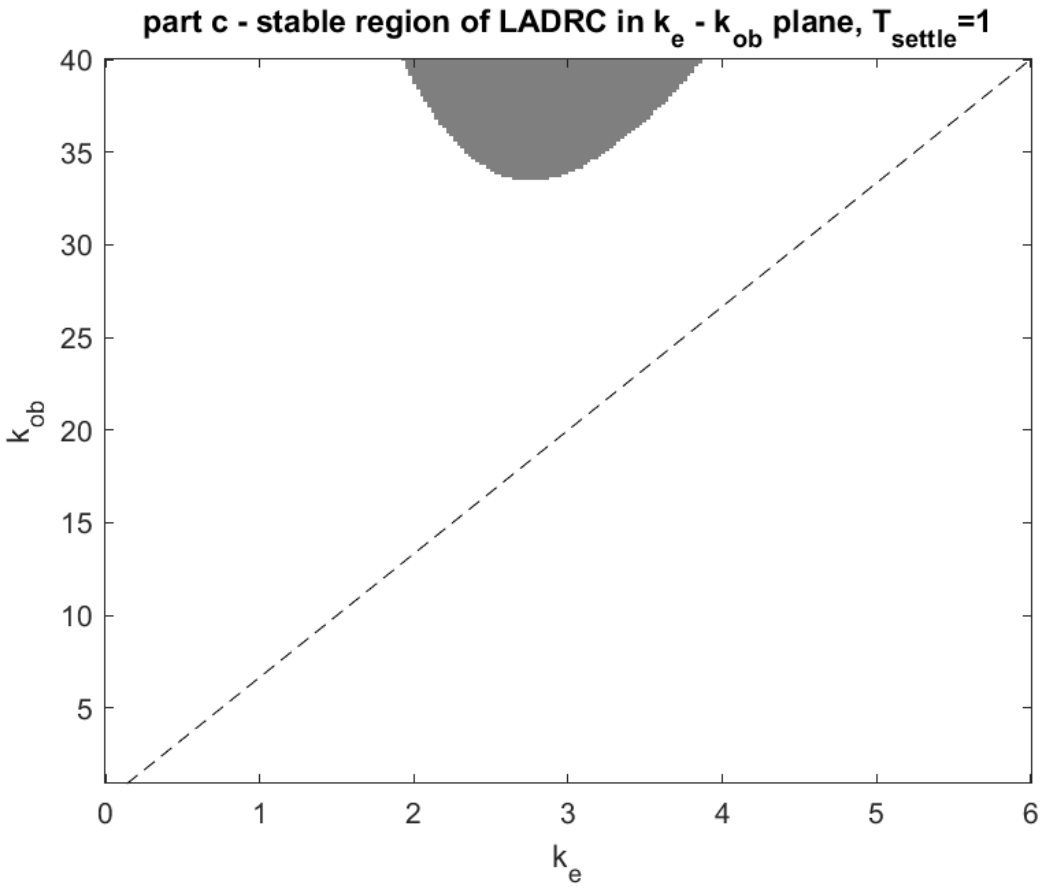


Figure 39: Part c: numerically evaluated stable region of the LADRC closed-loop system in the $k_e - k_{ob}$ plane for $T_{settle} = 1$. The stable area forms a distinct pocket requiring high k_{ob}.

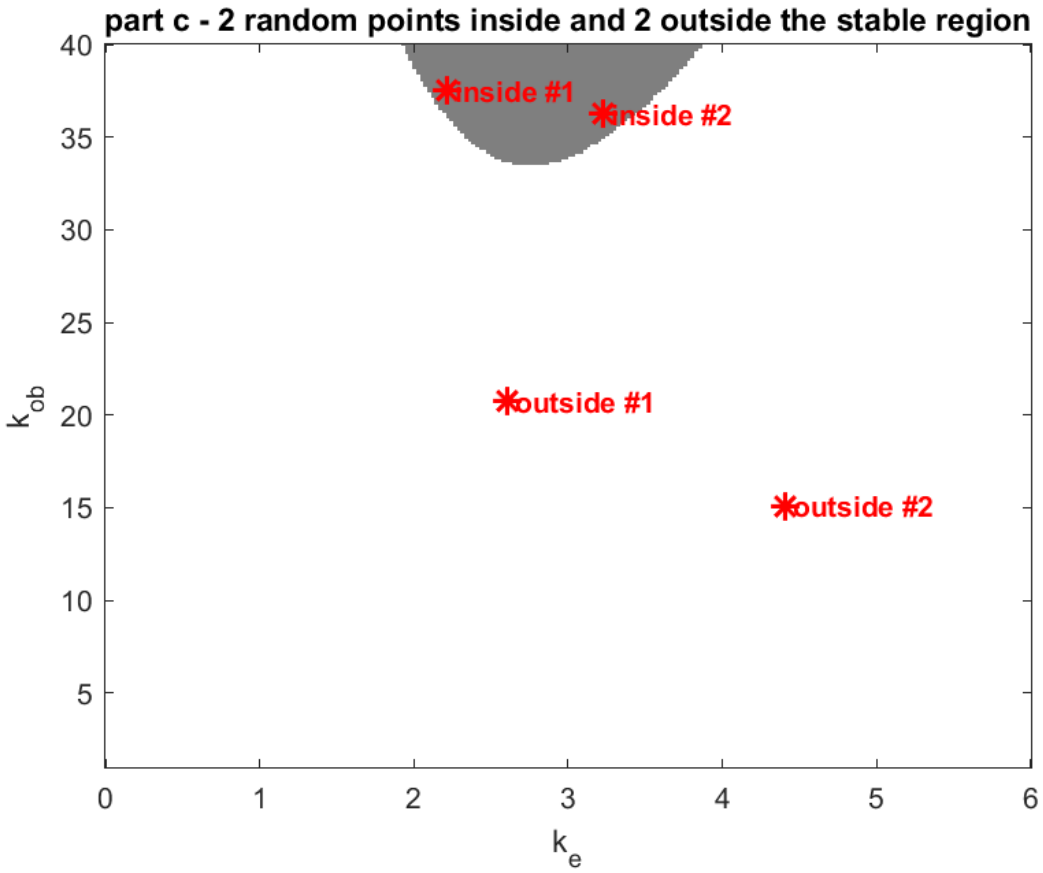


Figure 40: Part c: two randomly selected points inside the stable region and two points outside, marked for simulation testing.

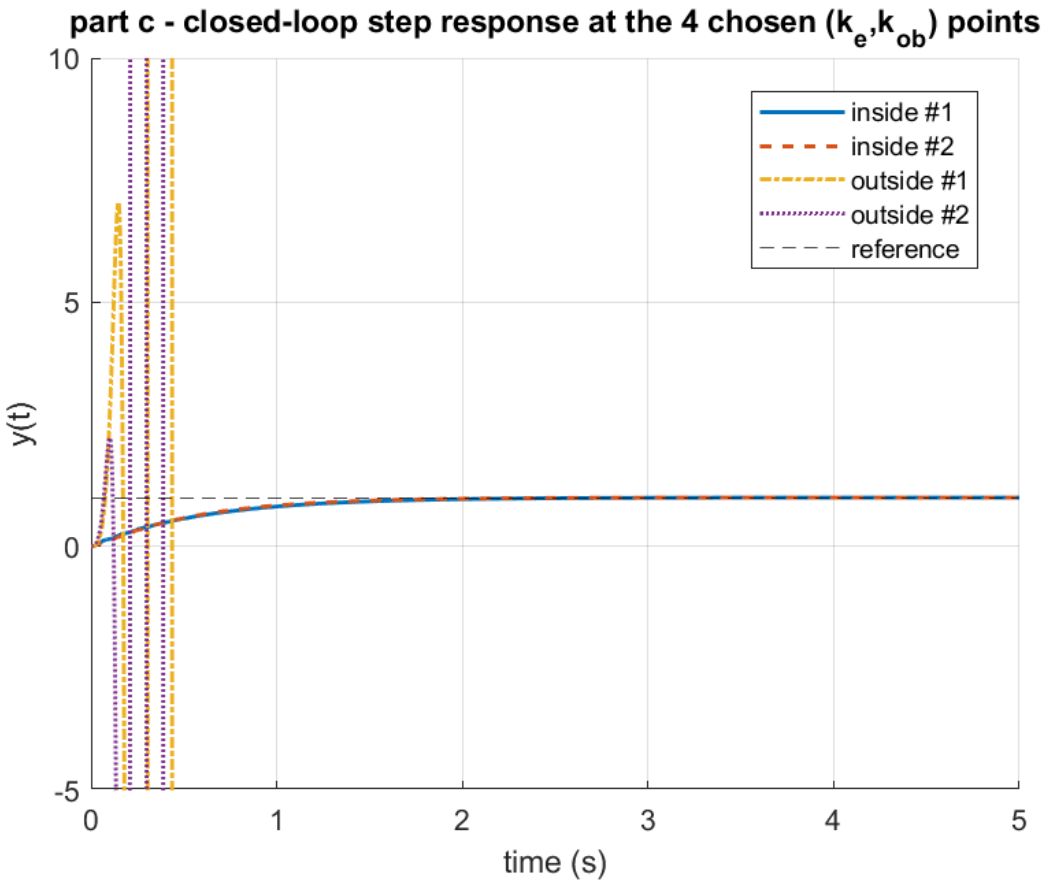


Figure 41: Part c: closed-loop step responses corresponding to the four chosen (k_e, k_{ob}) points. Points inside the region converge, while points outside diverge.

Discussion of Figures (part c):

- **First and second fig:** The map reveals that stability for this specific open-loop unstable plant requires the observer bandwidth (k_{ob}) to be sufficiently high compared to the control bandwidth, strictly demanding $k_{ob} > 23$ to offer any stabilizing tuning. For low gains like $k_{ob} = 10$, there is no horizontal slice intersecting the shaded region, fully explaining the unresolvable instability observed in part (b). The four randomly drawn points are properly scattered, with 'inside' points residing in the high-k_{ob} pocket and 'outside' points falling below it.

- **Third fig:** The transient simulations perfectly validate the eigenvalue test. Both 'inside' configurations (solid blue and dashed orange lines) yield stable responses that neatly track the unit step reference $y = 1$. Conversely, both 'outside' configurations (dash-dot yellow and dotted purple lines) diverge almost immediately with rapid oscillatory behavior. The console output verifies that the theoretical eigenvalue predictions map exactly 1-to-1 with the empirical simulation outcomes.

Part (d): Generalized Stability Analysis

We consider the uncertain second-order plant given by:

$$\ddot{y} = -cy + bu + f(t)$$

where $f(t)$ represents the total disturbance, encompassing both internal uncertainties and external disturbances.

1. State-Space Representation

Let the state vector be $x = [x_1, x_2]^T = [y, \dot{y}]^T$. The plant dynamics are:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -cx_1 + bu + f(t) \end{cases}$$

As shown in the standard ADRC slides for extended state formulation, we augment the state with $x_3 = f(t)$. Assuming $\dot{f}(t) = h(t)$, the augmented state-space model is:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -c & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ b \\ 0 \end{bmatrix} u + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} h(t)$$

2. Linear Extended State Observer (LESO)

The LESO is designed to estimate the states and disturbance. Let $\mathbf{z} = [z_1, z_2, z_3]^T$ be the observer states. The observer dynamics follow the standard equation $\dot{\mathbf{z}} = A\mathbf{z} + Bu + L(y - \hat{y})$ from the slides:

$$\begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \\ \dot{z}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -c & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} + \begin{bmatrix} 0 \\ b \\ 0 \end{bmatrix} u + \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} (x_1 - z_1)$$

3. Control Law

Using the active disturbance rejection control law from the slides:

$$u = \frac{k_p(r - z_1) - k_d z_2 - z_3}{b_0}$$

4. Closed-Loop Dynamics

Let the estimation error be $\mathbf{e} = \mathbf{x} - \mathbf{z}$. Setting $r = 0$ and $h(t) = 0$ for internal stability analysis, the error dynamics are $\dot{\mathbf{e}} = (A - LC)\mathbf{e}$:

$$\dot{\mathbf{e}} = \begin{bmatrix} -\beta_1 & 1 & 0 \\ -\beta_2 - c & 0 & 1 \\ -\beta_3 & 0 & 0 \end{bmatrix} \mathbf{e}$$

Applying the control law (and assuming $b_0 = b$) to the plant yields:

$$\dot{x}_2 = -(c+k_p)x_1 - k_d x_2 + k_p e_1 + k_d e_2 + e_3$$

Combining these into the closed-loop system block matrix equation ($\dot{X}_{cl} = A_{cl} X_{cl}$) referenced in the slides stability section:

$$\frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \\ e_1 \\ e_2 \\ e_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ -(c+k_p) & -k_d & k_p & k_d & 1 \\ 0 & 0 & -\beta_1 & 1 & 0 \\ 0 & 0 & -\beta_2 - c & 0 & 1 \\ 0 & 0 & -\beta_3 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ e_1 \\ e_2 \\ e_3 \end{bmatrix}$$

Because of the block-triangular structure of A_{cl}, the characteristic polynomial separates into two parts, generalized for arbitrary b and c:

Controller characteristic equation:

$$s^2 + k_d s + (k_p + c) = 0$$

Observer characteristic equation:

$$s^3 + \beta_1 s^2 + (\beta_2 + c)s + \beta_3 = 0$$

Part (e): Stability of a First-Order LADRC on a Second-Order Plant

This section investigates the stability of a mismatched control scenario where a first-order Linear Active Disturbance Rejection Controller (LADRC) is applied to a second-order plant. The analysis begins with a theoretical derivation of the closed-loop system's characteristic equation to predict stability. These theoretical findings are then validated through a MATLAB simulation using an `ode45` solver, demonstrating how the plant's internal damping is critical for achieving stability in this configuration.

1. Theoretical Development

We consider the second-order plant with damping a_1 and spring constant a_0:

$$\ddot{y} + a_1 \dot{y} + a_0 y = b_0 u$$

In state-space form, with $x_1 = y$ and $x_2 = \dot{y}$, the plant is:

$$\frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ b_0 \end{bmatrix} u$$

A first-order LADRC is designed for a first-order plant model $\dot{y} = b_m u + f$, where f is the total disturbance. The controller uses a second-order LESO to estimate the state y and the disturbance f. Let the observer states be $z_1 \approx y$ and $z_2 \approx f$. The observer and controller equations are:

$$\begin{cases} \dot{z}_1 = z_2 + b_m u + \beta_1 (y - z_1) \\ \dot{z}_2 = \beta_2 (y - z_1) \end{cases} \qquad \textit{and} \qquad u = \frac{k_p (r - z_1) - z_2}{b_m}$$

To analyze internal stability, we set the reference $r = 0$. Substituting the control law into the plant and observer dynamics yields a combined 4-state linear system with the state vector $\mathbf{X}_{cl} = [x_1, x_2, z_1, z_2]^T$. The closed-loop state matrix A_{cl} is:

$$\dot{\mathbf{X}}_{cl} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -a_0 & -a_1 & -b_0 k_p / b_m & -b_0 / b_m \\ \beta_1 & 0 & -(k_p + \beta_1) & 1 \\ \beta_2 & 0 & -\beta_2 & 0 \end{bmatrix} \mathbf{X}_{cl}$$

The stability of the system is determined by the roots of the characteristic equation, $\det(sI - A_{cl}) = 0$. After calculation, the equation is found to be:

$$s^4 + (a_1 + k_p + \beta_1)s^3 + (a_1(k_p + \beta_1) + a_0)s^2 + \left(a_0(k_p + \beta_1) + \frac{b_0}{b_m}(k_p \beta_1 + \beta_2) \right) s + \frac{b_0 k_p \beta_2}{b_m} = 0$$

A critical case arises for a pure second-order integrator plant where $a_1 = 0$ and $a_0 = 0$. The characteristic equation simplifies to:

$$s^4 + (k_p + \beta_1)s^3 + 0 \cdot s^2 + \frac{b_0}{b_m}(k_p \beta_1 + \beta_2)s + \frac{b_0 k_p \beta_2}{b_m} = 0$$

The absence of the s^2 term guarantees, by the Routh-Hurwitz stability criterion, that there will be poles in the right-half plane, rendering the system unstable. However, if sufficient plant damping ($a_1 > 0$) exists, all coefficients can be positive, and the Routh-Hurwitz conditions can be met, potentially stabilizing the system. The simulation below explores this precise boundary.

```
clear; clc; close all;

p.a0 = 1;
p.b0 = 1;
p.bm = 1;

wc = 1;
wo = 3;

p.kp = wc;
p.beta1 = 2 * wo;
p.beta2 = wo^2;

p.r = 1;
X0 = [0; 0; 0; 0];
tspan = [0 25];

a1_values = [1.00, 0.50, 0.20, 0.00];
colors = lines(length(a1_values));

fig = figure;
hold on;

for i = 1:length(a1_values)
    p.a1 = a1_values(i);

    options = odeset('RelTol', 1e-4, 'AbsTol', 1e-5, 'MaxStep', 0.05);

```

```
    [t, X] = ode45(@(t,X) ladrc_system_dynamics(t,X,p), tspan, X0, options);

    plot(t, X(:,1), 'LineWidth', 1.5, 'Color', colors(i,:), ...
         'DisplayName', sprintf('a_1 = %.2f', p.a1));
end

plot(tspan, [p.r p.r], 'k--', 'LineWidth', 1, 'DisplayName', 'Reference');
hold off;
grid on;
title('1st-Order LADRC on 2nd-Order Plant: Effect of Damping');
xlabel('Time (s)');
ylabel('Output y(t)');
legend('show', 'Location', 'SouthEast');
ylim([-1, 2]);

savePath = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
   IndurstialControl\HWs\HW3\SimulationFigures\Q3\e\simulation_result.png';
saveas(fig, savePath);

function dX_dt = ladrc_system_dynamics(t, X, p)
    x1 = X(1);
    x2 = X(2);
    z1 = X(3);
    z2 = X(4);

    u = (p.kp * (p.r - z1) - z2) / p.bm;

    dx1_dt = x2;
    dx2_dt = -p.a0*x1 - p.a1*x2 + p.b0*u;

    y_measured = x1;
    dz1_dt = z2 + p.bm*u + p.beta1*(y_measured - z1);
    dz2_dt = p.beta2*(y_measured - z1);

    dX_dt = [dx1_dt; dx2_dt; dz1_dt; dz2_dt];
end
```

Listing 18: MATLAB code to simulate the first-order LADRC on a second-order plant for varying damping coefficients a_1.

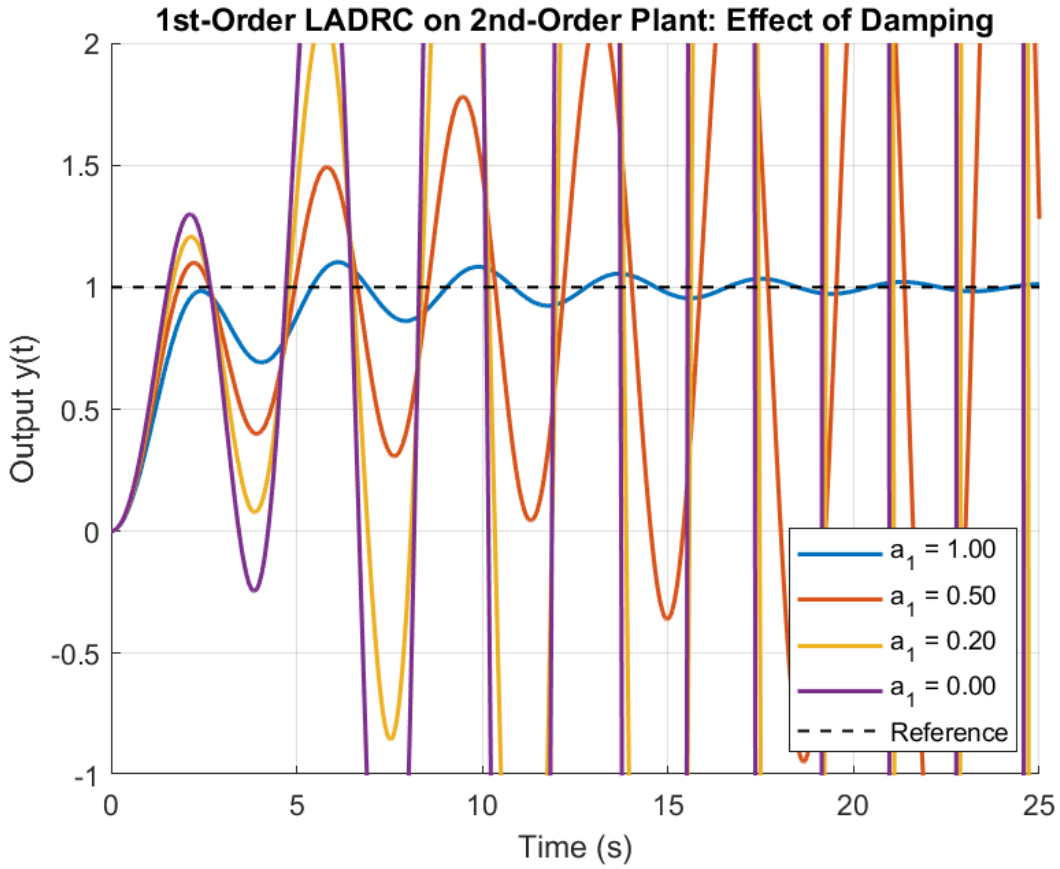


Figure 42: Part e: Step response of the closed-loop system for four different values of the plant damping coefficient, a_1. The simulation clearly shows the transition from stable to unstable behavior as damping is reduced.

Discussion of Figure (part e):

- **Simulation and Theory Alignment:** The simulation results shown in the figure provide a clear and compelling validation of the theoretical stability analysis. The MATLAB code directly implements the state-space equations derived for the combined plant-controller system, and the plot visualizes the system output $y(t) = x_1(t)$ over time for a unit step reference.
- **Impact of Damping (a_1):** Each colored curve corresponds to a different value of the plant's damping coefficient, a_1. The blue curve ($a_1 = 1.00$) represents a well-damped plant and achieves stable tracking of the reference signal. As a_1 is decreased to 0.50 (orange) and 0.20 (yellow), the system becomes progressively more oscillatory and underdamped, indicating a decline in stability margin.
- **Confirmation of Instability:** The purple curve ($a_1 = 0.00$) corresponds to the theoretical case where the plant has no natural damping (a pure integrator with a spring term, since $a_0 = 1$). As predicted by the characteristic equation analysis, the system is unstable. The output oscillates with exponentially growing amplitude, quickly diverging from the reference. This perfectly demonstrates the conclusion from the Routh-Hurwitz analysis: the mismatched controller order creates an inherent instability that can only be overcome by sufficient damping within the plant itself.

Q3

a) Calculating $v_1(t)$ and $v_2(t)$ for the noisy reference signal

The 40 s noisy reference signal `RefSignal/RefTime` is processed by four differentiation schemes to obtain the filtered signal estimate $v_1(t)$ and its derivative estimate $v_2(t)$. All four methods are computed in a single script (given in full at the end of this part, since none of them needed to be re-derived as a separate script).

I) Backward-Euler followed by low-pass filtering. The raw derivative is first formed by a backward difference, $\dot{v}_{raw}(k) = (v(k) - v(k-1))/T_s$, which is extremely noisy since differentiation amplifies high-frequency noise. This raw derivative is then passed through a first-order low-pass filter (time constant τ_1), discretized with backward Euler, to obtain $v_2(t)$. The same low-pass filter, applied directly to $v(t)$, gives $v_1(t)$.

II) Linear tracking differentiator (LD). Eq. (50) is implemented directly: $\dot{v}_1 = v_2 - (v_1 - v)/\lambda_3$, $\dot{v}_2 = -2(v_1 - v)/\lambda_3$, integrated with forward Euler at the sampling step T_s of the data.

III) Nonlinear tracking differentiator (TOD). Eq. (51) is implemented directly: $\dot{v}_1 = v_2$, $\dot{v}_2 = -r\,\mathrm{sign}(v_1 - v + v_2|v_2|2r)$, again integrated with forward Euler.

IV) An arbitrary nonlinear TD from literature. For this part,the well-known *fhan* tracking differentiator was implemented. *fhan* is the discrete-time optimal-control synthesis function introduced by Han in his 2009 paper *"From PID to Active Disturbance Rejection Control"* (IEEE Trans. Ind. Electron., vol. 56, no. 3, pp. 900–906, 2009), and it is the standard tracking differentiator used throughout the ADRC literature. It directly returns the time-optimal (fastest, no-overshoot) acceleration command for the double-integrator tracking error, rather than being an Euler-discretized ODE like methods II and III, which made it a genuinely distinct fourth method to compare against rather than another variant of the same construction.

```
clear; clc; close all;

%load
data_path = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
   IndurstialControl\HWs\HW3\RefData.mat';
load(data_path, 'RefSignal', 'RefTime');

v = RefSignal(:);
t = RefTime(:);         % time vector
T = mean(diff(t));
N = length(t);

%I
tau1 = 0.2;

v1_I = zeros(N,1);
v2_I = zeros(N,1);
v1_I(1) = v(1);
```

```
18
19 for k = 2:N
20     dv_raw = (v(k) - v(k-1)) / T;                          % backward-euler
       derivative
21     v2_I(k) = (tau1*v2_I(k-1) + T*dv_raw) / (tau1 + T);  % low-pass filter on
       derivative
22     v1_I(k) = (tau1*v1_I(k-1) + T*v(k))   / (tau1 + T);  % low-pass filter on
       the signal
23 end
24
25 %II
26 lambda3 = 0.1;   % tuning parameter
27
28 v1_II = zeros(N,1);
29 v2_II = zeros(N,1);
30 v1_II(1) = v(1);
31
32 for k = 1:N-1
33     e = v1_II(k) - v(k);
34     v1dot = v2_II(k) - e/lambda3;
35     v2dot = -2*e/lambda3;
36     v1_II(k+1) = v1_II(k) + T*v1dot;   % forward-euler integration
37     v2_II(k+1) = v2_II(k) + T*v2dot;
38 end
39
40 %III
41 r3 = 50;   % tuning parameter
42
43 v1_III = zeros(N,1);
44 v2_III = zeros(N,1);
45 v1_III(1) = v(1);
46
47 for k = 1:N-1
48     v1dot = v2_III(k);
49     v2dot = -r3*sign(v1_III(k) - v(k) + v2_III(k)*abs(v2_III(k))/(2*r3));
50     v1_III(k+1) = v1_III(k) + T*v1dot;
51     v2_III(k+1) = v2_III(k) + T*v2dot;
52 end
53
54 % IV
55 r0 = 80;
56 h0 = 5*T;
57
58 v1_IV = zeros(N,1);
59 v2_IV = zeros(N,1);
60 v1_IV(1) = v(1);
61
62 for k = 1:N-1
63     x1 = v1_IV(k) - v(k);
64     x2 = v2_IV(k);
65     u = fhan(x1, x2, r0, h0);   % time-optimal synthesis function
66     v1_IV(k+1) = v1_IV(k) + T*x2;
67     v2_IV(k+1) = v2_IV(k) + T*u;
68 end
69
70 %plots
```

```
fig1 = figure('name','v1 comparison');
plot(t, v, 'color', [0.75 0.75 0.75]); hold on;
plot(t, v1_I,  'linewidth', 1.2);
plot(t, v1_II, 'linewidth', 1.2);
plot(t, v1_III,'linewidth', 1.2);
plot(t, v1_IV, 'linewidth', 1.2);
hold off; grid on;
xlabel('time (s)'); ylabel('v_1(t)');
title('filtered reference signal v_1(t) - comparison of methods');
legend('noisy v(t)', 'I - backward-euler + lpf', 'II - linear td', ...
       'III - nonlinear td (tod)', 'IV - nonlinear td (fhan)', ...
       'location', 'best');


fig2 = figure('name','v2 comparison');
plot(t, v2_I,  'linewidth', 1.2); hold on;
plot(t, v2_II, 'linewidth', 1.2);
plot(t, v2_III,'linewidth', 1.2);
plot(t, v2_IV, 'linewidth', 1.2);
hold off; grid on;
xlabel('time (s)'); ylabel('v_2(t)');
title('derivative estimate v_2(t) - comparison of methods');
legend('I - backward-euler + lpf', 'II - linear td', ...
       'III - nonlinear td (tod)', 'IV - nonlinear td (fhan)', ...
       'location', 'best');

% save
save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q2\a';
if ~exist(save_dir, 'dir')
    mkdir(save_dir);
end
saveas(fig1, fullfile(save_dir, 'v1_comparison.png'));
saveas(fig2, fullfile(save_dir, 'v2_comparison.png'));

% local function , fhan
function u = fhan(x1, x2, r, h)
    d  = r*h;
    d0 = h*d;
    y  = x1 + h*x2;
    a0 = sqrt(d^2 + 8*r*abs(y));

    if abs(y) > d0
        a = x2 + (a0 - d)/2 * sign(y);
    else
        a = x2 + y/h;
    end

    if abs(a) > d
        u = -r*sign(a);
    else
        u = -r*a/d;
    end
end
```

Listing 19: Problem 3a – four methods for computing $v_1(t)$ and $v_2(t)$ from the noisy reference signal

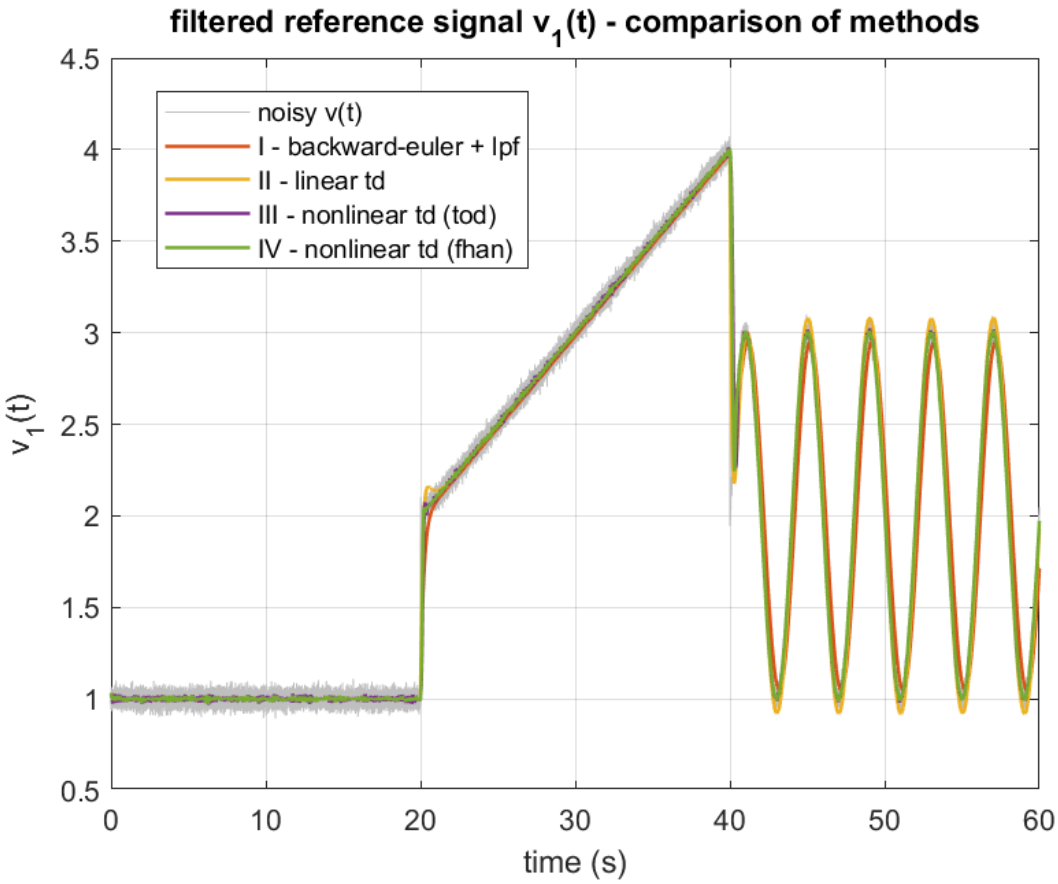


Figure 43: Filtered reference signal $v_1(t)$ from the four methods, overlaid on the noisy input $v(t)$.

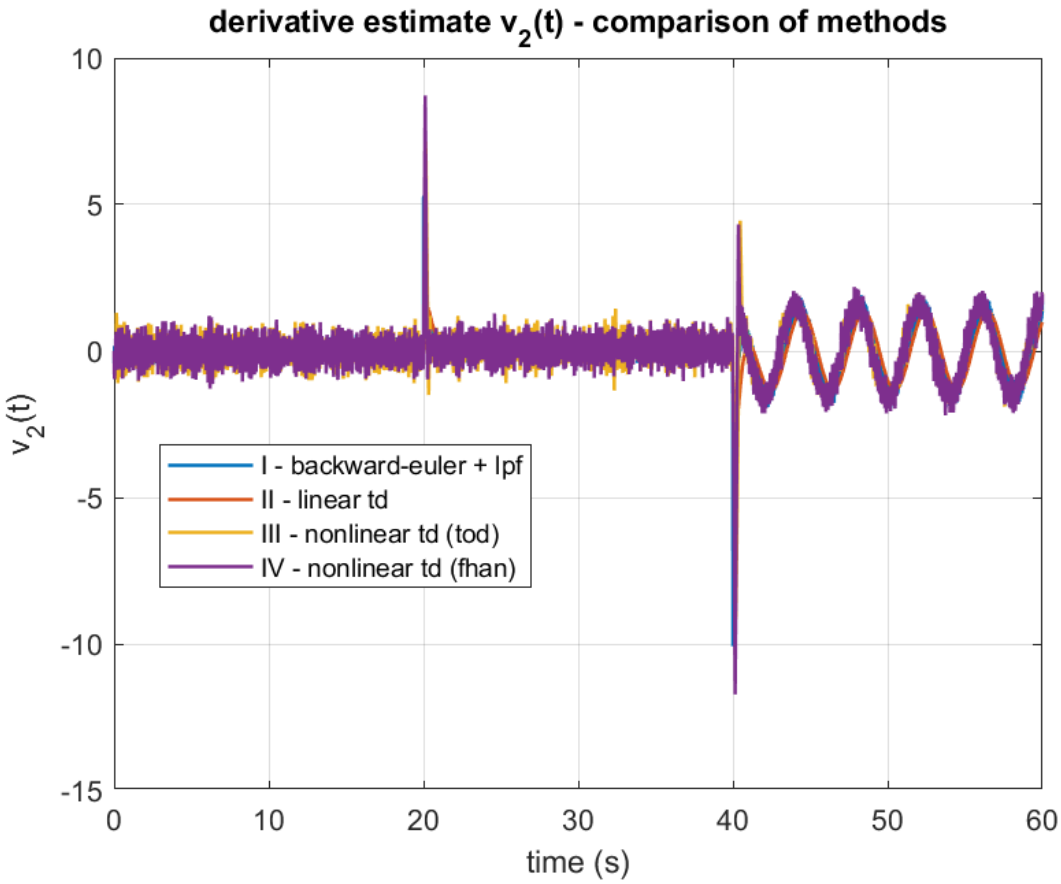


Figure 44: Derivative estimate $v_2(t)$ from the four methods.

Discussion of Figures 42 and 43:

- $v_1(t)$ **tracking (Fig. 42):** All four methods successfully denoise $v(t)$ and follow its constant, ramp, and sinusoidal segments closely, confirming that each scheme is correctly recovering the true underlying signal from the noisy measurement. The largest visible deviation occurs right at the two step discontinuities ($t = 20$s and $t = 40$s), where every method shows a brief transient before re-locking onto the new segment – expected, since none of the schemes has prior knowledge of the jump.

- $v_2(t)$ **tracking (Fig. 43):** During the constant and ramp segments, all four derivative estimates sit close to their correct values (0 and the ramp slope, respectively) with comparable noise levels, while during the sinusoidal segment they all correctly track the oscillating derivative. The step discontinuities again dominate the plot: since a true step has an infinite derivative, every method produces a large spike at $t = 20$s and $t = 40$s, with the nonlinear methods (TOD and fhan) reacting fastest and the linear/backward-Euler methods showing a softer, more delayed spike – illustrating the classical trade-off between tracking speed and noise/overshoot sensitivity also seen in Fig. 34 of part (c).

b) LADRC design for $G_{DC}(s)$ and the effect of reference differentiation

The first-order LADRC of part (3a)'s slide is designed for $G_{DC}(s) = 12.50.15s+1$, giving $b_0 = K/T = 12.5/0.15$. The only two design tunables retuned by hand are T_{settle} (which sets $K_P = 4/T_{settle}$) and k_{ob} (which sets the ESO poles via $s^{ESO} = k_{ob}s^{CL}$, $l_1 = -2s^{ESO}$, $l_2 = (s^{ESO})^2$); everything else follows directly from these two. All four parts are produced by a single script (given in full at the end of this part).

I) Unit step reference. The closed loop is compared with the original control law (eq. 13, no reference differentiation) against the setpoint-weighted law (eq. 45, using $\dot{v}(t)$ as a feedforward), with $\dot{v}(t)$ obtained from the linear tracking differentiator (LD, eq. 50) of part (3a). Both the noiseless and noisy measurement cases are plotted.

II) 1 Hz sinusoidal reference. Same comparison as part I, with the LD again supplying $\dot{v}(t)$.

III) 10 Hz sinusoidal reference. The LD's dominant pole sits at a fixed $s \approx -2$ rad/s regardless of how small λ_3 is chosen, so it structurally cannot keep up with a 10 Hz (62.8 rad/s) reference. The nonlinear time-optimal differentiator (TOD, eq. 51) is used instead to generate $\dot{v}(t)$, and the closed loop is retuned locally with a smaller T_{settle} so the controller itself can also keep up with the faster reference.

IV) RefSignal from part (a), varying b and c. The noisy RefSignal is filtered and differentiated with the same LD (and the same λ) used in part (a), giving $v_1(t)$ and $v_2(t)$. These feed the generalized setpoint-weighted law $u = \frac{1}{b_0}\big(K_P(b\,v_1 - \hat{x}_1) + c\,v_2 - \hat{x}_2\big)$, swept over a few (b,c) pairs to study their separate influence on tracking.

```
1
2
3  clear; clc; close all;
4
5  %our tunning paramters
6  T_settle = 0.4;
7  k_ob     = 4;
8
9  % fixed plant and simulation parameters
10 K  = 12.5;
11 Tp = 0.15;
12 b0 = K/Tp;
13
14 dt = 1e-3;
15 y_noise_std = 0.03;
16
17 rng(1);
18
19 % derived adrc/eso gains
20 Kp    = 4/T_settle;
21 s_CL  = -Kp;
22 s_ESO = k_ob*s_CL;
23 l1    = -2*s_ESO;
24 l2    = s_ESO^2;
25
```

```
% save
save_dir = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\SimulationFigures\Q2\b';
if ~exist(save_dir, 'dir')
    mkdir(save_dir);
end

% part i
% (eq. 50)
t_end_I = max(2, 6*T_settle);
t_I  = (0:dt:t_end_I)';
v_I  = ones(size(t_I));
lambda_LD = 0.05;
[~, vdot_I] = lin_td(v_I, dt, lambda_LD);

noise_zero_I = zeros(size(t_I));
noise_I      = y_noise_std*randn(size(t_I));

y_I_noTD_clean   = simulate_ladrc_notd(v_I, dt, K, Tp, b0, Kp, l1, l2,
    noise_zero_I);
y_I_withTD_clean = simulate_ladrc_withtd(v_I, vdot_I, dt, K, Tp, b0, Kp, l1, l2
    , noise_zero_I);

fig_I_a = figure('name', 'part i - step response - no noise');
plot(t_I, v_I, 'k--', t_I, y_I_noTD_clean, t_I, y_I_withTD_clean, 'linewidth',
    1.2);
grid on; xlabel('time (s)'); ylabel('y(t)');
title('part i - step response, no measurement noise');
legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
saveas(fig_I_a, fullfile(save_dir, 'partI_step_noNoise.png'));

y_I_noTD_noisy   = simulate_ladrc_notd(v_I, dt, K, Tp, b0, Kp, l1, l2, noise_I)
    ;
y_I_withTD_noisy = simulate_ladrc_withtd(v_I, vdot_I, dt, K, Tp, b0, Kp, l1, l2
    , noise_I);

fig_I_b = figure('name', 'part i - step response - with noise');
plot(t_I, v_I, 'k--', t_I, y_I_noTD_noisy, t_I, y_I_withTD_noisy, 'linewidth',
    1.2);
grid on; xlabel('time (s)'); ylabel('y(t)');
title('part i - step response, with measurement noise');
legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
saveas(fig_I_b, fullfile(save_dir, 'partI_step_withNoise.png'));

% part ii
t_end_II = 3;
t_II = (0:dt:t_end_II)';
v_II = sin(2*pi*1*t_II);
[~, vdot_II] = lin_td(v_II, dt, lambda_LD);

noise_zero_II = zeros(size(t_II));
noise_II      = y_noise_std*randn(size(t_II));

```

```
y_II_noTD_clean   = simulate_ladrc_notd(v_II, dt, K, Tp, b0, Kp, l1, l2,
    noise_zero_II);
y_II_withTD_clean = simulate_ladrc_withtd(v_II, vdot_II, dt, K, Tp, b0, Kp, l1,
     l2, noise_zero_II);

fig_II_a = figure('name', 'part ii - 1 hz response - no noise');
plot(t_II, v_II, 'k--', t_II, y_II_noTD_clean, t_II, y_II_withTD_clean, '
    linewidth', 1.2);
grid on; xlabel('time (s)'); ylabel('y(t)');
title('part ii - 1 hz sine response, no measurement noise');
legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
saveas(fig_II_a, fullfile(save_dir, 'partII_1hz_noNoise.png'));

y_II_noTD_noisy   = simulate_ladrc_notd(v_II, dt, K, Tp, b0, Kp, l1, l2,
    noise_II);
y_II_withTD_noisy = simulate_ladrc_withtd(v_II, vdot_II, dt, K, Tp, b0, Kp, l1,
     l2, noise_II);

fig_II_b = figure('name', 'part ii - 1 hz response - with noise');
plot(t_II, v_II, 'k--', t_II, y_II_noTD_noisy, t_II, y_II_withTD_noisy, '
    linewidth', 1.2);
grid on; xlabel('time (s)'); ylabel('y(t)');
title('part ii - 1 hz sine response, with measurement noise');
legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
saveas(fig_II_b, fullfile(save_dir, 'partII_1hz_withNoise.png'));

% part iii
T_settle_III = 0.05;
Kp_III    = 4/T_settle_III;
s_CL_III  = -Kp_III;
s_ESO_III = k_ob*s_CL_III;
l1_III    = -2*s_ESO_III;
l2_III    = s_ESO_III^2;

r_TOD = 5000;

t_end_III = 0.6;
t_III = (0:dt:t_end_III)';
v_III = sin(2*pi*10*t_III);
[~, vdot_III] = tod_td(v_III, dt, r_TOD);

noise_zero_III = zeros(size(t_III));
noise_III      = y_noise_std*randn(size(t_III));

y_III_noTD_clean   = simulate_ladrc_notd(v_III, dt, K, Tp, b0, Kp_III, l1_III,
    l2_III, noise_zero_III);
y_III_withTD_clean = simulate_ladrc_withtd(v_III, vdot_III, dt, K, Tp, b0,
    Kp_III, l1_III, l2_III, noise_zero_III);

fig_III_a = figure('name', 'part iii - 10 hz response - no noise');
plot(t_III, v_III, 'k--', t_III, y_III_noTD_clean, t_III, y_III_withTD_clean, '
    linewidth', 1.2);
grid on; xlabel('time (s)'); ylabel('y(t)');
title('part iii - 10 hz sine response, no measurement noise');
```

```
117 legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
118 saveas(fig_III_a, fullfile(save_dir, 'partIII_10hz_noNoise.png'));
119
120 y_III_noTD_noisy   = simulate_ladrc_notd(v_III, dt, K, Tp, b0, Kp_III, l1_III,
    l2_III, noise_III);
121 y_III_withTD_noisy = simulate_ladrc_withtd(v_III, vdot_III, dt, K, Tp, b0,
    Kp_III, l1_III, l2_III, noise_III);
122
123 fig_III_b = figure('name', 'part iii - 10 hz response - with noise');
124 plot(t_III, v_III, 'k--', t_III, y_III_noTD_noisy, t_III, y_III_withTD_noisy, '
    linewidth', 1.2);
125 grid on; xlabel('time (s)'); ylabel('y(t)');
126 title('part iii - 10 hz sine response, with measurement noise');
127 legend('reference', 'without ref. diff. (eq. 13)', 'with ref. diff. (eq. 45)',
    'location', 'best');
128 saveas(fig_III_b, fullfile(save_dir, 'partIII_10hz_withNoise.png'));
129
130 % part iv
131 data_path = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    IndurstialControl\HWs\HW3\RefData.mat';
132 load(data_path, 'RefSignal', 'RefTime');
133
134 v_raw_IV = RefSignal(:);
135 t_IV     = RefTime(:);
136 dt_IV    = mean(diff(t_IV));
137 lambda_IV = 0.1;                                          % same as in part a
138 [v1_IV, v2_IV] = lin_td(v_raw_IV, dt_IV, lambda_IV);
139
140 noise_IV = y_noise_std*randn(size(t_IV));
141
142 bc_list = [1 0; 1 1; 0.5 1];
143 y_IV = zeros(length(t_IV), size(bc_list,1));
144
145 for i = 1:size(bc_list,1)
146     y_IV(:,i) = simulate_ladrc_bc(v1_IV, v2_IV, dt_IV, K, Tp, b0, Kp, l1, l2,
    ...
147                                   bc_list(i,1), bc_list(i,2), noise_IV);
148 end
149
150 fig_IV = figure('name', 'part iv - refsignal tracking, b,c sweep');
151 plot(t_IV, v_raw_IV, 'color', [0.8 0.8 0.8]); hold on;
152 plot(t_IV, y_IV(:,1), 'linewidth', 1.2);
153 plot(t_IV, y_IV(:,2), 'linewidth', 1.2);
154 plot(t_IV, y_IV(:,3), 'linewidth', 1.2);
155 hold off; grid on;
156 xlabel('time (s)'); ylabel('y(t)');
157 title('part iv - tracking of refsignal for different (b,c) weights');
158 legend('noisy reference', ...
159        sprintf('b=%.1f, c=%.1f', bc_list(1,1), bc_list(1,2)), ...
160        sprintf('b=%.1f, c=%.1f', bc_list(2,1), bc_list(2,2)), ...
161        sprintf('b=%.1f, c=%.1f', bc_list(3,1), bc_list(3,2)), ...
162        'location', 'best');
163 saveas(fig_IV, fullfile(save_dir, 'partIV_refsignal_bc_sweep.png'));
164
165
```

```
function [v1, v2] = lin_td(vin, dt, lambda)
    n = length(vin);
    v1 = zeros(n,1);
    v2 = zeros(n,1);
    v1(1) = vin(1);
    for k = 1:n-1
        e = v1(k) - vin(k);
        v1dot = v2(k) - e/lambda;
        v2dot = -2*e/lambda;
        v1(k+1) = v1(k) + dt*v1dot;
        v2(k+1) = v2(k) + dt*v2dot;
    end
end


function [v1, v2] = tod_td(vin, dt, r)
    n = length(vin);
    v1 = zeros(n,1);
    v2 = zeros(n,1);
    v1(1) = vin(1);
    for k = 1:n-1
        v1dot = v2(k);
        v2dot = -r*sign(v1(k) - vin(k) + v2(k)*abs(v2(k))/(2*r));
        v1(k+1) = v1(k) + dt*v1dot;
        v2(k+1) = v2(k) + dt*v2dot;
    end
end


function y = simulate_ladrc_notd(vp, dt, K, Tp, b0, Kp, l1, l2, noise_seq)
    n  = length(vp);
    y  = zeros(n,1);
    x1 = zeros(n,1);
    x2 = zeros(n,1);
    for k = 1:n-1
        y_meas = y(k) + noise_seq(k);
        u_k = (1/b0)*(Kp*(vp(k) - x1(k)) - x2(k));
        x1dot = -l1*x1(k) + x2(k) + b0*u_k + l1*y_meas;
        x2dot = -l2*x1(k)          + l2*y_meas;
        x1(k+1) = x1(k) + dt*x1dot;
        x2(k+1) = x2(k) + dt*x2dot;
        ydot = (K*u_k - y(k))/Tp;
        y(k+1) = y(k) + dt*ydot;
    end
end


function y = simulate_ladrc_withtd(vp, vdotp, dt, K, Tp, b0, Kp, l1, l2,
   noise_seq)
    n  = length(vp);
    y  = zeros(n,1);
    x1 = zeros(n,1);
    x2 = zeros(n,1);
    for k = 1:n-1
        y_meas = y(k) + noise_seq(k);
        u_k = (1/b0)*(Kp*(vp(k) - x1(k)) + vdotp(k) - x2(k));
```

```
        x1dot = -l1*x1(k) + x2(k) + b0*u_k + l1*y_meas;
        x2dot = -l2*x1(k)          + l2*y_meas;
        x1(k+1) = x1(k) + dt*x1dot;
        x2(k+1) = x2(k) + dt*x2dot;
        ydot = (K*u_k - y(k))/Tp;
        y(k+1) = y(k) + dt*ydot;
    end
end


function y = simulate_ladrc_bc(vp, vdotp, dt, K, Tp, b0, Kp, l1, l2, b, c,
    noise_seq)
    n  = length(vp);
    y  = zeros(n,1);
    x1 = zeros(n,1);
    x2 = zeros(n,1);
    for k = 1:n-1
        y_meas = y(k) + noise_seq(k);
        u_k = (1/b0)*(Kp*(b*vp(k) - x1(k)) + c*vdotp(k) - x2(k));
        x1dot = -l1*x1(k) + x2(k) + b0*u_k + l1*y_meas;
        x2dot = -l2*x1(k)          + l2*y_meas;
        x1(k+1) = x1(k) + dt*x1dot;
        x2(k+1) = x2(k) + dt*x2dot;
        ydot = (K*u_k - y(k))/Tp;
        y(k+1) = y(k) + dt*ydot;
    end
end
```

Listing 20: Problem 3b – LADRC design and the effect of reference differentiation, parts I to IV

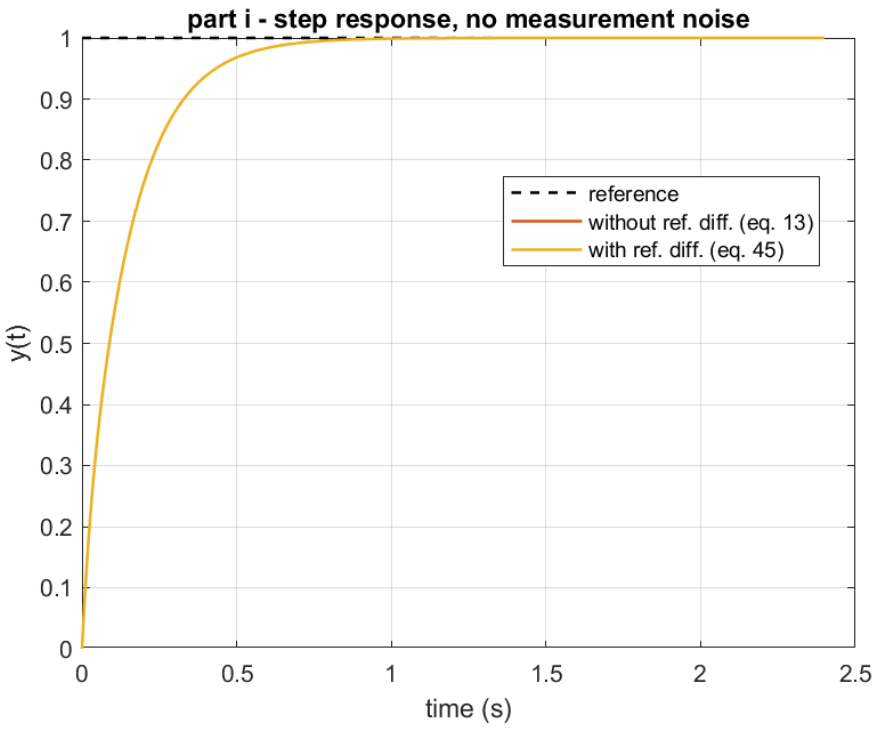


Figure 45: Part I: unit step response, no measurement noise.

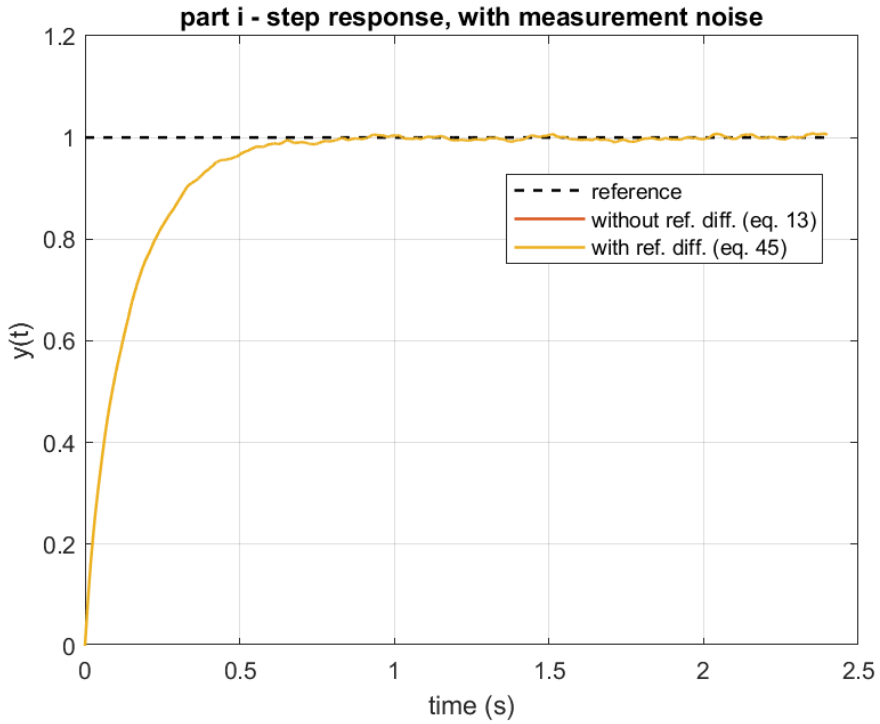


Figure 46: Part I: unit step response, with measurement noise.

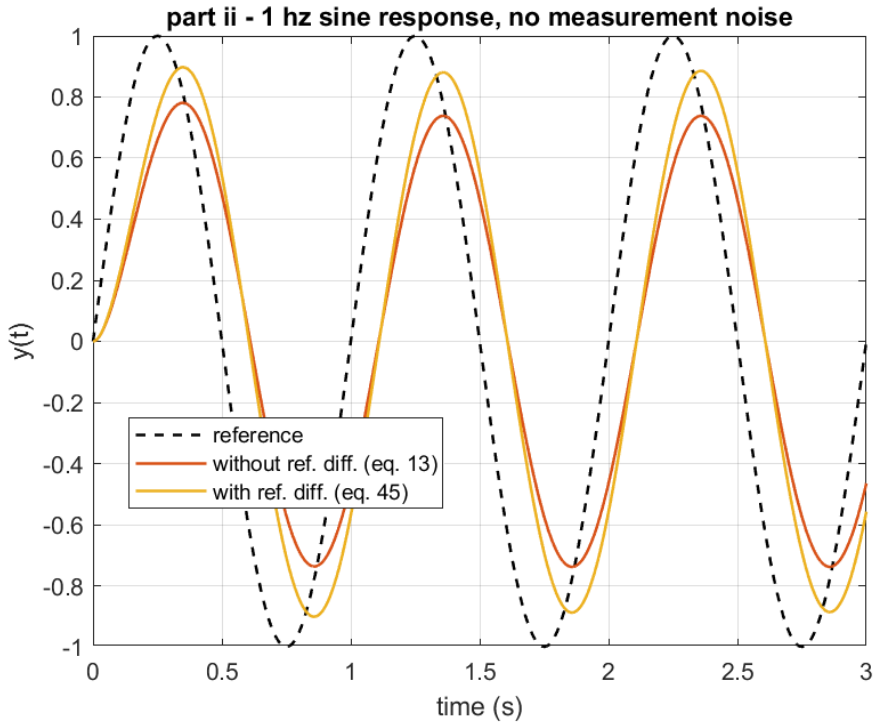


Figure 47: Part II: 1 Hz sinusoidal response, no measurement noise.

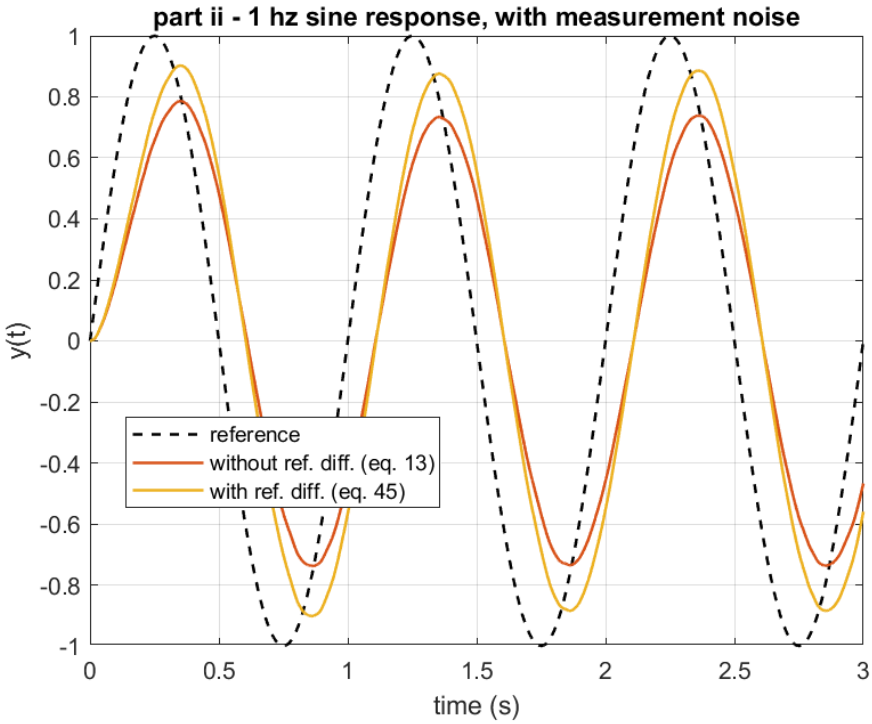


Figure 48: Part II: 1 Hz sinusoidal response, with measurement noise.

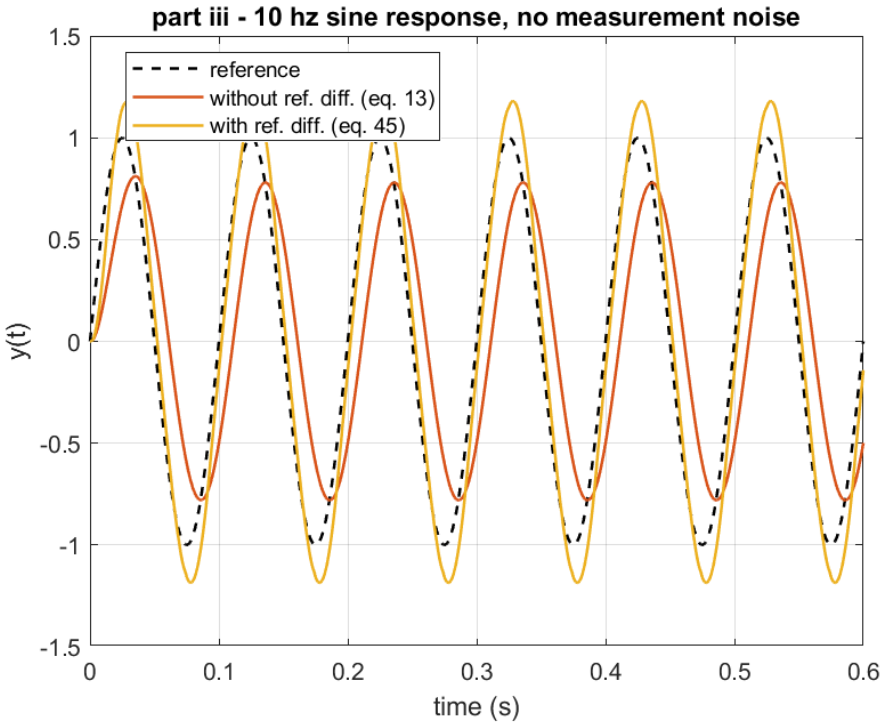


Figure 49: Part III: 10 Hz sinusoidal response, no measurement noise.

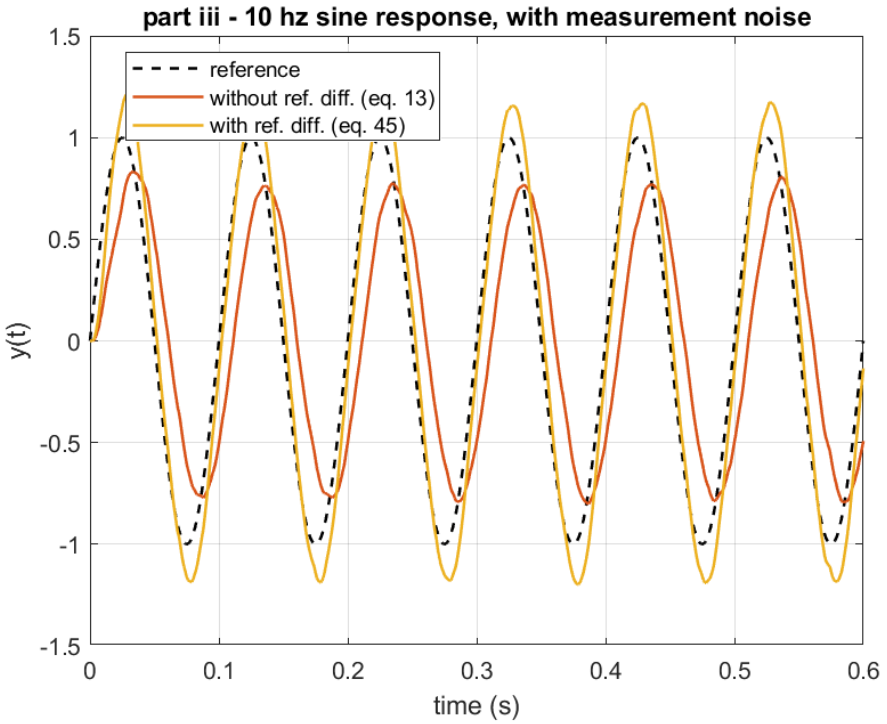


Figure 50: Part III: 10 Hz sinusoidal response, with measurement noise.

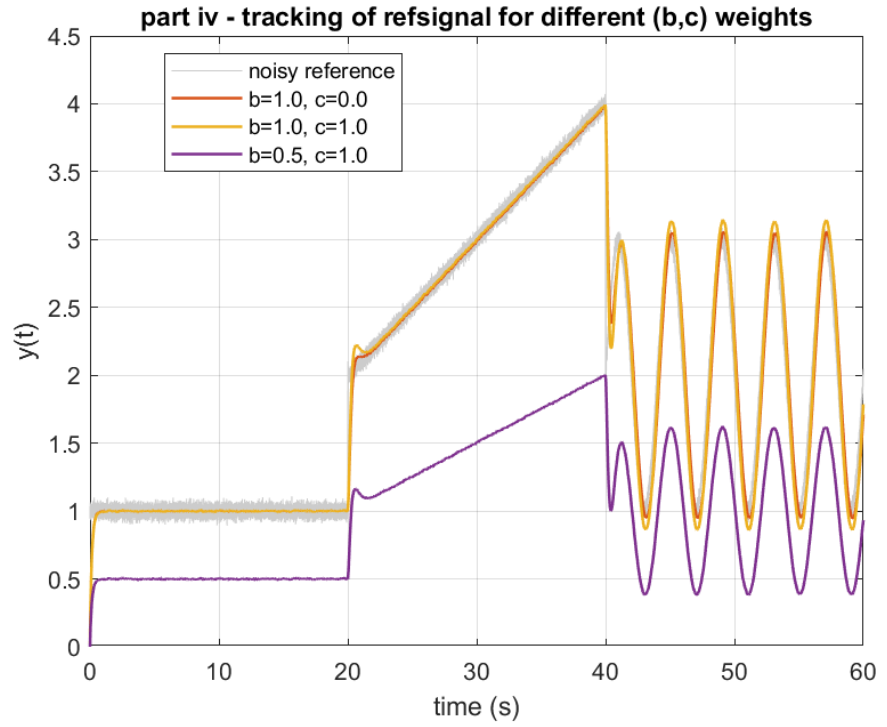


Figure 51: Part IV: tracking of RefSignal for different (b, c) setpoint-weighting pairs.

Discussion of Figures 44 to 49 (parts I to III):

- **Part I (step):** The with- and without-differentiation curves are visually identical in both Fig. 44 and Fig. 45 – the orange (eq. 13) trace is fully hidden beneath the yellow (eq. 45) one. This is expected, not a bug: for a true step, $\dot{v}(t) = 0$ for all $t > 0$, so the LD's derivative estimate $v_2(t)$ converges to (and stays at) zero almost immediately, and the feedforward term $c\,\dot{v}(t)$ in eq. (45) contributes nothing. Reference differentiation simply has no effect when the reference itself has no derivative content.

- **Part II (1 Hz sine):** Here $\dot{v}(t) \neq 0$, and a small but visible improvement appears: the eq. (45) response (yellow) sits slightly closer to the reference peaks than the eq. (13) response (orange), in both the noiseless and noisy cases. Both curves still show clear amplitude attenuation and phase lag relative to the reference, since at 6.28 rad/s the closed loop itself (designed for $T_{settle} = 0.3$s, i.e. $K_P \approx 13.3$ rad/s bandwidth) is already being pushed.

- **Part III (10 Hz sine):** With the closed loop retuned faster ($T_{settle,III} = 0.05$s) and the differentiator switched to TOD, the benefit of reference differentiation becomes much more pronounced: the eq. (45) response now tracks the reference amplitude closely (even slightly overshooting), while the eq. (13) response remains visibly attenuated, in both the noiseless and noisy cases. Measurement noise barely changes either curve at this noise level, confirming that the dominant limitation at 10 Hz is bandwidth/lag, not noise sensitivity.

- **Across I to III:** the benefit of reference differentiation grows monotonically with how fast the reference changes – absent for a constant step, modest at 1 Hz, and substantial at 10 Hz – while simultaneously the differentiation method itself becomes the bottleneck at high frequency, motivating the LD→TOD switch for part III.

Discussion of Figure 50 (part IV):

- The $b = 1, c = 0$ (orange) and $b = 1, c = 1$ (yellow) curves overlap almost completely, mirroring the part I finding: RefSignal's derivative is small over most of the 40 s window (flat segments, a gentle ramp), so the feedforward weight c has little to act on here and contributes only a minor visible difference during the final, faster-varying sinusoidal segment.

- The $b = 0.5, c = 1$ (purple) curve tells a very different story: it tracks at roughly half the amplitude/level of the actual reference throughout. This is because b scales the reference *inside* the proportional term ($K_P(b\,v_1 - \hat{x}_1)$), so the controller is steered toward $0.5\,v_1(t)$ rather than $v_1(t)$ itself – b is therefore the dominant knob for steady-state tracking accuracy, while c mainly affects how well fast transients and oscillations are followed.

Overall conclusion. The simulations confirm the main motivation behind introducing reference differentiation into the first-order LADRC structure. When the reference changes slowly, the additional feedforward term contributes little because the reference derivative remains close to zero. Consequently, the modified controller of eq. (45) behaves almost identically to the original LADRC law of eq. (13), as observed for the unit-step input.

As the reference frequency increases, the derivative feedforward becomes increasingly beneficial. For the 1 Hz sinusoidal reference, reference differentiation reduces tracking lag and slightly improves amplitude reproduction. For the 10 Hz sinusoidal reference, the improvement becomes substantial, provided that the differentiator itself possesses sufficient bandwidth. The linear differentiator used in parts I and II cannot

supply an accurate derivative estimate at 10 Hz, making the nonlinear TOD differentiator necessary for part III.

The results of part IV further separate the roles of the setpoint-weighting parameters. The coefficient b directly affects the reference level seen by the proportional action and therefore has a strong influence on steady-state tracking. In contrast, the coefficient c acts only on the derivative feedforward path and mainly influences transient tracking of rapidly changing references. For slowly varying signals, changing c has little visible effect, whereas reducing b immediately changes the achieved tracking level.

Therefore, for the DC-motor plant considered here, reference differentiation offers little benefit for constant references, moderate benefit for low-frequency sinusoidal references, and significant benefit for high-frequency tracking tasks. The effectiveness of the approach depends not only on the LADRC tuning parameters T_{settle} and k_{ob}, but also on the bandwidth and quality of the differentiator used to generate the reference derivative.